Where 4-Bit Quantization Breaks Small Tool-Calling Agents, and Which Repairs Fix It: A Controlled Testbed on llama.cpp

Project Workbook 2: Project Design and Early Implementation

By
Ekant Kapgate
Vritika Malhotra
Yashashav Devalapalli Kamalraj

October 29, 2026 (Workbook 2; Version 1 submitted September 30, 2026; Version 3 presented October 16, 2026)

Advisor: Dr. Bertin Cordova-Diba

TEAM MEMBER CONTRIBUTIONS

Table 1 lists each member's project ownership (see Chapter 9) and the workbook sections they led.

Table 1. Individual contributions to the project and to this workbook.

Member	Primary project ownership	Workbook sections led
Ekant Kapgate	Episode Runner (C2), Inference Backends (C3), Step Logger (C5), grammar-constrained repair (R1), Cost Profiler (C9)	Ch. 1 (constrained decoding, State-of-the-Art Summary), Ch. 5, Ch. 8
Vritika Malhotra	Benchmark Adapter (C1), Simulated-User Cache (C4), Failure Classifier (C6), analysis plan and statistics (C10)	Ch. 1 (agent benchmarks, evaluation methodology), Ch. 3, Ch. 7
Yashashav Devalapalli Kamalraj	Added-bits repair (R2), QAD Training Pipeline and control (R3, C8), model conversion, Experiment Config & Registry (C11) including the Slurm job layer	Ch. 1 (quantization, mixed precision, QAT/QAD), Ch. 2, Ch. 4, Ch. 6, Ch. 9

REVISION NOTES FOR VERSIONS 2 AND 3

Version 1 of this workbook (submitted 2026-09-30) was written before any experiment ran. Between 2026-10-01 and 2026-10-16 the team ran the first technology spikes, a diagnostic deep dive, and a full rerun under one pinned protocol, and presented the results on 2026-10-09. Version 2 adds what we measured and learned. It changes only the passages those results affect and leaves the rest of Version 1 as submitted. All numbers added here are pilot, or development, data: they test the tooling and replace planning assumptions, and they are excluded from the pre-registered study. Version 3, prepared for the advisor review on 2026-10-16, adds the plan from 2026-10-16 to milestone M2: per-member tasks, the cluster onboarding steps, the decisions we ask the advisor to confirm, and the matching schedule updates in Chapter 9.

Workbook 2 (submitted 2026-10-29) builds on Version 3 and records milestone M1. Chapter 8 adds the walking skeleton's results (Table 36a: 252 of 253 planned episodes on Colab, every step logged and classified), the step-by-step reproduction commands, the CoE HPC measurements and first cluster runs (Table 33 note, Tables 37 and 37b), and the decisions for the advisor; Chapter 9 adds the PERT network with the critical path (Figure 6a, Table 39a) and updates the Gantt chart, milestone status, sprint plan and risks. As the course asks, changes in Chapters 8 and 9 are in bold. Table 20a, at the start of Chapter 6, shows where each Workbook 2 rubric item is answered; the M1 results are development data, like the pilot. It also answers the six points of the 2026-10-16 advisor review, each in bold where it lands: dispersion on every bar graph (Figure 5a, notes under Tables 35a and 36a, Chapter 7), the mixed-effects analysis made a registered analysis (Chapters 3 and 7), Chapter 5 renamed System and Infrastructure Architecture with the cluster deployment (Figure 1a), the split between CMPE 295A and 295B (Table 38a, Figure 6a), the architecture changes the HPC forced (Table 19a), and a vision-language extension as an optional 295B layer (FR-O6).

Table 1a. What changed from Version 1 to Versions 2 and 3 and to Workbook 2.

Where	Change	Why
Title page; this section	Version and date	Midway update
Ch. 3, Reference and Quantized Baselines; Table 11	Q4_K_M measured at 5.15 bits per weight on Qwen3-0.6B; Qwen's official Qwen3-0.6B GGUF repository ships only Q8_0	Measured in spike S1 and checked against the publisher's repository during the pilot
Ch. 4, Table 16 (DEP-4, DEP-6)	Cluster-access status; prompt caching added to the pinned server settings	Access is still being activated; the pilot showed prompt caching changes greedy outputs
Ch. 8, opening; Tables 32, 35, 37	Status as of 2026-10-16	Spikes S1 to S3, single-turn S4, S6 and S7 ran on Colab as the DEP-4 fallback
Ch. 8, new section "Pilot Results, October 2026" (Tables 35a and 35b, Figure 5a)	Results, diagnostics, and the design consequences of the pilot	Replaces planning assumptions with measurements
Ch. 8, Progress Tracking	Advisor review moved to 2026-10-16	Rescheduled with the advisor (confirmed 2026-10-09)
Ch. 8, new section "Plan from October 16 to M2" (Tables 37a, 37b) — Version 3	Per-member plan to M1 and M2; cluster onboarding steps; decisions requested from the advisor	Forward plan for the advisor review
Ch. 9, Milestones, Table 39, Schedule Risks — Version 3	Milestone status; Sprint 4 and 5 rows updated with pilot follow-ups; cluster-activation risk	Keep the schedule consistent with the plan
Ch. 3, 5, 7, 8, 9 — Workbook 2	**Responses to the 2026-10-16 advisor review: dispersion on every bar graph, mixed-effects analysis registered, Chapter 5 retitled with the infrastructure view (Figure 1a, Table 19a), 295A/295B split (Table 38a), vision-language extension (FR-O6)**	**Advisor review, 2026-10-16**
Ch. 8, 9 — Workbook 2	**M1 results (Table 36a), reproduction steps, CoE HPC measurements and first cluster runs, decisions for the advisor; PERT network (Figure 6a, Table 39a) and refreshed Gantt**	**Milestone M1 (2026-10-26); Workbook 2 rubric**

Chapter 1. Literature Search, State of the Art

Literature Search

This review covers weight quantization of large language models (LLMs) and the methods that repair the damage it causes, focusing on 0.5B to 4B models that run as tool-calling agents in the GGUF formats of llama.cpp, the runtime behind Ollama and LM Studio. We searched the ACM Digital Library, IEEE Xplore, the ACL Anthology, arXiv, and the NeurIPS, ICML, ICLR, and MLSys proceedings, and added the technical reports, model releases, and runtime documentation through which industry ships quantized models [1, 2]. We cover the formats small models ship in, how quantization damage is measured, the three repair families (adding bits, retraining, and grammar-constrained decoding [3, 4]), tool-calling agents and their benchmarks, and evaluation methodology.

Post-Training Weight Quantization and On-Device Formats

Post-training quantization (PTQ) methods for LLMs follow two broad strategies. The first protects the few weights that dominate the error: LLM.int8() keeps a small set of emergent outlier feature dimensions in 16-bit while more than 99.9% of values are multiplied in 8-bit [5], AWQ uses activation statistics to find and rescale the roughly 1% of salient weight channels before rounding [6], and SpQR stores outlier weights at higher precision in a sparse side structure [7]. The second reduces the error of rounding everything: GPTQ quantizes one column at a time and uses approximate second-order information to compensate the remaining weights [8], and QuIP# makes weights incoherent with a randomized Hadamard transform before rounding them to lattice codebooks [9]. These methods were developed and mostly evaluated on models of 7B parameters and larger, using perplexity and zero-shot accuracy, and the SpQR authors note that 3 to 4-bit quantization usually causes moderate-to-high accuracy loss, especially in the 1 to 10B range suited to edge devices [7].

The formats small models actually ship in are simpler. llama.cpp stores models as GGUF files [1]. Its legacy `Q4_0` type rounds each block of 32 weights to 4 bits with one shared scale; its k-quant types group blocks into 256-weight super-blocks with quantized scales, giving 4.5 bits per weight for `Q4_K`; the widely used `Q4_K_M` recipe, as originally defined, also stores half of the attention value and feed-forward down-projection tensors in 6-bit `Q6_K`; and an importance matrix computed from activations on calibration text can weight the rounding error [1]. MLX quantizes weights affinely with a scale and bias per group, defaulting to 4 bits and groups of 64, and also supports the MXFP4 and NVFP4 microscaling modes [10]. Those two floating-point formats come from data-center hardware: both store 4-bit E2M1 values, but MXFP4 shares a power-of-two E8M0 scale across 32 values, whereas NVFP4 uses an FP8 E4M3 scale per 16 values plus a per-tensor FP32 scale [11]. By our arithmetic, Q4_0 and default 4-bit MLX both cost about 4.5 bits per weight, so “a 4-bit model” on-device usually means block-wise round-to-nearest quantization at about 4.5 bits, optionally guided by calibration statistics [1, 10].

Model vendors now publish their own 4-bit builds instead of leaving quantization to users. Google released Gemma 3 checkpoints trained for the `Q4_0` format used by llama.cpp, Ollama, and MLX [12]; Liquid AI released distillation-trained `Q4_0` files as

updates to its PTQ builds of LFM2.5 models from 230M to 2.6B parameters [2]; NVIDIA publishes NVFP4 checkpoints recovered by distillation [13]; and Apple compresses its roughly 3B-parameter on-device model to 2 bits per weight [14]. Because the same "4-bit" label covers different block sizes, scale types, and mixed-precision recipes, our Inference Backends (C3) treat the format itself (`Q4_0`, `Q4_K_M`, `IQ4_XS`) and the code path (CUDA, CPU) as experimental factors.

Measuring Quantization Damage Beyond Aggregate Scores

The common view that 4-bit weight quantization is nearly free rests mainly on aggregate benchmark scores. The largest such study, with over 500,000 evaluations of the Llama-3.1 family, found 4-bit weight-only quantization competitive with 8-bit formats [15], and a study of 11 model families from 125M to 180B parameters found 4-bit weight-only quantization to cost most models less than 2% [16]. Several peer-reviewed studies show why such averages mislead. Dutta et al. count *flips*, answers that change from correct to incorrect or the reverse: with accuracy within 0 to 2% of the 16-bit model, most quantization schemes flipped up to 13.6% of answers, and the authors propose flips and KL divergence as distance metrics to report alongside accuracy [17]. Jaiswal et al. show that perplexity misses the losses compressed models suffer on knowledge-intensive tasks [18], and Li et al. find that multiple-choice accuracy can rise under quantization through borderline cases and that multi-step reasoning tolerates quantization less than instruction following [16]. Distance metrics also reveal what training-based repairs do: NVIDIA reports a quantization-aware-trained model whose cross-entropy matches its BF16 teacher while its KL divergence from that teacher is 0.311, against 0.004 after distillation [13].

Damage grows as models shrink. Tolerance to weight-only quantization increases with model size [16], larger models flip fewer answers although a 70B model still flips 3 to 8% at 4 bits [17], and NVIDIA observes that models with hundreds of billions of parameters survive NVFP4 PTQ without training [13]. At the small end, under llama.cpp's `Q4_K_M`, Qwen3-0.6B without its thinking mode falls from 62.3% to 16.8% on BFCL's non-live function-calling split in the authors' instruction-following setup, while Qwen3-1.7B in the same mode loses only 2.3 points [19].

Evidence on agents is recent. ACBench, covering 15 models, found that 4-bit GPTQ and AWQ cost only 1 to 3% on workflow generation and tool use but 10 to 15% on real-world application tasks [20]. On τ^2-bench, for 9 to 35B models served with AWQ on vLLM, 4-bit quantization leaves the task score flat while amplifying the failure a model already makes by up to 2.5×, because the benchmark's error budget absorbs the extra failures [21]; whether the same masking occurs at 0.5 to 4B in GGUF formats on llama.cpp's CUDA and CPU code paths has not been measured. Two studies since then look at tool-use decisions under quantization without repairs: Wu et al. measure the decision margins that low-bit quantization flips across 16 models [22], and a small laptop-scale benchmark of Qwen3 at Q4_K_M on single-turn BFCL reports that a GBNF grammar fixed structural validity but not argument correctness while adding 6 to 86% latency [23]. Li et al. find that the per-step KL divergence between a 4-bit and a BF16 agent is low on most steps, with a small high-divergence mode of rare but consequential

steps [24]. Inside the network, Hu and Ramachandran find damage to be diffuse: measured by restoring layers to 8 bits, 8 of 9 models need roughly half their layers restored to recover 75% of the accuracy gap, and cheap signals that correlate with damage do not identify where restored precision helps [25]. Together these results argue for measuring damage per step and per failure type against a full-precision reference, which is what our Step Logger (C5) and Failure Classifier (C6) do.

Adding Bits: Mixed-Precision and Adaptive Bit Allocation

The oldest repair gives the sensitive parts of a model more bits. HAWQ assigns per-layer precision from each layer's Hessian spectrum [26]; SqueezeLLM uses second-order sensitivity to place non-uniform quantization levels and keeps outliers and sensitive weights in a sparse full-precision matrix [27]; LLM.int8() and SpQR likewise hold outliers at higher precision [5, 7]. llama.cpp's `Q4_K_M` recipe and importance-matrix quantization are the deployed, heuristic form of the same idea [1]. All of these choose *where in the network* to spend bits, and all were validated with perplexity or aggregate accuracy.

Hu and Ramachandran test that premise causally on nine open models from 0.5B to 8B parameters, including Qwen3-0.6B and Qwen3-1.7B [25]. Starting from per-row 4-bit round-to-nearest quantization, they spend an extra 0.146 bits per weight either globally, by moving every layer to group-128 scales, or locally, by restoring the most recoverable whole layers to 8 bits. Global granularity recovered 21 to 52 percentage points more of the accuracy lost to 4-bit quantization than the best local repair, even with the local layers chosen by an oracle, on all eight models where it applied, including Qwen3-8B, the one model whose damage was concentrated [25]. Two caveats limit the transfer to our setting. Damage was measured by continuation scoring on 22 knowledge and reasoning tasks, with no generation or agent tasks [25]. On-device formats already use groups of 32 to 64 weights, so the next global step on llama.cpp is a higher-bit format everywhere, such as `Q5_0` or `Q6_K`, rather than finer scales [1]. Unsloth's Dynamic GGUF builds are the deployed form of the layer-selective idea: they assign quantization types per tensor from calibration error and, by their authors' statement, use no quantization-aware training [28].

A second line of work varies precision over time instead of over layers. Any-Precision LLM overlays models of 3 to n bits in the memory of one n-bit model so that precision can be chosen per request [29]. Progressive Mixed-Precision Decoding runs the prefill at higher precision and lowers precision deeper into the generated sequence [30]. MoBiQuant adds a router that picks each token's precision from nested residual quantization and reports up to 1.34× the throughput of earlier any-precision methods [31]. Closest to agents, Dynamic Mixed-Precision Routing trains a router that sends each step of a multi-step episode to a BF16 or a 4-bit copy of Qwen3: with a budget of 40% of steps at full precision, Qwen3-4B with AWQ reached 95.1% success on ALFWorld (BF16 94.3%, all 4-bit 85.4%) and 61.7% on WebShop (BF16 65.2%, all 4-bit 56.4%) [24].

Step-selective precision is therefore possible, but the question our R2 arm asks remains untested: does spending extra bits on selected conversation steps recover more agent failures than spreading them uniformly or placing them in selected layers, and at what

measured memory and token cost? No study compares the three allocations on the same models. The step-routing study switches between two separate models, so both must be available at every step; it models cost as latency on a data-center GPU, reports no memory figures, and uses household and shopping environments rather than tool-calling benchmarks [24]. The nested formats that would avoid a second copy rely on their own kernels and are not among the documented GGUF quantization types [29, 31]. On a small device, a second copy costs the very memory quantization was meant to save, so our Cost Profiler (C9) measures it and charges it to the repair.

Retraining: Quantization-Aware Training and Distillation

Retraining lets the weights adapt to the quantization grid. LLM-QAT brought quantization-aware training (QAT) to LLMs with a data-free recipe that distills from the full-precision model's own generations, reaching 4-bit weights, activations, and KV cache on 7B to 30B models [32]. BitDistiller combines QAT with a confidence-aware KL self-distillation objective for 2 and 3-bit models [33]; EfficientQAT trains block by block and then only the quantization step sizes, producing a 2-bit Llama-2-70B on one A100 in 41 hours at 69.48 versus 72.41 average accuracy [34]; and ParetoQ finds a transition between 2 and 3 bits, above which fine-tuned models stay close to their pre-trained distributions [35]. Parameter-efficient variants freeze the quantized base and train low-rank adapters, as in QLoRA's 4-bit NormalFloat base [36] and Apple's quality-recovery adapters on its 2-bit on-device model [14]. Most of this work targets 2 to 3 bits on models of 7B parameters and more, where PTQ breaks down; repairing 4-bit agents below 4B parameters is much less studied.

Quantization-aware distillation (QAD) has become the industrial form of this idea. NVIDIA trains NVFP4 students on the KL divergence to their BF16 teacher and shows that plain QAT damages models post-trained with reinforcement learning: on AIME25, AceReason-Nemotron-1.1-7B scored 63.5 in BF16, 58.7 after PTQ, 46.1 after QAT, and 62.0 after QAD, with 0.3 to 6 billion training tokens depending on the model [13]. Google trained Gemma 3 for about 5,000 steps with the unquantized checkpoint's probabilities as targets and reports a 54% smaller perplexity drop at `Q4_0` [12]. Liquid AI's QAD closes 48.4% to 73.4% of the BF16-to-`Q4_0` gap for four LFM2.5 models, which retain 96.5% to 97.4% of BF16 performance [2]. Multiverse Computing's quantization-aware healing distills a structurally compressed MXFP4 student directly from the compressed BF16 model; its released 60B model matches or beats that BF16 source on 7 of 9 benchmarks, and in a 20B-to-9B experiment the method reaches a comparable peak about 7× faster than cross-entropy QAT [37]. Google now ships QAT Q4_0 GGUFs for Gemma 4, including the 2B-effective E2B model [38], and a mixed-precision QAD recipe has been demonstrated on Qwen3-0.6B, though without agent tasks [39].

Three gaps matter for agents. First, every evaluation above reports aggregate scores. Liquid AI reports a seven-task average in which BFCLv4 is one component [2], NVIDIA and Google report reasoning benchmarks and perplexity [13, 12], and the single agent result, 61.7 versus 59.4 on τ^2-bench [37], uses the pass rate that can stay flat while failures grow [21]. NVIDIA also picks, among the ten checkpoints with the lowest

validation loss, the one with the best average on the evaluation benchmarks themselves [13]. Second, none of these evaluations includes the control that separates retraining under quantization from simply training more. NVIDIA describes QAT as "effectively acting as an additional post-training stage" [13], and Multiverse attributes its 4-bit model's gains over BF16 to a second distillation pass that the BF16 checkpoint never received [37]. The control matters because in pretraining the loss-optimal split of compute between full-precision training and QAT shifts with the total budget [40], so a BF16 model distilled for the same steps and then quantized is a strong baseline. Third, QAD keeps the format and therefore the inference cost, but its training cost for small models is not reported in usable terms: Liquid AI gives no training details and Google gives only a step count [2, 12]. Our QAD Training Pipeline (C8) therefore runs each QAD configuration next to a train-longer control at full precision. Table 2 summarizes the three families.

Table 2. The three repair families for quantized small-model agents, as evaluated in the literature.

Repair family	Representative methods	What is changed	Inference cost	How it has been evaluated	Gap for agents
R1 Restrict output	Grammar-constrained decoding [3, 4]; input-side schema compilation [41]	Decoder: invalid tokens masked	No extra weights; per-token masking overhead	Tool-call accuracy	Cannot fix wrong-but-valid calls; never compared with R2 or R3
R2 Add bits	`Q4_K_M`, SpQR, SqueezeLLM; any-precision, MoBiQuant, step routing [25, 24]	Precision of some weights, per layer, token, or step	More memory; step routing may keep a second model copy	Perplexity, continuation scoring, ALFWorld and WebShop success	Step vs. layer vs. uniform allocation untested on tool-calling tasks with measured cost
R3 Retrain	QAT, QAD (NVIDIA, Google, Liquid AI, Multiverse) [13, 2]	Weight values within the same format	None at inference; training compute	Aggregate scores; τ^2-bench pass rate	No train-longer control; failures by type unmeasured

Small Language Models as Tool-Calling Agents

Toolformer showed that a model can teach itself when to call an API and with which arguments [42]. Gorilla fine-tuned a model to write API calls and reduced hallucinated API usage [43], and ToolLLM scaled training and evaluation to more than 16,000 real APIs [44]. ReAct established the loop these agents still run: reason, act, observe, repeat [45]. Each step is a model output that must parse into a valid call and then changes the environment, so one invented or malformed call alters every later step.

Running this loop on the user's device is now a design goal. TinyAgent fine-tuned 1.1B and 7B models for function calling, added tool retrieval and quantization, and ran the agent locally on a MacBook [46]. Paramanayakam et al. showed that shrinking the tool list offered to an edge model improves success, latency, and power [47]. The three families we study are released for this setting. Qwen3 ships dense models from 0.6B parameters upward, with switchable thinking and non-thinking modes [48]. LFM2's

hybrid convolution-attention backbone was found by hardware-in-the-loop search for edge devices [49], and LFM2.5-1.2B is presented as a building block for on-device agents, with GGUF and MLX checkpoints at release [50]. SmolLM3 is a 3B model with tool calling and a dual reasoning mode [51]. At 16 bits they strain phone and laptop memory: Taheri et al. report 8.0 GB for Qwen3-4B at FP16 against 2.5 GB at llama.cpp's Q4_K_M format [19], so in practice they ship at about 4 bits.

The evidence on what 4 bits do to tool calling, reviewed earlier in this chapter, points in different directions: ACBench finds tool use almost unaffected by 4-bit GPTQ and AWQ [20], while Qwen3-0.6B in non-thinking mode loses most of its BFCL V3 non-live accuracy (62.3% to 16.8%) under Q4_K_M [19]. An edge study adds that the format matters: on BFCL, Llama-3.1-8B served through Ollama fell from 63.0% at full precision to 20.4% at q4_0 and 39.6% at q4_K_M [47]. The studies differ in model size, format, reasoning mode, and prompting, and all report single-turn or aggregate scores, so none shows where in a multi-turn conversation the damage lands.

Benchmarks and Failure Analysis for Tool-Calling Agents

Tool-use benchmarks have moved from single calls to stateful conversations. BFCL scores individual calls by abstract-syntax-tree matching and adds stateful multi-step categories [52]. Its V3 multi-turn set (base, missing parameters, missing functions, long context) checks the environment state and the response after every turn, returns execution errors to the model, and cuts off a turn after 20 steps [53]. τ-bench added an LLM-simulated user and a domain policy, graded the final database state, and introduced pass^k, the probability that all k independent trials succeed. With it, GPT-4o solved fewer than 50% of tasks and had pass^8 below 25% in retail [54]. τ²-bench adds a dual-control telecom domain in which the simulated user also acts on the shared state [55]. Amazon AGI's audit of that benchmark, the SABER paper, released corrected airline and retail tasks that its authors call τ-Bench Verified [56]; Amazon's repository extends the corrections to the telecom domain under the name τ²-Bench Verified [57]. Sierra has since folded most of the airline and retail fixes into its official τ³-bench release, v1.0.1 [58], so one benchmark name now covers several task versions and the version must be pinned. Table 3 compares the benchmarks we considered.

Table 3. Agent benchmarks compared on the properties that matter for measuring quantization damage.

Benchmark	Multi-turn	Simulated user	Main metric	Error tolerance	Used by us
BFCL single-turn [52]	No	No	AST or execution match	None; one call scored	Candidate easier set
BFCL V3 multi-turn [53]	Yes	No (scripted turns)	State and response checks per turn	Errors returned; 20-step cap per turn	Yes
τ-bench [54]	Yes	Yes (LLM)	Final database state; pass^k	Errors returned; step cap	No
τ²-bench [55]	Yes	Yes (LLM, can act)	Final state and assertions; pass^k	Ends after 10 failed calls	Through Verified
τ²-Bench	Yes	Yes (LLM,	As τ²-bench,	As τ²-bench	Yes

Benchmark	Multi-turn	Simulated user	Main metric	Error tolerance	Used by us
Verified [56]		can act)	corrected tasks		
API-Bank [59]	Yes (annotated dialogues)	No	API-call accuracy; response ROUGE-L	None; each call scored	No
ToolBench [44, 60]	Multi-step, one query	No	LLM-judged pass and win rate	Not applicable	No
AgentBench [61]	Yes (environment turns)	No	Per-environment score	Varies by environment	No

The error-tolerance column is where quantization damage hides: τ²-bench ends an episode only after ten failed tool calls, and the agent can recover between failures [21]. Jang et al. ran two open-weight families at 16, 8, and 4 bits on two τ²-bench domains (456 episodes per cell). No change in final score survived multiple-comparison correction [21]. Underneath, 4-bit weights amplified failures the models already made at full precision, and only in models that made them. In the most affected cell (Gemma-4-31B, telecom), the share of tool calls naming a nonexistent tool rose from 19.5% to 38.3% (about 2.0×). Each tool name the 16-bit model had already invented was called about 2.5× more often, and only 0.18% of the 4-bit model's hallucinated calls named a tool the 16-bit model had never used [21]. Shrinking the budget from ten errors to two re-exposed a 17-point score gap, but only in the cell that had gained errors, and in that cell the 4-bit to 16-bit hallucination-rate ratio stayed between 1.3× and 2.0× at every turn depth [21]. This matches a general result for compressed models: matching accuracy can hide many answers that flip between correct and incorrect [17]. The study's scope leaves room for ours: 9B to 35B (total) parameter models, AWQ weights served by vLLM on A100 GPUs, one benchmark, and one prompt-based repair [21]. Small models in the GGUF formats they ship in have not been measured this way.

Failure taxonomies show why one score is not enough. MAST annotated more than 1,600 multi-agent traces into 14 failure modes with high inter-annotator agreement ($\kappa = 0.88$) and then scaled the annotation with an LLM judge [62]. Xu et al. split tool hallucination into tool-selection and tool-usage errors [63]. SABER found that deviations at state-changing ("mutating") steps decide task success, while deviations at read-only steps barely matter [56]. Failure types differ in consequence, and a pass/fail score merges them. Our Failure Classifier therefore labels each failed step with one of the ten classes defined in Chapter 6 (hallucinated tool, malformed call, schema-invalid arguments, wrong tool, wrong argument value, premature termination, repetition loop, policy violation, user-facing response error, other) and records whether the step was mutating.

Restricting Output: Grammar-Constrained Decoding

Grammar-constrained decoding masks, at each step, every token that would violate a formal grammar, so the output is valid by construction. Willard and Louf compiled regular expressions and grammars into a finite-state index over the vocabulary, the basis of Outlines [64]. Geng et al. showed that input-dependent grammars let a model solve structured tasks without fine-tuning [3]. XGrammar prechecks tokens whose validity

does not depend on context and interprets only the rest at runtime. It reports up to 100× faster mask generation and near-zero end-to-end overhead when co-designed with the serving engine [4]. llama.cpp exposes the same mechanism through GBNF grammars and converts JSON schemas to grammars in its server [1].

Masking is cheap in the best engines but not everywhere. In XGrammar's evaluation, llama.cpp's grammar engine was among the baselines slowed by longer preprocessing and per-token processing [4]. JSONSchemaBench tested six frameworks, including llama.cpp and XGrammar, on 10,000 real-world schemas and found that they differ in efficiency, in which schema features they support, and in output quality [65]. Constraints also change what the model says. Park et al. showed that greedy masking distorts the model's distribution, producing outputs that are grammatical but improbable under the model [66]. Tam et al. found that strict format restrictions degrade reasoning, and that stricter formats degrade it more [67].

A grammar fixes a narrow set of failures. One that lists the available tool names makes a call to a nonexistent tool impossible, and so it removes by construction the failure that Jang et al. found amplified [21]. It also guarantees well-typed arguments; Apple's on-device framework uses this to prevent hallucinated tool names and arguments [14]. It cannot choose the right tool among valid ones, supply the right record identifier, decide when to stop, or follow a policy. It may also turn a loud failure, a "tool not found" error that the agent can recover from, into a silent one: a valid but wrong call, the kind SABER identifies as decisive at mutating steps [56]. Lee decomposes the effect on unquantized Qwen3 models from 0.6B to 4B and finds that the grammar repairs form, restoring unreadable outputs, without improving the decision itself [68]. Whether grammars remove quantization damage or only relabel it is a question for RQ3. TSCG works on the input side instead. It deterministically compiles JSON tool schemas into compact structured text, attributes most small-model tool-use failures at production catalog sizes to the JSON format itself, and lifts Phi-4 14B from 0% to 84.4% accuracy at 20 tools while saving 52 to 57% of schema tokens [41]. Neither TSCG nor any grammar engine above was evaluated on quantized agents.

Evaluation Methodology: Variance, Statistical Testing, and Pre-Registration

Agent benchmarks are small and noisy. A τ^2-bench domain has 50 to 114 tasks with binary outcomes, and repeated trials of a task are correlated [21]. Miller treats evaluations as experiments: report standard errors, cluster them when items are grouped (clustered errors can exceed three times the naive ones), compare models on per-question paired differences, and size the study by power analysis [69]. Dror et al. give a protocol for choosing significance tests in NLP: McNemar's test for paired binary outcomes, and paired bootstrap or permutation tests for other metrics [70]. Jang et al. apply this toolkit to quantized agents: a cluster bootstrap over tasks, paired-by-task comparisons, and two one-sided equivalence tests [71] at ±7.5 points to support a "no change" claim [21].

Two further problems apply to our design. First, crossing models, formats, repairs, benchmarks, and failure types produces hundreds of tests. We correct the confirmatory tests with Holm's procedure [72]; false-discovery-rate control [73] is the more powerful option for screening exploratory comparisons. Second, inference is not reliably

deterministic. Greedy decoding can change between runs because kernels are not batch-invariant [74], and a repeated 456-episode τ²-bench run on vLLM shifted the mean score by about 1.5 points [21]. We therefore repeat trials, record seeds, runtime versions, and hardware, and measure a noise floor for each backend instead of assuming determinism. External services add a third source of drift; StableToolBench replaced live APIs with a cache and simulators to keep scores stable [60], and our Simulated-User Cache applies the same idea to the simulated user.

Pre-registration addresses the remaining risk: choosing the analysis after seeing the data. It is established practice in the empirical sciences [75], and the NeurIPS 2020 pre-registration workshop reviewed and accepted ML protocols before any results existed [76]. The NeurIPS reproducibility program introduced a checklist covering code, data, and statistical reporting [77]. Our testbed follows both. The Step Logger stores every step, so per-failure-type rates, success under a shrinking error budget, and pass^k can all be recomputed without rerunning a model. The analysis plan (hypotheses, primary metrics, tests, correction, and the rule excluding items that the full-precision model already fails) is registered on OSF before the main runs. With three repair families and many candidate metrics, this keeps us from picking a winner by picking the metric.

Research Gap

Each repair family has been judged by a different yardstick. Output restriction is measured on syntactic validity and throughput [64, 4, 65], and its input-side relative TSCG on unquantized models [41]. Added-bits work has just shown, on continuation scoring rather than agent tasks, that choosing which layers receive extra precision works worse than spreading the bits evenly [25]. Choosing which steps get more precision has first evidence: a router that sends each ALFWorld or WebShop step to a BF16 or a 4-bit Qwen3 model [24], alongside token-adaptive any-precision kernels [31]. What remains untested is step-selective against uniform and layer-selective bits on tool-calling benchmarks, on llama.cpp's production code paths, with deployment cost counted; the router keeps two full models and reports only modelled latency [24]. Quantization-aware distillation retrains the quantized model to imitate its full-precision self and now ships from NVIDIA, Multiverse Computing, and Liquid AI [13, 37, 2]. Its published evaluations report aggregate scores, and none that we reviewed includes a control that trains for the same number of steps without quantization, which is needed to separate quantization-aware retraining from simply training longer, a confound Multiverse's own write-up acknowledges [37].

The only process-level study of quantized agents shows that aggregate scores cannot see the damage [21], yet it covers 9B to 35B models on server GPUs and tests one prompt repair. The single-turn and aggregate studies disagree on how large the damage is [20, 47, 19]. No published work compares the three repairs on the same small models and multi-turn benchmarks, tests whether a repair removes failures or only restores the score, or reports deployment costs on the runtimes these models ship on. This project addresses four research questions:

1. **RQ1:** Where does 4-bit damage concentrate in small-agent conversations, by step type and by conversation depth?

2. **RQ2:** How do grammar-constrained decoding, added bits (uniform, layer-selective, and step-selective), and QAD retraining compare, each with its control, on the same models and benchmarks?
3. **RQ3:** Do repairs validated on overall scores fix the underlying failures, or only the score?
4. **RQ4:** What does each repair cost to deploy (memory, latency, extra tokens, and energy) on llama.cpp's real 4-bit code paths, CUDA and CPU, for the same GGUF files?

State-of-the-Art Summary

On-device runtimes and formats. Small agents reach users through a few open runtimes. llama.cpp's GGUF format offers k-quants such as Q4_K_M (4.89 bits per weight on Llama-3.1-8B), i-quants from IQ1 to IQ4, and an importance matrix (imatrix) computed from calibration text to guide quantization, and its server accepts GBNF grammars or JSON schemas to constrain output [1]. On Apple silicon, MLX [10] and mlx-lm quantize and serve models, and mlx-lm's learned quantizers include DWQ, which tunes quantization scales and biases with the unquantized model as teacher, so a distillation-style repair already sits inside that runtime [78]; Unsloth's Dynamic GGUFs assign per-tensor types from calibration error without retraining [28]; and OpenAI's gpt-oss models ship with their expert weights post-trained in MXFP4 [79]. Meta ships quantized Llama 3.2 through ExecuTorch on Arm phone CPUs [80]. Desktop apps such as Ollama and LM Studio distribute these builds, both offer JSON-schema structured output, and LM Studio warns that models under 7B may fail at it [81]; vendors now name these apps as launch targets [38, 82], and Microsoft's Foundry Local, generally available since April 2026, serves tool-calling models on NPU, GPU, or CPU [83]. Apple's own on-device model shows how far this goes: about 3B parameters at 2 bits per weight, served through a framework whose guided generation prevents hallucinated tool names and arguments [14], rebuilt with improved tool calling at WWDC 2026 [84].

Vendor QAT and QAD. Retraining under quantization has moved from research papers into product releases (Table 4).

Table 4. Vendor quantization-aware training (QAT) and distillation (QAD) releases, 2024 to 2026.

Vendor	Release	Technique	Deployment target
Meta	Llama 3.2 1B/3B, Oct 2024	QAT with LoRA; SpinQuant	4-bit weights, 8-bit activations; ExecuTorch [80]
Google	Gemma 3 QAT, Apr 2025	QAT toward unquantized model's outputs	Q4_0 for llama.cpp, Ollama, MLX [12]
Apple	On-device model, 2025	2-bit QAT plus LoRA recovery adapters	Apple silicon [14]
NVIDIA	NVFP4 QAD, 2026	KL distillation from BF16 teacher	NVFP4 GPUs [13]
Google	Gemma 4 QAT, Jun 2026	QAT; Q4_0 plus a mobile format	E2B at about 1 GB, under 1 GB text-only [38]
Multiverse	Quantization-aware healing, 2026	Distillation into a compressed 4-bit model	MXFP4 [37]
Liquid AI	LFM2.5 QAD, Aug 2026	QAD for 230M to 2.6B models	GGUF Q4_0 [2]

Structured output in production. Constraining output is now a standard serving feature. OpenAI's Structured Outputs guarantees schema adherence, including for function calls in strict mode [85]. XGrammar is integrated into SGLang and MLC-LLM [4] and is a structured-output backend in vLLM [86]. llama.cpp's GBNF [1] and Outlines [64] bring the same guarantee to open models. These engines are judged on validity and speed; whether a grammar helps a quantized agent pick the right tool is outside what they measure.

Agent evaluation and on-device agents. The BFCL leaderboard has grown from single calls to stateful multi-turn and agentic categories [52, 53]. Sierra's τ-bench leaderboard added a knowledge-retrieval banking domain, where the best model reached about 25% (25.5% pass^1 in the accompanying paper [87]), and a voice track in March 2026 [58]. Both rank models by task success, the metric that Jang et al. showed can stay flat while the rate of invented tool calls doubles [21]. Google's FunctionGemma (270M parameters) targets on-device function calling, and fine-tuning raised its Mobile Actions accuracy from 58% to 85% [82]. Liquid AI positions LFM2.5-1.2B as a building block for on-device agents [50].

What industry still lacks. Vendors validate quantized releases on aggregate scores; Apple combines 2-bit QAT, recovery adapters, and constrained tool calling without separating their contributions [14], and Liquid AI reports only a seven-task average [2]. We found no vendor that reports per-failure-type agent results for its quantized builds, compares output restriction, added bits, and retraining against each other, or publishes each repair's cost on the runtime it ships with. This project supplies that measurement for small agents on llama.cpp's CUDA and CPU code paths.

References

1. Gerganov, G., & The ggml authors. (2023). *llama.cpp: LLM inference in C/C++* [Computer software]. GitHub. https://github.com/ggml-org/llama.cpp
2. Liquid AI. (2026, August 19). LFM2.5 Q4_0: Quantization-aware distillation for edge deployment. *Liquid AI Blog*. https://www.liquid.ai/blog/qad
3. Geng, S., Josifoski, M., Peyrard, M., & West, R. (2023). Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 10932–10952). Association for Computational Linguistics. https://aclanthology.org/2023.emnlp-main.674/
4. Dong, Y., Ruan, C. F., Cai, Y., Lai, R., Xu, Z., Zhao, Y., & Chen, T. (2025). XGrammar: Flexible and efficient structured generation engine for large language models. In *Proceedings of Machine Learning and Systems 7 (MLSys 2025)*. https://proceedings.mlsys.org/paper_files/paper/2025/file/5c20ca4b0b20b0bd2f1d839dc605e70f-Paper-Conference.pdf
5. Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. https://arxiv.org/abs/2208.07339
6. Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2024). AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems 6 (MLSys 2024)*. https://arxiv.org/abs/2306.00978
7. Dettmers, T., Svirschevski, R., Egiazarian, V., Kuznedelev, D., Frantar, E., Ashkboos, S., Borzunov, A., Hoefler, T., & Alistarh, D. (2024). SpQR: A sparse-quantized representation for near-lossless LLM weight compression. In *International Conference on Learning Representations (ICLR 2024)*. https://arxiv.org/abs/2306.03078
8. Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.17323
9. Tseng, A., Chee, J., Sun, Q., Kuleshov, V., & De Sa, C. (2024). QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)* (Proceedings of Machine Learning Research, Vol. 235). https://arxiv.org/abs/2402.04396
10. Hannun, A., Digani, J., Katharopoulos, A., & Collobert, R. (2023). *MLX: Efficient and flexible machine learning on Apple silicon* [Computer software]. GitHub. https://github.com/ml-explore/mlx

11. Alvarez, E., Almog, O., Chung, E., Layton, S., Stosic, D., Krashinsky, R., & Aubrey, K. (2025, June 24). Introducing NVFP4 for efficient and accurate low-precision inference. *NVIDIA Technical Blog*. https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/

12. Yvinec, E., & Culliton, P. (2025, April 18). Gemma 3 QAT models: Bringing state-of-the-art AI to consumer GPUs. *Google Developers Blog*. https://developers.googleblog.com/en/gemma-3-quantized-aware-trained-state-of-the-art-ai-to-consumer-gpus/

13. Xin, M., Priyadarshi, S., Xin, J., Kartal, B., Vavre, A., Thekkumpate, A. K., Chen, Z., Mahabaleshwarkar, A. S., Shahaf, I., Bercovich, A., Patel, K., Velury, S. V., Luo, C., Cheng, Z., Chen, J., Yu, C.-H., Ping, W., Rybakov, O., Tajbakhsh, N., . . . Mao, H. (2026). *Quantization-aware distillation for NVFP4 inference accuracy recovery* (arXiv:2601.20088). arXiv. https://doi.org/10.48550/arXiv.2601.20088 https://arxiv.org/abs/2601.20088

14. Apple. (2025). *Apple Intelligence foundation language models: Tech report 2025* (arXiv:2507.13575). arXiv. https://doi.org/10.48550/arXiv.2507.13575 https://arxiv.org/abs/2507.13575

15. Kurtic, E., Marques, A. N., Pandit, S., Kurtz, M., & Alistarh, D. (2025). "Give me BF16 or give me death"? Accuracy-performance trade-offs in LLM quantization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (ACL 2025). https://aclanthology.org/2025.acl-long.1304/

16. Li, S., Ning, X., Wang, L., Liu, T., Shi, X., Yan, S., Dai, G., Yang, H., & Wang, Y. (2024). Evaluating quantized large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)* (Proceedings of Machine Learning Research, Vol. 235, pp. 28480-28524). https://proceedings.mlr.press/v235/li24bb.html

17. Dutta, A., Krishnan, S., Kwatra, N., & Ramjee, R. (2024). Accuracy is not all you need. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. https://arxiv.org/abs/2407.09141

18. Jaiswal, A., Gan, Z., Du, X., Zhang, B., Wang, Z., & Yang, Y. (2024). Compressing LLMs: The truth is rarely pure and never simple. In *International Conference on Learning Representations (ICLR 2024)*. https://arxiv.org/abs/2310.01382

19. Taheri, Y., Heydari, M. H., Naaman, E., & Fatemi, A. (2026). *Small reasoning models are instruction followers in function calling* (arXiv:2608.22472). arXiv. https://doi.org/10.48550/arXiv.2608.22472 https://arxiv.org/abs/2608.22472

20. Dong, P., Tang, Z., Liu, X., Li, L., Chu, X., & Li, B. (2025). Can compressed LLMs truly act? An empirical evaluation of agentic capabilities in LLM compression. In *Proceedings of the 42nd International Conference on Machine*

Learning (ICML 2025) (Proceedings of Machine Learning Research, Vol. 267). https://proceedings.mlr.press/v267/dong25k.html

21. Jang, J., Yang, K., Lim, H., & Park, H. (2026). *Flat score, amplified failures: How the error budget masks damage in quantized LLM agents* (arXiv:2607.27275). arXiv. https://doi.org/10.48550/arXiv.2607.27275 https://arxiv.org/abs/2607.27275

22. Wu, Z., Dhiman, S., & Koshiyama, A. (2026). *Which decisions low-bit quantization breaks, and how to predict them* (arXiv:2608.06564). arXiv. https://arxiv.org/abs/2608.06564

23. QuantCall. (2026, July 5). *Does quantization break tool calling? I measured it on a 4 GB laptop GPU (BFCL, 3 seeds, bootstrap)*. DEV Community. https://dev.to/happynood/does-quantization-break-tool-calling-i-measured-it-on-a-4gb-laptop-gpu-bfcl-3-seeds-bootstrap-185l

24. Li, Y., Deng, J., Hu, J., Chen, T., Wang, S., & Yang, H. (2026). *Dynamic mixed-precision routing for efficient multi-step LLM interaction* (arXiv:2602.02711). arXiv. https://doi.org/10.48550/arXiv.2602.02711 https://arxiv.org/abs/2602.02711

25. Hu, J., & Ramachandran, S. (2026). *The structure of quantization damage in LLMs: Why the next bit should be spent globally* (arXiv:2609.01587). arXiv. https://doi.org/10.48550/arXiv.2609.01587 https://arxiv.org/abs/2609.01587

26. Dong, Z., Yao, Z., Gholami, A., Mahoney, M. W., & Keutzer, K. (2019). HAWQ: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2019)*. https://arxiv.org/abs/1905.03696

27. Kim, S., Hooper, C., Gholami, A., Dong, Z., Li, X., Shen, S., Mahoney, M. W., & Keutzer, K. (2024). SqueezeLLM: Dense-and-sparse quantization. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)* (Proceedings of Machine Learning Research, Vol. 235). https://arxiv.org/abs/2306.07629

28. Unsloth. (2026). *Unsloth Dynamic 2.0 GGUFs*. Unsloth documentation. Retrieved September 30, 2026, from https://unsloth.ai/docs/basics/unsloth-dynamic-2.0-ggufs

29. Park, Y., Hyun, J., Cho, S., Sim, B., & Lee, J. W. (2024). Any-precision LLM: Low-cost deployment of multiple, different-sized LLMs. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)* (Proceedings of Machine Learning Research, Vol. 235, pp. 39682-39701). https://proceedings.mlr.press/v235/park24e.html

30. Chen, H. M., Tan, F., Kouris, A., Lee, R., Fan, H., & Venieris, S. I. (2025). Progressive mixed-precision decoding for efficient LLM inference. In *International Conference on Learning Representations (ICLR 2025)*. https://arxiv.org/abs/2410.13461

31. Wang, D., Kim, J., Han, S., Gudovskiy, D., Nakata, Y., Okuno, T., Peong, K., Jeon, K. E., Ko, J. H., Chen, Y., & Yang, H. (2026). *MoBiQuant: Mixture-of-bits quantization for token-adaptive any-precision LLM* (arXiv:2602.20191). arXiv. https://doi.org/10.48550/arXiv.2602.20191 https://arxiv.org/abs/2602.20191

32. Liu, Z., Oguz, B., Zhao, C., Chang, E., Stock, P., Mehdad, Y., Shi, Y., Krishnamoorthi, R., & Chandra, V. (2024). LLM-QAT: Data-free quantization aware training for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024* (pp. 467-484). https://aclanthology.org/2024.findings-acl.26/

33. Du, D., Zhang, Y., Cao, S., Guo, J., Cao, T., Chu, X., & Xu, N. (2024). BitDistiller: Unleashing the potential of sub-4-bit LLMs via self-distillation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (ACL 2024). https://aclanthology.org/2024.acl-long.7/

34. Chen, M., Shao, W., Xu, P., Wang, J., Gao, P., Zhang, K., & Luo, P. (2025). EfficientQAT: Efficient quantization-aware training for large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 10081-10100). https://aclanthology.org/2025.acl-long.498/

35. Liu, Z., Zhao, C., Huang, H., Chen, S., Zhang, J., Zhao, J., Roy, S., Jin, L., Xiong, Y., Shi, Y., Xiao, L., Tian, Y., Soran, B., Krishnamoorthi, R., Blankevoort, T., & Chandra, V. (2025). ParetoQ: Improving scaling laws in extremely low-bit LLM quantization. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. https://arxiv.org/abs/2502.02631

36. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. https://arxiv.org/abs/2305.14314

37. Ryskulov, B., García-Ferrero, I., Montero, D., Jansen, D., Hashemi, A., Garcia, J. R., Tiene, A., & Orús, R. (2026). *Quantization-aware healing: A practical recipe for recovering compressed, 4-bit LLMs* (arXiv:2608.20953). arXiv. https://doi.org/10.48550/arXiv.2608.20953 https://arxiv.org/abs/2608.20953

38. Lacombe, O., & Sanseviero, O. (2026, June 5). *Gemma 4 QAT models: Optimizing model compression for mobile and laptop efficiency* [Blog post]. Google. https://blog.google/innovation-and-ai/technology/developers-tools/quantization-aware-training-gemma-4/

39. EdgeRazor authors. (2026). *EdgeRazor: Mixed-precision quantization-aware distillation for edge LLMs* (arXiv:2605.04062). arXiv. https://arxiv.org/abs/2605.04062

40. Dremov, A., Grangier, D., Katharopoulos, A., & Hannun, A. (2026). Compute-optimal quantization-aware training. In *International Conference on Learning Representations (ICLR 2026)*. https://arxiv.org/abs/2509.22935

41. Sakizli, F. (2026). *TSCG: Deterministic tool-schema compilation for agentic LLM deployments* (arXiv:2605.04107). arXiv. https://doi.org/10.48550/arXiv.2605.04107 https://arxiv.org/abs/2605.04107

42. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html

43. Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2024). Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. https://proceedings.neurips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html

44. Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., & Sun, M. (2024). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR 2024)*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/28e50ee5b72e90b50e7196fde8ea260e-Abstract-Conference.html

45. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629

46. Erdogan, L. E., Lee, N., Jha, S., Kim, S., Tabrizi, R., Moon, S., Hooper, C., Anumanchipalli, G., Keutzer, K., & Gholami, A. (2024). TinyAgent: Function calling at the edge. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 80–88). Association for Computational Linguistics. https://aclanthology.org/2024.emnlp-demo.9/

47. Paramanayakam, V., Karatzas, A., Anagnostopoulos, I., & Stamoulis, D. (2025). Less is more: Optimizing function calling for LLM execution on edge devices. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE. https://arxiv.org/abs/2411.15399

48. Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., . . . Qiu, Z. (2025). *Qwen3 technical report* (arXiv:2505.09388). arXiv. https://arxiv.org/abs/2505.09388

49. Amini, A., Banaszak, A., Benoit, H., Böök, A., Dakhran, T., Duong, S., Eng, A., Fernandes, F., Härkönen, M., Harrington, A., Hasani, R., Karwa, S., Khrustalev,

Y., Labonne, M., Lechner, M., Lechner, V., Lee, S., Li, Z., Loo, N., . . . Tumma, N. (2025). *LFM2 technical report* (arXiv:2511.23404). arXiv. https://arxiv.org/abs/2511.23404

50. Liquid AI. (2026, January 5). *Introducing LFM2.5: The next generation of on-device AI* [Blog post]. https://www.liquid.ai/blog/introducing-lfm2-5-the-next-generation-of-on-device-ai

51. Bakouch, E., Patiño, C. M., Lozhkov, A., Beeching, E., Roucher, A., Tazi, N., et al. (2025, July 8). *SmolLM3: Smol, multilingual, long-context reasoner* [Blog post]. Hugging Face. https://huggingface.co/blog/smollm3

52. Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., & Gonzalez, J. E. (2025). The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 267, pp. 48371–48392). PMLR. https://proceedings.mlr.press/v267/patil25a.html

53. Mao, H., Yan, F., Ji, C. C.-J., Huang, J., Suresh, V., Huang, Y., Yu, X., Gonzalez, J. E., & Patil, S. G. (2024, September 19). *BFCL V3: Multi-turn & multi-step function calling* [Blog post]. Gorilla, University of California, Berkeley. https://gorilla.cs.berkeley.edu/blogs/13_bfcl_v3_multi_turn.html

54. Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2025). τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR 2025)*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html

55. Barres, V., Dong, H., Ray, S., Si, X., & Narasimhan, K. (2026). τ²-Bench: Evaluating conversational agents in a dual-control environment. In *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*. https://arxiv.org/abs/2506.07982 https://icml.cc/virtual/2026/oral/71171

56. Cuadron, A., Yu, P., Liu, Y., & Gupta, A. (2025). *SABER: Small actions, big errors — Safeguarding mutating steps in LLM agents* (arXiv:2512.07850). arXiv. Poster at the ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems (MemAgents). https://arxiv.org/abs/2512.07850

57. Amazon AGI. (2025). *τ²-Bench-Verified* [Computer software and dataset]. GitHub. https://github.com/amazon-agi/tau2-bench-verified

58. Barres, V., Shi, B., Zytek, O., Ray, S., Dhandhania, K., & Razavi, P. (2026, March 18). *τ³-Bench: Advancing agent benchmarking to knowledge and voice* [Blog post]. Sierra. https://sierra.ai/blog/bench-advancing-agent-benchmarking-to-knowledge-and-voice

59. Li, M., Zhao, Y., Yu, B., Song, F., Li, H., Yu, H., Li, Z., Huang, F., & Li, Y. (2023). API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language*

Processing (pp. 3102–3116). Association for Computational Linguistics. https://aclanthology.org/2023.emnlp-main.187/

60. Guo, Z., Cheng, S., Wang, H., Liang, S., Qin, Y., Li, P., Liu, Z., Sun, M., & Liu, Y. (2024). StableToolBench: Towards stable large-scale benchmarking on tool learning of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024* (pp. 11143–11156). Association for Computational Linguistics. https://aclanthology.org/2024.findings-acl.664/

61. Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., . . . Tang, J. (2024). AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR 2024)*. https://arxiv.org/abs/2308.03688

62. Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*. https://neurips.cc/virtual/2025/poster/121528

63. Xu, H., Zhu, Z., Pan, L., Wang, Z., Zhu, S., Ma, D., Cao, R., Chen, L., & Yu, K. (2025). Reducing tool hallucination via reliability alignment. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)* (Proceedings of Machine Learning Research, Vol. 267). PMLR. https://icml.cc/virtual/2025/poster/45001

64. Willard, B. T., & Louf, R. (2023). *Efficient guided generation for large language models* (arXiv:2307.09702). arXiv. https://arxiv.org/abs/2307.09702

65. Geng, S., Cooper, H., Moskal, M., Jenkins, S., Berman, J., Ranchin, N., West, R., Horvitz, E., & Nori, H. (2025). *JSONSchemaBench: A rigorous benchmark of structured outputs for language models* (arXiv:2501.10868). arXiv. https://arxiv.org/abs/2501.10868

66. Park, K., Wang, J., Berg-Kirkpatrick, T., Polikarpova, N., & D'Antoni, L. (2024). Grammar-aligned decoding. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. https://proceedings.neurips.cc/paper_files/paper/2024/hash/2bdc2267c3d7d01523e2e17ac0a754f3-Abstract-Conference.html

67. Tam, Z. R., Wu, C.-K., Tsai, Y.-L., Lin, C.-Y., Lee, H.-y., & Chen, Y.-N. (2024). Let me speak freely? A study on the impact of format restrictions on large language model performance. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track* (pp. 1218–1236). Association for Computational Linguistics. https://aclanthology.org/2024.emnlp-industry.91/

68. Lee, J. (2026). *Repair, not improvement: Decomposing constrained decoding in tool-call abstention* (arXiv:2608.13959). arXiv. https://arxiv.org/abs/2608.13959

69. Miller, E. (2024). *Adding error bars to evals: A statistical approach to language model evaluations* (arXiv:2411.00640). arXiv. https://doi.org/10.48550/arXiv.2411.00640 https://arxiv.org/abs/2411.00640

70. Dror, R., Baumer, G., Shlomov, S., & Reichart, R. (2018). The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 1383–1392). Association for Computational Linguistics. https://aclanthology.org/P18-1128/

71. Lakens, D. (2017). Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science, 8*(4), 355–362. https://doi.org/10.1177/1948550617697177

72. Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics, 6*(2), 65–70. https://www.jstor.org/stable/4615733

73. Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological), 57*(1), 289–300. https://doi.org/10.1111/j.2517-6161.1995.tb02031.x https://academic.oup.com/jrsssb/article/57/1/289/7035855

74. He, H., & Thinking Machines Lab. (2025, September 10). Defeating nondeterminism in LLM inference. *Thinking Machines Lab: Connectionism*. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/

75. Nosek, B. A., Ebersole, C. R., DeHaven, A. C., & Mellor, D. T. (2018). The preregistration revolution. *Proceedings of the National Academy of Sciences, 115*(11), 2600–2606. https://doi.org/10.1073/pnas.1708274114 https://www.pnas.org/doi/10.1073/pnas.1708274114

76. Bertinetto, L., Henriques, J. F., Albanie, S., Paganini, M., & Varol, G. (Eds.). (2021). *NeurIPS 2020 Workshop on Pre-registration in Machine Learning* (Proceedings of Machine Learning Research, Vol. 148). PMLR. https://proceedings.mlr.press/v148/

77. Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F., Fox, E., & Larochelle, H. (2021). Improving reproducibility in machine learning research (A report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research, 22*, 1–20. https://www.jmlr.org/papers/v22/20-303.html

78. Apple (ml-explore). (2026). *MLX LM: Run LLMs with MLX* (Version 0.31.3) [Computer software]. GitHub. https://github.com/ml-explore/mlx-lm

79. OpenAI. (2025, August 5). *Introducing gpt-oss*. https://openai.com/index/introducing-gpt-oss/

80. Meta AI. (2024, October 24). *Introducing quantized Llama models with increased speed and a reduced memory footprint* [Blog post]. https://ai.meta.com/blog/meta-llama-quantized-lightweight-models/

81. LM Studio. (2026). *Structured output*. LM Studio documentation. Retrieved September 30, 2026, from https://lmstudio.ai/docs/developer/core/structured-output

82. Black, K., & Kumar, R. (2025, December 18). *FunctionGemma: Bringing bespoke function calling to the edge* [Blog post]. Google. https://blog.google/innovation-and-ai/technology/developers-tools/functiongemma/

83. Microsoft. (2026, April 9). *Foundry Local is generally available*. Microsoft Developer Blogs. https://devblogs.microsoft.com/foundry/foundry-local-ga/

84. Apple. (2026, June). *Foundation Models framework: What's new* (WWDC26 session 241). Apple Developer. https://developer.apple.com/videos/play/wwdc2026/241

85. OpenAI. (2024, August 6). *Introducing Structured Outputs in the API* [Blog post]. https://openai.com/index/introducing-structured-outputs-in-the-api/

86. vLLM Project. (2026). *Structured outputs* [Documentation]. Retrieved September 27, 2026, from https://docs.vllm.ai/en/latest/features/structured_outputs/

87. Barres, V., et al. (2026). *τ-Knowledge: Evaluating agents on knowledge-intensive customer service* (arXiv:2603.04370). arXiv. https://arxiv.org/abs/2603.04370

88. Belcak, P., Heinrich, G., Diao, S., Fu, Y., Dong, X., Muralidharan, S., Lin, Y. C., & Molchanov, P. (2025). *Small language models are the future of agentic AI* (arXiv:2506.02153). arXiv. https://doi.org/10.48550/arXiv.2506.02153 https://arxiv.org/abs/2506.02153

89. Hugging Face. (2026). *Qwen/Qwen3-0.6B* [Model repository; download statistics]. Retrieved September 27, 2026, from https://huggingface.co/Qwen/Qwen3-0.6B

90. EuroMLSys. (2026). *The 6th Workshop on Machine Learning and Systems (EuroMLSys 2026)*. Retrieved September 27, 2026, from https://euromlsys.eu/

91. EuroSys. (2026). *EuroSys 2027: European Conference on Computer Systems* and *Call for workshops*. Retrieved September 27, 2026, from https://2027.eurosys.org/

92. International Conference on Learning Representations. (2026). *ICLR 2027 call for workshops*. Retrieved September 30, 2026, from https://iclr.cc/Conferences/2027/CallForWorkshops

93. San José State University, Department of Computer Engineering. (2026, September 4). *College of Engineering high-performance computing (HPC)*. Retrieved September 30, 2026, from https://www.sjsu.edu/cmpe/resources/hpc.php

94. Google. (2026). *Colaboratory: Frequently asked questions*. Retrieved September 27, 2026, from https://research.google.com/colaboratory/faq.html

95. Kaggle. (2026). *Notebooks* [Documentation]. Retrieved September 27, 2026, from https://www.kaggle.com/docs/notebooks

96. NVIDIA. (2026). *NVIDIA Academic Grant Program for researchers*. Retrieved September 27, 2026, from https://www.nvidia.com/en-us/industries/higher-education-research/academic-grant-program/

97. Hugging Face. (2026). *Using GPU Spaces* [Documentation]. Retrieved September 27, 2026, from https://huggingface.co/docs/hub/spaces-gpus

98. IEEE. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). IEEE. https://doi.org/10.1109/IEEESTD.1998.88286 https://ieeexplore.ieee.org/document/720574

99. Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement, 20*(1), 37–46. https://doi.org/10.1177/001316446002000104

100. Google Cloud. (2026). *Accelerator-optimized machine family pricing*. Retrieved September 30, 2026, from https://cloud.google.com/products/compute/pricing/accelerator-optimized

101. Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall/CRC. https://doi.org/10.1201/9780429246593

102. Connor, R. J. (1987). Sample size for testing differences in proportions for the paired-sample design. *Biometrics, 43*(1), 207–211. https://doi.org/10.2307/2531961

103. Liquid AI. (2026). *LFM Open License v1.0*. Retrieved September 27, 2026, from https://huggingface.co/LiquidAI/LFM2.5-2.6B/raw/main/LICENSE https://www.liquid.ai/lfm-license

104. Association for Computing Machinery. (2020). *Artifact review and badging* (Version 1.1, August 24, 2020). https://www.acm.org/publications/policies/artifact-review-and-badging-current

105. OpenAI. (2026). *API pricing* [Documentation]. Retrieved September 27, 2026, from https://developers.openai.com/api/docs/pricing

106. OpenAI. (2026, April 22). *Deprecations*. OpenAI API documentation. Retrieved September 30, 2026, from https://developers.openai.com/api/docs/deprecations

107. Rajbhandari, S., Rasley, J., Ruwase, O., & He, Y. (2020). ZeRO: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis* (pp. 1–16). IEEE. https://arxiv.org/abs/1910.02054

108. Liquid AI. (2026, August 4). *Deploy local agents everywhere with LFM2.5-2.6B* [Blog post]. Hugging Face. https://huggingface.co/blog/LiquidAI/lfm2-5-2-6b

109. Qwen Team. (2026, February). *Qwen3.5-2B* [Model card]. Hugging Face. Retrieved September 30, 2026, from https://huggingface.co/Qwen/Qwen3.5-2B

110. McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika, 12*(2), 153–157. https://doi.org/10.1007/BF02295996

111. Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT '*19)* (pp. 220–229). ACM. https://doi.org/10.1145/3287560.3287596 https://dl.acm.org/doi/10.1145/3287560.3287596

112. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)* (pp. 611–626). Association for Computing Machinery. https://doi.org/10.1145/3600006.3613165 https://dl.acm.org/doi/10.1145/3600006.3613165

113. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. https://www.pearson.com/en-us/subject-catalog/p/design-patterns-elements-of-reusable-object-oriented-software/P200000009480/9780321700698

114. MacIver, D. R., Hatfield-Dodds, Z., & Many Other Contributors. (2019). Hypothesis: A new approach to property-based testing. *Journal of Open Source Software, 4*(43), 1891. https://doi.org/10.21105/joss.01891

115. Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics, 33*(1), 159–174. https://doi.org/10.2307/2529310

116. EuroSys 2022 Artifact Evaluation Committee. (2022). *Badges*. Systems Research Artifacts. https://sysartifacts.github.io/eurosys2022/badges

117. EuroMLSys. (2025). *The 5th Workshop on Machine Learning and Systems (EuroMLSys 2025)*. Retrieved September 27, 2026, from https://2025.euromlsys.eu/

Chapter 2. Project Justification

Small Agents Are Moving On-Device

Small open models now run tool-calling agents on personal devices. NVIDIA researchers argue that small models are capable enough, and far cheaper, for most agent calls [88]. Apple opens its roughly 3B-parameter on-device model, trained with 2-bit quantization-aware training, to third-party apps with constrained tool calling [14]. Demand is large: Qwen3-0.6B alone was downloaded about 30 million times from Hugging Face in the 30 days before 30 September 2026 [89]. Downloads are not deployments, but they show where practitioners are heading.

Quantization Is Unavoidable

Weight memory is roughly parameters × bits ÷ 8. A 4B model needs about 8 GB at 16 bits and about 2 to 2.5 GB at 4 to 5 bits (approximate; Taheri et al. report 8.0 GB and 2.5 GB for Qwen3-4B at FP16 and Q4_K_M [19]). On a 16 GB laptop, the 16-bit model would take half of memory before the KV cache and other apps, and phones have less. At 0.5 to 4B parameters, 4-bit storage is the default.

The Damage Is Real and Hidden

Jang et al. showed the masking effect on τ^2-bench for 9B to 35B models quantized with AWQ and served with vLLM on A100 GPUs. In the most affected of ten cells, 4-bit weights doubled the share of calls to nonexistent tools (19.5% to 38.3%), amplifying errors the 16-bit model already made, yet the score did not change significantly because the ten-error budget absorbed them [21]. At small scale the score can collapse: Qwen3-0.6B (no-think) falls from 62.3% to 16.8% on single-turn BFCL (V3 non-live) under Q4_K_M, while Qwen3-1.7B loses 2.3 points [19]. Whether and where hidden damage appears at 0.5 to 4B in the GGUF formats these models ship in, on llama.cpp's CUDA and CPU code paths, is unmeasured. RQ1 measures it.

Three Repairs, No Evidence to Choose

Practitioners can constrain decoding with a grammar [3, 4], add bits [25, 31], or retrain the quantized model to imitate its full-precision version, as NVIDIA, Liquid AI, Multiverse Computing, and Google already do [13, 2, 37, 12]. No study compares the three, and none reports deployment cost. Retraining results use overall scores and lack a train-longer control. Finer scales in every layer beat restoring whole layers to 8 bits, but only on continuation scoring [25]. Step routing between a 16-bit and a 4-bit model was tested on ALFWorld and WebShop with both models in memory [24], never on tool-calling tasks with measured memory and re-prefill cost.

Who Benefits

- **Developers deploying small agents on GGUF runtimes (Ollama, LM Studio, llama.cpp):** a repair choice matched to their memory and latency budget.
- **Vendors shipping retrained quantized models:** step-level validation with the missing control.
- **Runtime maintainers (llama.cpp):** per-format and per-backend damage maps, grammar overhead on CUDA and CPU, and reproducible failing cases.
- **Benchmark designers:** evidence of how error budgets hide damage.

Why It Is Publishable

We contribute (1) the first controlled head-to-head comparison of the three repair families, each with its control; (2) a step-level measurement method, released as an open testbed with a failure taxonomy and a pre-registered analysis; (3) the first test of step-selective precision against uniform and layer-selective bits on tool-calling benchmarks with its memory and re-prefill cost measured rather than modelled, where a null result is as informative as a positive one; and (4) deployment-cost figures on real 4-bit code paths, CUDA and CPU, for the same GGUF files. The target venue is EuroMLSys, whose 2026 call covered hardware-efficient ML, compression, LLM optimization, agentic systems, and benchmarking in a six-page format [90]; the 2027 call is not out yet, and EuroSys 2027 holds its workshops on 19 April 2027 [91]. The backup is an ICLR 2027 workshop (suggested deadline 1 February 2027) [92]. Related work published since we planned the study [21, 22, 68] compares no repairs, has no control, and reports no deployment cost, so our contribution still stands.

Feasibility

All models and benchmarks are openly licensed. BFCL multi-turn scripts its user turns and costs nothing. Only τ^2-bench-verified needs a simulated user; we host an open-weight one on a GCP L4 Spot VM from the team's cloud credits, about $130 for the study, and replay it free from cache. No LLM API is paid for. All compute runs on SJSU's College of Engineering HPC cluster, which offers H100, A100, and P100 GPU nodes and CPU compute nodes under Slurm to students through a faculty-sponsored account valid for one year [93]. Retraining takes about 20 to 30 H100-hours in total, and the roughly 1,150 machine-hours of evaluation run as parallel jobs on the P100 and compute nodes. Free Colab and Kaggle GPUs are the fallback for retraining the smallest model [94, 95], and the members' laptops are the fallback for development and small runs. **Update after measuring the cluster (2026-10-26): compute nodes cannot reach an outside VM, so the simulated user is planned as an A100 job on the cluster, and the VM becomes a fallback (Chapter 5, Table 19a).**

Funding Prospects

We expect modest, not guaranteed, support. NVIDIA's Academic Grant Program has an "AI Inference, Agents, and Systems Software" track for faculty-led proposals; it is closed as of 30 September 2026, and our advisor could apply if it reopens [96]. Hugging Face community GPU grants can host our public damage-map report [97]. Vendors shipping retrained quantized models have a direct stake in the results, and we will share them.

Chapter 3. Project Requirements and Baseline Approaches

Requirements

This is a combined development and research project: we build a measurement testbed (components C1 to C11) and use it for a controlled study of RQ1 to RQ4. This section gives the testbed's requirements; Baseline Approaches then defines the references, repairs, controls, metrics, and hypotheses. Following IEEE 830 [98], every requirement has an ID, a priority, an acceptance criterion or numeric target, and a trace to a component and a research question.

All experiments run on the SJSU College of Engineering HPC cluster (CoE HPC), a Slurm cluster with NVIDIA H100, A100, and P100 GPU nodes and CPU-only compute nodes [93]. The runtime under study is llama.cpp [1]; we measure its CUDA and CPU backends on the same GGUF files.

Stakeholders and Use Cases

The testbed serves six stakeholders: the team as researchers, llama.cpp maintainers, vendors of retrained quantized models, developers deploying small agents, benchmark designers, and artifact reviewers. Table 5 gives their user stories.

Table 5. User stories (priority E = Essential, D = Desired).

ID	As a…	I want…	So that…	Priority
US-1	researcher	to declare a model × format × backend × repair × benchmark × seed grid in one config and run it as Slurm jobs with one command	every result traces to a config, a commit, and a model-file hash	E
US-2	researcher	every step of every episode logged and labeled with a failure type	I can locate damage by step type and conversation depth, not only count failed tasks	E
US-3	researcher	a self-hosted, pinned simulated user and free replays from a cache	no LLM API bill, no vendor deprecation, and identical user turns in every condition	E
US-4	researcher	retraining and evaluation jobs that checkpoint and resume across Slurm walltime limits	a killed job never costs more than 30 minutes of work	E
US-5	developer deploying on GGUF runtimes	each repair's failure reduction next to its memory, latency, token, and energy cost on the same GPU	I can choose a repair that fits my device budget	E
US-6	model vendor	step-level failure counts for a retrained checkpoint against plain quantization and a train-longer control	I know whether retraining fixes the failures or only the score	E
US-7	reviewer	a one-command re-run that uses cached simulated-user replies	I can regenerate every figure without API spending	E
US-8	runtime maintainer	per-format and per-backend damage maps with minimal failing episodes	I can prioritize fixes to formats, kernels, grammars, or chat templates	D

ID	As a…	I want…	So that…	Priority
US-9	benchmark designer	success re-scored at smaller error budgets, plus pass^k	I can see how much damage the default metric hides	D

Functional Requirements

Essential requirements are the minimum needed to answer RQ1 to RQ4 on the core grid: five models, three 4-bit formats on the CUDA backend and one on the CPU backend, three repair families, two multi-turn benchmarks, and one single-turn benchmark. Desired requirements widen coverage; Optional ones are stretch goals. Comp. names the component that implements each requirement.

Table 6. Essential functional requirements.

ID	Requirement	Acceptance criterion	Comp.	RQ
FR-E1	Load all 800 BFCL v4 multi-turn tasks, the τ²-bench-verified `base` split (airline 50, retail 114, telecom 114 of the 2,285 generated telecom tasks), and 1,000 BFCL v4 non-live single-turn tasks into one task schema (tools as JSON Schema, initial state, checker).	All 2,078 tasks load and pass schema validation; counts match the pinned source files.	C1	All
FR-E2	Score episodes with the official BFCL and τ²-bench checkers, and check the harness against a published result before the main study.	100% verdict agreement with the official evaluators on ≥ 200 replayed trajectories; the Qwen3-0.6B Q4_K_M single-turn drop [19] is re-run with our prompts and its direction reported.	C1, C2	All
FR-E3	Drive multi-turn episodes (model turn, tool execution, scripted or simulated user turn) under each benchmark's limits (τ² defaults: 200 steps, 10 errors) with fixed seeds, a 16,384-token context (12,288 for Qwen3-4B BF16 on 12 GB P100s), and at most 1,024 new tokens per step; a context overflow ends the episode as an outcome.	Killing and restarting a 500-episode batch, including by a Slurm walltime limit, leaves no missing or duplicate episodes.	C2	All
FR-E4	Offer one backend interface over `llama-server` on the CUDA and CPU backends of one pinned llama.cpp build, using the raw completion endpoint with our own rendering of the model's chat template, identical sampling settings, and one tool-call parser. Baseline requests must not apply any grammar.	Identical prompt token IDs on both backends for 100 sampled prompts; server logs show no grammar on baseline requests; the native tool-calling chat endpoint, which attaches a lazy grammar when tools are passed [1], is never used for reported runs.	C3	All
FR-E5	Build every variant from the same original checkpoint with pinned tools: BF16 GGUF, Q4_0 with `--pure` so that embedding and output tensors are also	SHA-256 and measured whole-file bits per weight logged per file; imatrix calibration text shares no 13-	C3, C11	RQ1, RQ2

ID	Requirement	Acceptance criterion	Comp.	RQ
	Q4_0, Q4_K_M, IQ4_XS with imatrix, and all R2 variants.	gram with any benchmark task.		
FR-E6	Serve τ²-bench's simulated user and its natural-language-assertion judge from one pinned open-weight model in GGUF on a `llama-server` instance (GCP L4 spot VM by default), reached through LiteLLM's OpenAI-compatible route, with a content-addressed cache in front.	Replaying a finished episode makes 0 server calls; two greedy runs of the S5 airline tasks give byte-identical user turns; the simulator model, file hash, and build are in every manifest; a simulator outage is retried, never scored.	C4	RQ1–RQ3
FR-E7	Write one record per step: episode, step index, depth, step type, prompt hash, raw output, parsed call, validity flags, tool result, token counts, TTFT, decode time, backend, config ID, seed.	100% of records schema-valid; logged steps equal the runner's step count; Parquet export reloads without loss.	C5	All
FR-E8	Label each failed step with one class: malformed call, hallucinated tool, schema-invalid arguments, wrong tool, wrong argument value, premature termination, policy violation, repetition loop, user-facing response error (τ²-bench), or other.	Cohen's $\kappa \geq 0.80$ [99] between classifier and adjudicated human labels on 300 stratified steps by M3; codebook frozen and registered at M2.	C6	RQ1, RQ3
FR-E9	R1: compile each task's tool schemas into a GBNF grammar that admits free text or one or more schema-valid calls in the model's native tool-call format, applied per request on both backends, at 16 and 4 bits.	0 malformed or hallucinated-tool calls in 1,000 constrained steps per backend; parallel-call tasks remain solvable under the grammar.	C7	RQ2–RQ4
FR-E10	R2: build uniform (Q5_0, Q6_K, Q8_0) and layer-selective (5.5 bits per weight) variants with `llama-quantize`, and the two-copy step-selective variant (Q4_0 and Q8_0 both resident, routed per step).	Whole-file bits per weight logged per file; layer-selective within ±0.1 of 5.5; resident GPU memory of the two-copy pair measured from NVML; every routing decision logged per step.	C7	RQ2–RQ4
FR-E11	R3: run QAD-KL (KL to the model's own 16-bit output through fake quantization that reproduces llama.cpp Q4_0 rounding), QAD-KL+CE (adds a next-token loss), and a train-longer control that repeats QAD-KL+CE with fake quantization off, full-parameter in BF16 on one H100 or A100 node for all three core models. Resume across Slurm walltime limits.	Both arms match on data order, tokens, steps, optimizer, and seed (checked from manifests); a resumed run tracks the uninterrupted loss within 1%.	C7, C8	RQ2, RQ3
FR-E12	Export retrained weights to GGUF Q4_0 and verify them on the real 4-bit code path of both backends.	Exported 4-bit codes and scales equal the trained ones bit for bit; runtime and	C8	RQ2, RQ3

ID	Requirement	Acceptance criterion	Comp.	RQ
		PyTorch top-1 tokens agree on ≥ 99% of 10,000 held-out positions.		
FR-E13	Measure peak GPU and host memory, TTFT, decode tokens/s, extra tokens, and GPU-board energy per task on one exclusively allocated P100 node, and CPU-backend latency and memory (no energy) on one exclusively allocated compute node, for every baseline and repair.	≥ 10 repetitions per configuration; coefficient of variation ≤ 5% for TTFT and decode tokens/s; profiler overhead ≤ 2%.	C9	RQ4
FR-E14	Compute every metric in the Metrics table and produce the damage map (failure rate by step type × depth for each model × format × backend) and the repair comparison matrix.	Metric code recovers known values on synthetic logs; all tables and figures regenerate from logs with one command.	C10	RQ1–RQ4
FR-E15	Apply the pre-registered inclusion rule and statistical tests exactly as registered.	OSF timestamp precedes the first main-study run; registered scripts run unchanged; deviations logged with reasons.	C10	RQ1–RQ4
FR-E16	Keep grids as declarative configs; give every run an immutable ID and a manifest (commit, config hash, model SHA-256, llama.cpp build, backend, GPU model, driver and CUDA versions, node name, OS, seeds, simulator model file hash and build); submit runs as Slurm jobs that resume from the registry.	Every result row traces to a manifest; any run re-executes from its manifest with one command.	C11	All
FR-E17	Measure CUDA-versus-CPU agreement: greedy outputs on the same GGUF file and prompt tokens, for BF16 and Q4_0 on every model.	Share of token-identical outputs on 200 calibration steps per model for BF16 and Q4_0 reported; RQ1 amplification ratios are reported per backend, and pooled across backends only if the share is ≥ 99%.	C3, C10	RQ1, RQ4

Table 7. Desired functional requirements.

ID	Requirement	Acceptance criterion	Comp.	RQ
FR-D1	Extend RQ1 and the vendor-QAD audit to LFM2.5-2.6B, and R1 and R2 to Qwen3-4B and SmolLM3-3B.	Same criteria as the Essential grid.	C7, C11	RQ1, RQ2
FR-D2	Compare Liquid AI's released QAD Q4_0 GGUFs (1.2B-Instruct, 2.6B) with Liquid's own published Q4_0 files, not our rebuilds, so that conversion differences cannot explain the result.	Step-level metrics and flip rates with CIs for both sizes.	C3, C10	RQ3

ID	Requirement	Acceptance criterion	Comp.	RQ
FR-D3	Replicate the R1 and R2 contrasts on Q4_K_M.	Contrasts and repair ranking reported for both base formats.	C7	RQ2
FR-D4	Hand-audit 100 episodes where the 16-bit model succeeded and the 4-bit model failed.	≥ 90% of audited failures fall in a taxonomy class other than “other”; gaps feed a codebook revision before v1.0.	C6	RQ1
FR-D5	Re-run a stratified 5% of τ²-bench episodes with a stronger open-weight simulated user (gpt-oss-120b or Qwen3-32B on a cluster A100).	Kendall’s τ between repair rankings under the two simulators reported; any ranking flip flagged in the paper.	C4, C10	RQ2
FR-D6	Run the Q4_K_M and IQ4_XS formats on the CPU backend as well, so the format × backend interaction is complete.	Scored as in FR-E2; interaction contrast with a 95% CI.	C3, C10	RQ1
FR-D7	Add the BFCL multi-turn long-context category (200 tasks) to the RQ1 grid.	Scored as in FR-E2.	C1	RQ1
FR-D8	Build a static step-log explorer that filters failures by model, format, backend, repair, type, and depth and links to raw steps.	Opens offline; summary for the full log loads in < 5 s on a laptop; any step reachable in ≤ 3 clicks.	C10	All

Table 8. Optional functional requirements.

ID	Requirement	Acceptance criterion	Comp.	RQ
FR-O1	Run the RQ1 grid in thinking mode for Qwen3 and SmolLM3.	Same metrics reported for both modes.	C2, C3	RQ 1
FR-O2	Combine repairs (R1 with R2, R1 with R3) to test whether effects add.	Interaction contrast with a 95% CI.	C7	RQ 2
FR-O3	Add a 3-bit stress point (Q3_K_M, IQ3_XS).	Built and scored as in FR-E5.	C3	RQ 1
FR-O4	Train an in-domain QAD variant on the τ²-bench-verified train split and evaluate it only on the test split.	Zero overlap between training and evaluation task IDs.	C8	RQ 2
FR-O5	Replicate a subset on an A100 or H100 node to test whether GPU generation changes flip rates.	Flip rate relative to the P100 reported.	C3	RQ 1
FR-O6	**Vision-language extension (RQ5, 295B, optional): run one small open vision-language model as GGUF under the same 4-bit formats on a visual tool-calling task set, and apply R1 and R2 unchanged.**	**Go/no-go at M6 (2027-02-05) after a one-day spike S11; if go, the same metrics and step classes as FR-E5, with 95% intervals.**	**C1, C3, C6**	**RQ 5**

FR-O6 answers the advisor's suggestion (2026-10-16) to test the repairs on a different domain. We scope it as a vision-language agent rather than classic computer vision: quantization damage to a tool-calling VLM can be measured with the same testbed, step classes and repairs, whereas a CNN classifier has no tool calls to repair. It is a 295B layer behind a go/no-go at M6 (2027-02-05): spike S11 (at most one day) checks that llama.cpp serves the model with its vision projector on

the P100 and that a scored visual tool-call task set exists. No work on it starts before M5, so it cannot delay K, L1 or L2 on the critical path.

Non-Functional Requirements

Table 9. Non-functional requirements with measurable targets.

ID	Attribute	Requirement and target	Verified by
NFR-1	Reproducibility	Pin all versions (uv lockfile, llama.cpp build tag, CUDA and driver versions from the manifest), seeds, and configs; cache simulated-user replies. Target: a second member re-runs a 200-episode sample from manifests on the same node type and reproduces ≥ 99% of task verdicts.	Scripted reproduction test before M7
NFR-2	Determinism and replay fidelity	Fix server settings (slots, batch size, prompt caching) for all correctness runs, because llama.cpp does not guarantee bit-identical logits across batch sizes with prompt caching on [1, 74]. Target: per model and backend, measure a noise floor (flip rate between two identical 16-bit runs); claim an effect only if it exceeds this floor under the registered test.	Noise-floor runs in Stage 1
NFR-3	Measurement accuracy	Cost runs on one exclusively allocated node (`--exclusive`), one request at a time, GPU clocks logged per sample (pinning needs admin rights), 3 warm-up runs discarded, configurations interleaved in random order, NVML power sampled every 200 ms. Target: ≥ 10 repetitions; coefficient of variation ≤ 5% for TTFT and decode tokens/s; profiler overhead ≤ 2%. Energy is compared only within one GPU model.	C9 self-test; with/without-profiler A/B run
NFR-4	Cost	No paid LLM API. The simulated user runs on a GCP g2-standard-4 (one NVIDIA L4) Spot VM on Yashashav's Google Cloud credits ($0.32 per hour Spot, $0.71 on demand, us-central1, 30 September 2026 [100]); C4's ledger stops scheduling τ^2 episodes at $300, and the VM is stopped between batches. Evaluation and training run on the CoE HPC, with free Colab and Kaggle GPUs as a fallback for Qwen3-0.6B training only. Target: about 400 VM-hours (about $130 Spot); no interrupted job loses more than 30 minutes. **Update: with the simulator as an A100 job on the cluster (Table 19a), the VM is a fallback and expected spend falls toward $0.**	C4 ledger; GCP billing export; C8 resume test
NFR-5	Scalability and throughput	Essential grid ≈ 86,600 episodes, ≈ 1,150 single-stream machine-hours (volume table below). Target: ≥ 8 concurrent `llama-server` jobs on the cluster, measured in the M2 pilot, so the grid fits about 145 wall-clock hours. If the pilot shows fewer than 4 concurrent jobs, apply the registered reduction ladder: 3 trials on BFCL, then 200 BFCL tasks, then R2 on two models.	Pilot throughput report at M2
NFR-6	Portability	Run on Linux with the llama.cpp CUDA and CPU backends under Slurm; runner, classifier, and analysis also run on any Linux or macOS laptop with the CPU backend; Python 3.12. Target: switching backend changes one config field; CI passes on Linux.	CI matrix
NFR-7	Extensibility	Repairs, benchmarks, and backends are plugins registered by name behind fixed interfaces. Target: a new repair needs one module and one config file and no edits to C2, C5, or C10; R1	Code review; plugin test

ID	Attribute	Requirement and target	Verified by
		to R3 are themselves built this way.	
NFR-8	Statistical validity	Register on OSF before the main study [75]; Holm correction within each RQ family [72]; 95% bootstrap CIs over tasks with 10,000 resamples [101]; exploratory analyses labeled as such. Target: ≥ 80% power to detect a 10-point success difference after Holm correction over 10 contrasts, which needs about 200 16-bit-solvable tasks per model at 15% discordant pairs [102].	Power analysis after Stage 1
NFR-9	Data management and licensing	Honor upstream licenses: Qwen3 and SmolLM3 Apache-2.0; LFM2.5 LFM Open License v1.0, which allows derivatives with its notices but no commercial use by entities with ≥ $10M annual revenue [103]; BFCL Apache-2.0; τ²-bench and τ²-bench-verified MIT. Logs hold only synthetic benchmark data, and the simulated user is an Apache-2.0 model, so its text can be released. Target: every released file carries its upstream license; logs (about 1 to 3 GB of Parquet) on Hugging Face Datasets with a Zenodo DOI.	License checklist at M8
NFR-10	Security and isolation	Cloud and Hub credentials only in environment variables and never logged; the simulator VM accepts connections only from the cluster's egress address; benchmark tools are in-process mocks with no real side effects; `llama-server` bound to the node's loopback interface. Target: a secret scan of logs and repository finds 0 keys.	Secret scan before each release
NFR-11	Release quality	Testbed under Apache-2.0 with documentation, examples, ≥ 80% line coverage on C2, C5, C6, and C10, semantic versioning, and a checklist for the ACM Available, Functional, and Reproduced badges [104]. Target: a clean machine regenerates all figures from released logs with one command in < 1 hour.	Clean-machine test before M8

The volume estimate behind NFR-4 and NFR-5 is below. Its inputs are planning assumptions that the pilot replaces with measurements. The task pool is 578 multi-turn tasks per model: 300 BFCL multi-turn tasks (100 each from base, missing-function, and missing-parameter, sampled with a fixed seed) and all 278 τ²-bench-verified tasks. We assume 35% of tasks are solvable at 16 bits (≈ 202 per model, 97 from τ²), 4 sampled trials plus 1 greedy pass per task and configuration, and mean episode times of 30 s at 4 to 8 bits and 60 s at 16 bits on a P100, and 90 s at 4 bits on the CPU backend. Spike S1 replaces the GPU times with measured tokens/s before the M2 pilot. For τ² we assume about 25,000 input and 600 output tokens of simulated-user traffic per episode; `llama-server`'s prompt cache reuses the shared prefix, which is what makes one L4 sufficient.

Table 10. Planning estimate of experiment volume and simulated-user traffic for the Essential grid.

Stage	Configurations	Tasks × trials	Episodes	τ² episodes	Where it runs
Stage 1: 16-bit screening and noise floor	5 models × BF16 on CUDA, run twice (NFR-2)	578 × 5 × 2	28,900	13,900	P100 nodes

Stage	Configurations	Tasks × trials	Episodes	τ² episodes	Where it runs
Stage 2: RQ1 formats and backends	5 models × (Q4_0, Q4_K_M, IQ4_XS on CUDA; Q4_0 on CPU)	202 × 5	20,200	9,700	P100 and compute nodes
Stage 3: RQ2 and RQ3 repairs	3 core models × 13, plus 1 vendor QAD	202 × 5	37,490	16,490	P100 nodes; CPU for R1
Total			86,590	40,090	

The 13 Stage 3 configurations per core model are R1 with and without quantization on CUDA and at 4 bits on the CPU backend (3); Q5_0, Q6_K, Q8_0, layer-selective, and step-selective under each of its two routing policies (6); and QAD-KL at Q4_0, QAD-KL+CE at Q4_0, and the train-longer control at 16 bits and at Q4_0 (4). Q6_K and Q8_0 run on BFCL only, so Stage 3 has fewer τ² episodes. The FR-E17 parity runs take a few hours and are not tabulated. The total is about 1,150 machine-hours at one episode at a time, or about 145 wall-clock hours at the NFR-5 target of 8 concurrent jobs, plus about 1.0 billion input and 24 million output tokens of simulated-user traffic.

Besides the agent, the τ²-bench-verified harness calls two LLM roles: the simulated user (default GPT-5.1) and a judge for natural-language assertions (default GPT-4o-mini), used by all 50 airline and 8 retail tasks [57]. At OpenAI list prices the user role alone would cost about $220 with GPT-4.1-mini and $700 with GPT-5.1 [105], and GPT-4.1-nano retires on 23 October 2026 [106]. We therefore serve both roles from one open-weight GGUF model, greedy, on llama-server, which the harness reaches through LiteLLM's `openai/` route without modification. Spike S5 picks the model from Apache-2.0 candidates with official GGUF builds (gpt-oss-20b, Qwen3-30B-A3B-Instruct-2507, Gemma 4 26B-A4B); a 20B-class model at 4 bits (about 13 GB) fits the L4. The trade-off is that absolute τ² scores are not comparable with the Verified leaderboard, which uses GPT-5.1 as the user. Every claim in this study is a paired contrast under one fixed user, and FR-D5 re-checks the repair ranking with a stronger open simulator.

R3 training volume is set in the pilot. Our planning figure is about 50 million tokens per run, or 2 to 3 GPU-hours on one H100 at an assumed 5,000 to 7,000 tokens/s in BF16. Three runs per core model (two QAD variants and the train-longer control) make nine runs and 20 to 30 H100-hours. Each run checkpoints every 30 minutes and is submitted as a chain of requeued jobs under the 48-hour limit [93]; on A100 nodes the same runs take about 60 to 90 GPU-hours.

Baseline Approaches

Every condition is compared with two anchors: the 16-bit reference and the 4-bit baseline that the repair tries to fix. Each repair also has a control that isolates the mechanism it claims. All conditions share tasks, prompts, chat templates, tool-call parsing, sampling settings, seeds, the simulated-user model and judge, and the llama.cpp build, so paired per-task comparisons are valid.

Reference and Quantized Baselines

The reference for each model is its original released weights as BF16 GGUF on the CUDA backend. Qwen3 and SmolLM3 run in non-thinking mode, and every model uses the sampling settings its model card recommends for the 4 sampled trials. One extra greedy pass per task supports the determinism, backend-parity, and flip-rate analyses. BFCL runs in its function-calling prompting mode, with each model's native tool-call format: JSON inside `<tool_call>` tags for Qwen3 and SmolLM3, and Pythonic calls for LFM2.5. The quantized baselines are the formats publishers ship: third-party publishers ship Qwen3 at Q4_K_M (Qwen's own Qwen3-0.6B GGUF repository ships only Q8_0, as we found during the October pilot), and Liquid AI ships Q4_0, Q4_K_M, and a QAD Q4_0. We rebuild every file from the original checkpoint with pinned tools, so any difference comes from the format, not from a third party's conversion.

Table 11. Reference and default 4-bit baselines (bits per weight are whole-file figures including block scales).

ID	Backend	Format	Bits per weight	Why included
B0	CUDA	BF16 GGUF	16	Reference
B1	CUDA	Q4_0 (`--pure`)	4.5	Simplest grid (32 weights share one FP16 scale); target of vendor QAD [2] and of our R3; base format for RQ2
B2	CUDA	Q4_K_M	5.15 measured on Qwen3-0.6B (≈ 4.9 planned)	K-quant mix; the format behind the published 0.6B collapse [19]
B3	CUDA	IQ4_XS with imatrix	4.25	Non-linear 4-bit guided by an importance matrix
B4	CPU	Q4_0 (`--pure`), same file as B1	4.5	Same weights on the other production code path; isolates kernel effects from format effects

Q4_0 is 4.5 bits per weight only when every tensor is Q4_0. By default `llama-quantize` stores the output tensor, and the token embedding when it is tied to the output, at Q6_K [1]; for Qwen3-0.6B that embedding is about 21% of the parameters. We therefore quantize with `--pure` and log the measured whole-file figure for every variant. The Q4_K_M and IQ4_XS figures come from llama.cpp's quantize tool, where Q4_K_M measured 4.89 bits per weight on Llama 3.1 8B and varies by model [1]; on Qwen3-0.6B our own file measured 5.15, because Q4_K_M keeps 28.5% of that model's parameters, including the tied embedding, at Q6_K (spike S1, October 2026). B0 also runs on the CPU backend for the parity check of FR-E17, and every B0 run is repeated once for the noise floor of NFR-2.

Repair Baselines and Their Controls

Each repair is compared with its own control at a stated matched quantity. RQ2 uses Q4_0 as the base format because its grid can be reproduced exactly during retraining; FR-D3 repeats R1 and R2 on Q4_K_M.

Table 12. Repairs: variants, controls, and matched quantities.

Repair	Variants	Control	What the control isolates	Matched quantity
R1 Grammar	GBNF grammar built from the task's tool schemas: free text or one or more schema-valid calls in the model's native format, sent in the request's `grammar` field on either backend.	(a) Same 4-bit model without grammar; (b) grammar applied at 16 bits; (c) the 4-bit grammar run on the CPU backend	(a) grammar effect under 4-bit; (b) grammar effect without quantization; (a) minus (b) is the interaction; (c) whether grammar overhead depends on the backend	Weights, prompt, sampling, seeds
R2a Uniform bits	Q5_0 (5.5), Q6_K (6.56), Q8_0 (8.5)	Q4_0	Dose response of added bits	None (a ladder)
R2b Layer-selective	Q4_0 everywhere except the most sensitive 25% of linear weights at Q8_0 (`llama-quantize --tensor-type`)	Uniform 5.5 (Q5_0)	Whether where the bits go matters	Memory: 5.5 bits per weight
R2c Step-selective	Q4_0 and Q8_0 models both resident as two `llama-server` processes, routed per step (about 13 bits per weight resident, reported as such)	Q8_0 uniform (8.5); R2b (5.5); Q4_0	Whether spending the extra bits only on routed steps recovers what Q8_0 on every step recovers, and at what memory and re-prefill cost	None: the cost side of RQ4 reports the true residency and extra tokens
R3 Retraining (QAD)	QAD-KL: KL to the model's own 16-bit outputs through Q4_0 fake quantization. QAD-KL+CE: adds a next-token loss on the training mix. Both full-parameter in BF16.	(a) Plain Q4_0 of the original; (b) train-longer: the QAD-KL+CE run with fake quantization off, evaluated both at 16 bits (to show what the extra training did to the unquantized model) and after plain Q4_0 (the matched control); (c) vendor QAD Q4_0 (LFM2.5)	(a) total gain; (b) training under quantization versus extra training	Data, order, tokens, steps, optimizer, seed, final format

Grammar scope. R1 is grammar-constrained decoding [3, 64, 4], applied through llama.cpp's GBNF engine; its per-token overhead [4] is measured on both backends under RQ4. R1 needs no retraining and no extra weight memory. Input-side tool-schema compilation such as TSCG [41] is complementary and out of scope, because it changes the prompt rather than the decoder.

Layer sensitivity. Hu and Ramachandran showed that cheap proxies can misidentify where extra precision helps [25]. We therefore rank tensors by a causal test: we quantize

one tensor at a time to Q4_0 and measure the KL divergence from the 16-bit model on calibration conversations that are disjoint from both benchmarks. The same ranking feeds R2b and R2c, so the two differ only in when the extra bits are used.

Step routing. R2c uses two pre-registered policies. The first routes tool-call steps, detected from the first generated token. The second routes high-entropy steps, with the threshold set on pilot data so that about 25% of steps route. Li et al. instead train a router and keep a full 16-bit model resident [24]. The two-copy variant holds two `llama-server` processes with separate KV caches, so a switch re-prefills every token added since that copy last ran; C9 counts those tokens as extra tokens and charges the second process's memory to the repair. llama.cpp has no per-step precision swap inside one process (its `--override-tensor` flag places tensors on devices, not at precisions), so a step-selective variant at the 5.5-bit budget is not buildable on the runtime we measure and we do not claim one. What we test is what Li et al. left open: whether routing recovers Q8_0's failures at Q8_0's decode cost only on routed steps, with the memory and re-prefill cost measured rather than modelled [24].

Why the train-longer control uses the KL+CE loss. With pure self-distillation, a 16-bit student starts at zero loss and zero gradient, so a train-longer arm would not move. We confirm this with a unit test rather than spend GPU time on it. The control is therefore run for QAD-KL+CE, where the next-token term can change a 16-bit model. This is the confound that published retraining results leave open.

Training on the cluster. Full-parameter training with mixed-precision Adam holds about 16 bytes of model state per parameter [107]: about 12 GB for Qwen3-0.6B (0.75B stored parameters), 19 GB for LFM2.5-1.2B-Instruct, and 32 GB for Qwen3-1.7B (2.03B stored parameters), plus the BF16 teacher and activations. All three fit one 80 GB H100 or A100 with gradient checkpointing, so every core model trains full-parameter in BF16, and each control uses the same parameterization as its QAD arm. The training mix is public function-calling and general text, filtered to share no 13-gram with any benchmark task.

Models, Benchmarks, and Metrics

Table 13. Model grid (stored parameter counts from the Hugging Face checkpoints).

Model	Size (stored parameters)	License	Role
Qwen3-0.6B [48]	0.6B (0.75B)	Apache-2.0	Core: RQ1 to RQ4; QAD
Qwen3-1.7B [48]	1.7B (2.03B)	Apache-2.0	Core: RQ1 to RQ4; QAD
LFM2.5-1.2B-Instruct [50]	1.2B (1.17B)	LFM Open License v1.0	Core: RQ1 to RQ4; vendor QAD Q4_0 available
Qwen3-4B [48]	4B (4.02B)	Apache-2.0	RQ1; R1 and R2 in FR-D1
SmolLM3-3B [51]	3B (3.08B)	Apache-2.0	RQ1; R1 and R2 in FR-D1
LFM2.5-2.6B [108]	2.6B (2.70B)	LFM Open License v1.0	Desired (FR-D1, FR-D2); vendor QAD Q4_0 available

We keep Qwen3 rather than the newer Qwen3.5 small models (0.8B, 2B, 4B, released February 2026 [109]) for three reasons: Qwen3 is a plain dense transformer, which keeps

tensor-level sensitivity ranking and exact Q4_0 export simple, whereas Qwen3.5 uses a hybrid Gated DeltaNet architecture; the published damage results we replicate [19, 25, 24] all use Qwen3; and Qwen3 includes the 0.6B size with the largest published collapse. Qwen3.5-2B is the first candidate for the extended study. LFM2.5 also comes in 230M and 350M sizes; both are below our 0.5B floor and are excluded.

Table 14. Benchmarks.

Benchmark	Tasks in the grid	User turns	Success criterion	API cost
BFCL v4 multi-turn [52]	300 (100 each: base, missing function, missing parameter)	Scripted in the dataset	State and response checkers pass on every turn	None
τ²-bench-verified [57, 56]	278, the `base` split (airline 50, retail 114, telecom 114)	LLM-simulated user, self-hosted	Final database state and required outputs match (reward 1); 58 tasks also use an LLM judge, self-hosted	GCP L4 VM-hours
BFCL v4 non-live single-turn [52]	1,000 (simple Python 400, multiple 200, parallel 200, parallel multiple 200)	None	Abstract-syntax-tree match	None

τ²-bench-verified is Amazon AGI's corrected release of τ²-bench [57, 56]. Sierra has since folded most of the airline and retail corrections into its official τ³-bench release, v1.0.1 [58], but only the Amazon release corrects the telecom domain, and its task files are frozen, so we pin it and compare published scores only with runs on the same task-set version. The single-turn set is the easier benchmark for the smallest models and the setting of the published 0.6B result. **Inclusion rule (pre-registered):** a task stays in a model's analysis set if the 16-bit reference succeeds in at least 1 of 4 sampled trials. The same set is used for every configuration of that model, including retrained ones. A model with fewer than 120 included multi-turn tasks moves to single-turn-only confirmatory analysis, and its multi-turn cells are reported descriptively.

Table 15. Metrics.

Metric	Definition	Level	RQ
Task success	The benchmark's native binary verdict for one trial; success rate is the mean over tasks and trials	Trial	1–3
pass^k	Mean over tasks of C(c, k) / C(n, k), with n = 4 trials and c successes, for k = 1, 2, 4 [54]	Task	1–3
Step failures by type	Count of steps that C6 labels with class f, per episode and per 100 steps	Step	1, 3
Amplification ratio	Mean failures of class f per task in a configuration divided by the same quantity at 16 bits, on the inclusion set, with a bootstrap CI	Model × format × backend × class	1, 3
Flip rate	Share of (task, seed index) pairs whose verdict differs from the 16-bit reference, reported separately for success-to-failure and failure-to-success, next to the 16-bit noise floor	Configuration	1–3

Metric	Definition	Level	RQ
Backend agreement	Share of greedy steps whose output tokens are identical on the CUDA and CPU backends for the same file and prompt	Model × format	1, 4
Gap closed	(m_repair − m_4bit) / (m_16bit − m_4bit) for metric m, computed separately for success and for failures	Repair	2, 3
Budget-limited success	τ^2 success re-scored offline as if the run stopped after b tool errors, for b = 10 (default) and b = 2	Trial	3
Peak memory	Maximum GPU memory of the inference process (weights, KV cache, runtime buffers) from NVML, and peak host RSS, in GB	Task	4
TTFT	Time from request to first generated token, per step, in ms (median per task)	Step	4
Decode throughput	Generated tokens divided by decode time, per step, in tokens/s	Step	4
Extra tokens	Tokens prefilled plus generated per task, minus the base 4-bit configuration on the same task, including re-prefill after precision switches	Task	4
Energy per task	Integral of GPU-board power from NVML (200 ms samples) over the task, minus a same-session idle baseline, in J; not measured on the CPU backend	Task	4
Training cost	GPU-hours and training tokens to produce a retrained checkpoint, with GPU type	Run	4

Step type is the kind of output a step produces (tool call, text reply to the user, or end of conversation). Depth is the step's index within the episode, grouped into pre-registered bins. RQ4 cost is measured in two ways. Fixed-workload runs use identical prefixes and output lengths, so they isolate runtime cost. Natural episodes include behavioral effects such as loops or longer outputs.

Hypotheses

Every hypothesis, test, exclusion rule, and threshold below is registered on OSF before the main study (milestone M2). The unit of analysis is the task. Trials are averaged within a task before paired tests; the greedy pass is analyzed separately for flip rates and backend agreement. Holm correction is applied within each RQ family, with the family sizes fixed at registration [72], and every effect is reported with a 95% bootstrap CI over tasks [101].

- **H1 (RQ1).** 4-bit formats raise per-step failures above the 16-bit reference (amplification ratio > 1), and the excess concentrates in tool-call steps and at greater depth. We make no directional prediction about the CUDA versus CPU backend.
  - Test: for each model × format × backend, a paired permutation test (10,000 permutations) on per-task failure counts. Concentration is tested with a Poisson GEE of failures on precision × step type and precision × depth, clustered by task.
- **H2 (RQ2).** Each repair reduces the excess failures relative to its control. We predict that R1 removes format failures (malformed call, hallucinated tool, schema-invalid arguments) but not semantic ones (wrong tool, wrong value, policy violation), as constrained decoding did on unquantized Qwen3 models [68], and that QAD beats the train-longer control. For step-selective routing versus Q8_0 uniform we make no directional prediction.

  - Test: paired permutation tests on task-level success rates and failure counts; McNemar's exact test on the greedy pass as a secondary test [110]. Gap closed is reported with bootstrap CIs. A claim that R2c matches Q8_0 uniform is supported only if a two one-sided tests equivalence procedure with a ±3-point margin rejects [71].
- **H3 (RQ3).** Repairs close more of the score gap than of the failure gap, and their score gains shrink when τ^2 is re-scored with an error budget of 2.
  - Test: a paired bootstrap over tasks of the difference between gap closed on success and gap closed on failures; the 95% CI must exclude 0.
- **H4 (RQ4).** R1 adds no weight memory, and we predict it adds ≤ 5% to median per-step latency on the CUDA backend for decode; a practitioner measurement reported 6 to 86% overhead on a laptop GPU with the whole request timed [23], so we register TTFT and decode throughput separately and expect any overhead to fall on grammar compilation at TTFT. Because llama.cpp's grammar engine does per-token work on the CPU [4], we predict a larger overhead on the CPU backend. Uniform bits cost weight memory in proportion to bits (Q8_0 ≈ 1.9 × Q4_0). R2c costs about 2.9 × Q4_0 weight memory plus re-prefill tokens. R3 adds no inference cost relative to Q4_0; its cost is training GPU-hours.
  - Test: 10 or more interleaved repetitions; medians with bootstrap CIs; equivalence tests with a ±5% margin for every "no overhead" claim; a Pareto frontier of failure reduction against each cost.
- **Mixed-effects analysis (registered; not optional). A generalized linear mixed model with random intercepts for task (and episode within task) is fitted for every primary contrast with lme4 in R and reported next to the bootstrap; Chapter 7, Statistical Analysis Plan, gives the models and the rule for when they disagree.**

Chapter 4. Dependencies and Deliverables

Dependencies

Table 16 lists the dependencies whose failure would halt the project or remove a research question, each with a fallback and one owner. Ordinary dependencies, such as Python packages, are pinned in the lockfile. Likelihood is our estimate for the project period.

Table 16. Showstopper dependencies.

ID	Dependency	Why it is a showstopper	Likelihood	Mitigation or fallback	Owner
DEP-1	Open weights for Qwen3 (0.6B, 1.7B, 4B), LFM2.5 (1.2B-Instruct, 2.6B), and SmolLM3-3B, under licenses that allow modification and release: Apache-2.0 for Qwen3 and SmolLM3, LFM Open License v1.0 for LFM2.5 [103]	Without the original 16-bit weights we cannot build variants, retrain, or release derivatives	Low	Download, hash, and pin every checkpoint at the start; release LFM2.5 derivatives only under its license. If a model becomes unusable, substitute another open 0.5B to 4B family and log the change in the pre-registration.	Yashashav
DEP-2	BFCL v4 multi-turn (Apache-2.0) and τ²-bench-verified (MIT) remain available and runnable against a local OpenAI-compatible endpoint [52, 57]	They are the tasks and the official checkers; without them there is no study	Low	Fork and pin both repositories at a fixed commit on day one. If τ²-bench-verified breaks, fall back to the original τ²-bench (MIT) [55] and report the fixes we lose.	Vritika
DEP-3	A GPU to host the simulated user and judge for about 400 VM-hours in Sprints 5 to 9:	Only τ² needs a simulated user; without a host, τ² results stop	Low for cost (about $130 Spot, $280 on demand); Medium for GPU quota and Spot	Request L4 quota when the credits are activated; C4 stops scheduling at a $300 ceiling, and the VM stops between batches. The cache keeps every finished episode, and an unreachable simulator is a	Vritika (host, cache); Yashashav (GCP project)

ID	Dependency	Why it is a showstopper	Likelihood	Mitigation or fallback	Owner
	a GCP g2-standard-4 (L4) Spot VM on the team's Google Cloud credits [100]		preemption	retry, not an outcome. Fallbacks: on-demand L4; a CoE HPC A100 if S10 shows long jobs are allowed; BFCL alone, which needs no simulator, still covers RQ1 to RQ3.	
DEP-4	An account on SJSU's CoE HPC cluster (Slurm; 2 H100, 2 A100, and 10 P100 GPU nodes; 17 CPU nodes) [93]. Only faculty may request access, for one year for a capstone, and the cluster is reachable only from campus or the VPN	Every correctness run, every cost run, and all QAD training run on this cluster; without it there is no study	Low for access, Medium for queue contention around finals	Status at 2026-10-16: the advisor agreed on 2026-10-01 to request accounts for all three members and filed the faculty request on 2026-10-11; the CMPE 295A course instructor, Prof. Gao, also forwarded a request for the class, and our student IDs were sent on 2026-10-12. Accounts are being activated. The October pilot ran on a Colab A100 in the meantime (Chapter 8). On day one we check walltime and GPU limits with `sinfo` and size jobs to 48 hours; the C11 registry resumes killed jobs. H100 nodes are kept for training; evaluation runs on the less contended P100 and compute nodes. If access slips past M1, the walking skeleton runs on laptops with the CPU backend and Qwen3-0.6B training moves to free Colab and Kaggle GPUs [94, 95]. If access is denied, we cut the grid to the last rung of the NFR-5 ladder and record the change in the pre-registration.	Ekant (access, evaluation jobs); Yashashav (training jobs)
DEP-5	Exclusive allocation of one P100 node and one compute node for cost measurement, with NVML power and clocks readable without root	RQ4 needs an undisturbed node; on shared nodes, latency and energy figures are meaningless	Low	Request exclusive nodes for three cost blocks in Sprint 9, after finals: A, baselines and R1 and R2 on the P100 (about 30 node-hours); B, R3 arms and controls (about 15); C, CPU-backend latency on a compute node (about 15). `nvidia-smi` power and clock queries need no root; clocks are logged per sample. If exclusive nodes are refused,	Ekant

ID	Dependency	Why it is a showstopper	Likelihood	Mitigation or fallback	Owner
				run at night and report co-located jobs from `squeue`; if NVML power is unreadable, report energy as not measured and keep memory, latency, and token costs.	
DEP-6	llama.cpp supports the chosen models, formats, grammars, and tool-call templates on both the CUDA and CPU backends, and stays stable during the main study [1]	The runtime is the object of study; a breaking change mid-study invalidates comparisons	Low for support, Medium for churn	Official GGUF builds exist for every model. Pin one build tag from M2 to M8, build it for CUDA and CPU, and run conformance tests in CI. All reported runs use the raw completion endpoint, because the chat endpoint attaches a lazy grammar when tools are passed (confirmed in spike S3, October 2026). Prompt caching is off on every request: in the pilot, turning it on changed 61 of 400 greedy Q4_0 outputs.	Ekant
DEP-7	Retrained weights export to GGUF without leaving the Q4_0 grid	If export re-quantizes, the retraining gain can vanish and R3 would be measured on the wrong code path	Medium	Fake quantization reproduces Q4_0 rounding exactly (32-weight blocks, FP16 scale). Write the trained codes and scales directly into the GGUF rather than re-running quantization. Verify bit-exact codes and top-1 agreement ≥ 99% on both backends (FR-E12).	Yashashav
DEP-8	The 16-bit models solve enough tasks to analyze	The inclusion rule keeps only tasks the 16-bit model solves; too few tasks means no statistical power	High for Qwen3-0.6B on multi-turn; Low to Medium for the others	Include the easier BFCL single-turn set. Run the power analysis after Stage 1: about 200 solvable tasks give 80% power for a 10-point difference after Holm correction [102]. Models under 120 solvable tasks move to single-turn-only analysis, as pre-registered.	Vritika
DEP-9	Nobody else publishes the same head-to-head comparison first	Loss of novelty for the paper	Medium	If one repair is published first, we cite it and keep the comparison; the controls and measured costs stay new. The OSF registration (M2) timestamps our design, and each member tracks new work	All

ID	Dependency	Why it is a showstopper	Likelihood	Mitigation or fallback	Owner
				in one area every week.	
DEP-10	Outbound access from cluster compute nodes to the GCP simulator VM and for model downloads	Without it, τ² episodes cannot reach the simulated user	Medium (compute-node internet access is not documented)	Stage model files from the login node. If compute nodes lack outbound access, run the C4 proxy on the login node and tunnel it to the VM, or host the simulator on a cluster A100 (DEP-3 fallback). Checked in spike S10. **Measured 2026-10-26: compute nodes have no outbound access (Hugging Face, GitHub and PyPI time out); the login node has it. Model files are staged from the login node; the simulator host for M2 is a decision for the 2026-10-30 review.**	Vritika

Deliverables

Targets refer to the milestones in Chapter 9: M1 walking skeleton (2026-10-23), M2 references and pre-registration (2026-11-06), M3 damage map (2026-12-04), M4 R1 and R2 (2026-12-18), M5 QAD (2027-01-29), M6 costs (2027-02-05), M7 paper (2027-02-19), and M8 release (2027-05-14). M1 to M4 fall in CMPE 295A and M5 to M8 in CMPE 295B.

Table 17. Core project deliverables.

ID	Deliverable	Description	Form	Target	Owner
DL-1	Open-source testbed	C1 to C11: replays BFCL multi-turn and τ²-bench-verified against llama.cpp on its CUDA and CPU backends, submits runs as Slurm jobs, logs every step, and runs the repairs and profiler	GitHub repository, Apache-2.0, tagged releases, documentation, CI	v0.1 at M4; v1.0 with DOI at M8	Ekant (core, CI); Yashashav (v1.0 release, DOI)
DL-2	Failure taxonomy and classifier	Codebook of step-failure classes, rule-based classifier, 300 human-labeled steps, and Cohen's κ report	Code, PDF codebook, and labeled sample, all public	Codebook v1 frozen at M3 with the κ report; v0 registered at M2	Vritika
DL-3	Pre-registered analysis plan	Hypotheses H1 to H4, metrics, inclusion rule, tests, corrections, power analysis, and analysis code	Frozen OSF registration	M2	Vritika
DL-4	Damage map (RQ1)	Failure rates by step type and depth for each model and 4-bit	Figures, tables, and	v1 at M3; final at M8	Vritika

ID	Deliverable	Description	Form	Target	Owner
		format, with amplification ratios and CIs	CSV in a report		
DL-5	Repair comparison matrix (RQ2, RQ3)	Every repair against its control: success, pass^k, failure counts, gap closed, and budget-limited success	Tables and figures	R1 and R2 at M4; R3 at M5; with costs at M6	Ekant (R1), Yashashav (R2, R3), Vritika (statistics)
DL-6	QAD pipeline	Resumable QAD and train-longer training as Slurm jobs, exact Q4_0 export, and code-path verification on both backends	Notebooks and scripts in the repository	M5	Yashashav
DL-7	Cost measurements (RQ4)	Peak memory, TTFT, decode tokens/s, extra tokens, and GPU-board energy per task for every baseline and repair on one P100 node, CPU-backend latency on one compute node, plus training GPU-hours	Dataset, figures, and measurement protocol	M6	Ekant
DL-8	Step-log dataset	Every main-study episode, one record per step, with run manifests	Parquet on Hugging Face Datasets with a dataset card; Zenodo DOI	v1 two weeks after M7 (Sprint 11, with the arXiv preprint); final at M8	Vritika
DL-9	Workshop paper	Six-page paper on RQ1 to RQ4 for EuroMLSys 2027	PDF	M7 (planning date; backup: ICLR 2027 workshop)	All

Additional Deliverables

These go beyond the core project; each has an acceptance check.

Table 18. Additional deliverables and their acceptance checks.

ID	Deliverable	Acceptance check	Target	Owner
AD-1	Retrained checkpoints on Hugging Face: QAD-KL, QAD-KL+CE, and train-longer controls for the three core models, in GGUF Q4_0	Each file downloads and loads in its runtime; its model card states base model, format, repair, control, license, and step-level failure rates with CIs [111]	M8	Yashashav
AD-2	One-command reproducibility package	On a clean machine, one command regenerates every paper figure and table from released logs in under 1 hour with no API or GPU cost; another re-	M8	Yashashav (environments, DOI);

ID	Deliverable	Acceptance check	Target	Owner
		runs any 200-episode sample from the cache; ACM badge checklist included [104]		Ekant (CI)
AD-3	Public damage-map report and leaderboard	Static site built only from released data; every number links to its config and run IDs; new models can be submitted as a config plus logs, validated by CI	M8	Vritika
AD-4	Practitioner decision guide (“which repair when”)	Two to four pages and a flowchart keyed to memory budget, latency budget, and dominant failure class; every recommendation cites a cell of the comparison matrix	M8	Yashashav (lead), all
AD-5	Standalone tool-schema-to-grammar compiler	pip-installable; emits GBNF for llama.cpp in each model’s native tool-call format; tests compile every tool schema in BFCL and τ^2-bench-verified	M4	Ekant
AD-6	Upstream bug reports and fixes	Every confirmed bug in llama.cpp (grammar, tool-call parsing, quantization, export, CUDA-versus-CPU divergence) is reported upstream with a minimal reproducer within two weeks of confirmation; links kept in the README	Ongoing to M8	Ekant (backends, grammar); Yashashav (quantization, export)
AD-7	arXiv preprints	Workshop version posted at submission; extended version with all RQ3 and RQ4 results and replications by the end of CMPE 295B	M7 and M8	All
AD-8	Expo poster and live demo	On a laptop with the CPU backend, 16-bit, 4-bit, and repaired agents run side by side on the same τ^2 task with live failure labels and cost meters, fully offline from the cache	CMPE 295B Expo	Ekant (demo), all (poster)
AD-9	Hypothesis outcome table	Each registered hypothesis listed with its result, every deviation from the plan, and exploratory findings labeled as such; published with the paper and on OSF	M7	Vritika

Chapter 5. System and Infrastructure Architecture

The testbed is an experiment pipeline built around a pluggable inference core. Figure 1 shows the eleven components (C1 to C11) with their function, technology, and data flow. Everything runs on the CoE HPC cluster [93]. The same code runs on laptops with the llama.cpp CPU backend for development, but every reported number comes from a cluster node named in its run manifest. **After the 2026-10-16 review this chapter gives two views: Figure 1 is the software architecture (components and data flow) and Figure 1a is the infrastructure (where each part runs on the CoE HPC and how data and jobs move); Table 19a lists the design changes the cluster forced.**

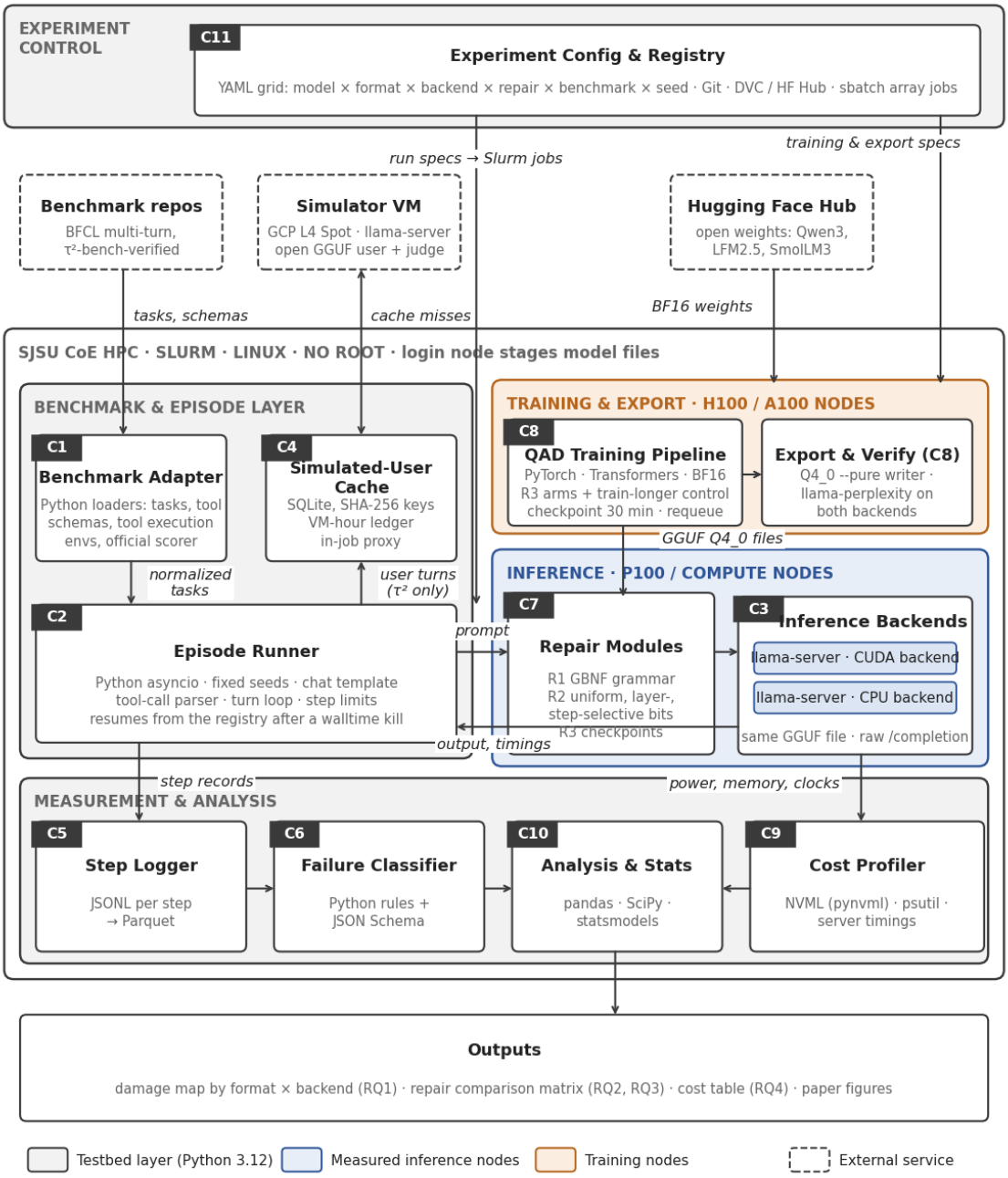


Figure 1. Testbed architecture. Components C1 to C11 are grouped by layer; each box gives the function (bold) and the technology (gray), and arrows show data flow. The blue zone is the measured inference code path on P100 and compute nodes, and the orange zone is the training pipeline on H100 or A100 nodes; both are Slurm partitions of the same cluster. ***Since Workbook 2 the Simulator VM box is the fallback; the simulated user runs as an A100 job inside the cluster (Figure 1a, Table 19a).***

Infrastructure on the CoE HPC

We measured the cluster on 2026-10-26 (Chapter 8, Table 33 note). Members reach the login node over the SJSU VPN; the login node has outbound internet and does git, downloads and job submission only. Slurm (account coe) places work on four kinds of node: 15 P100 12 GB GPUs for the agent array tasks, 4 nodes with 4 A100 40 GB each for the simulated user and judge, 3 H100 80 GB GPUs for QAD training, and 24 CPU nodes for the CPU backend, labeling and statistics. Compute nodes have no outbound network, so everything they read (GGUF files, Hugging Face checkpoints, uv environments, llama.cpp builds for sm_60, sm_80 and sm_90) is staged in the shared group folder /scratch/cmpe295-bertin, and every log, step record and cache entry is written there; /data/cmpe295-bertin holds long-lived copies. Colab and a GCP L4 VM remain fallbacks off the cluster.

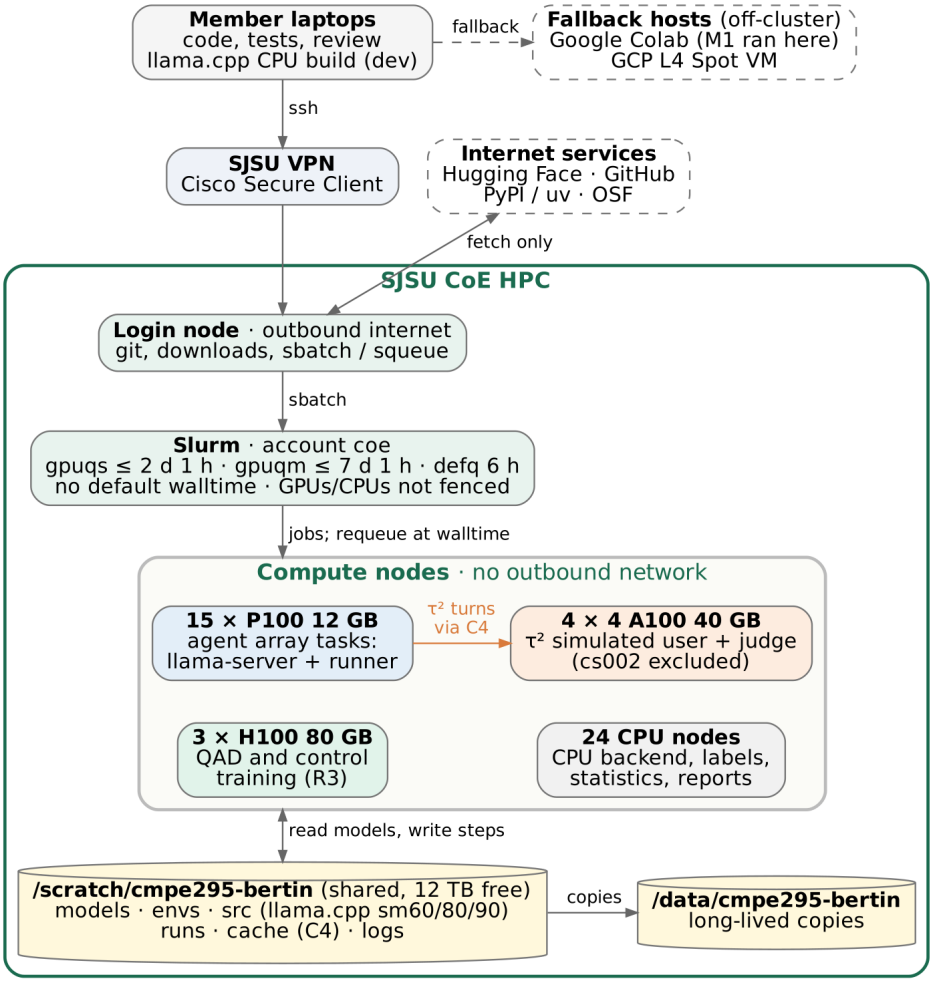


Figure 1a. Infrastructure architecture on the SJSU CoE HPC: access path, Slurm partitions, node types and what each runs, shared storage, and the off-cluster fallbacks. Internet services are reached from the login node only.

Table 19a. Architecture changes made after measuring the CoE HPC (2026-10-26).

Cluster fact	Change to the architecture	Where

Compute nodes have no outbound network	**Split every setup step into fetch (login node) and build or run (job): setup_env.sh --fetch-only / --build-only, make_probe_models.sh DOWNLOAD_ONLY=1; Hugging Face and uv run offline in jobs**	**scripts/, slurm/**
An outside VM cannot be reached from a job	**The τ² simulated user and NL judge move into the cluster: a llama-server job on an A100 (gpuqm) that writes its host and port to /scratch; agent tasks read it, through the C4 cache (planned for M2)**	**C4, DEP-3**
No default walltime; gpuqs 2 d 1 h, gpuqm 7 d 1 h, defq 6 h	**Every job sets -t; array tasks catch SIGUSR1 180 s before the limit, requeue themselves (max 5) and resume without duplicate episodes; QAD checkpoints every 30 minutes**	**C2, C9, slurm/array.sbatch.tmpl**
GPUs and CPUs are not fenced	**Threads come from SLURM_CPUS_PER_TASK, and CUDA_VISIBLE_DEVICES, the node and nvidia-smi are logged in every run manifest; RQ4 cost blocks use exclusive nodes**	**hpc_env.sh, C5, C11**
Mixed GPU generations and drivers (575.57 and 590.44)	**One llama.cpp build for sm_60, sm_80 and sm_90 (CUDA 12.6), with the node, GPU and driver recorded as factors; the backend parity test runs per GPU type**	**build_llama.sbatch, FR-E17**
Home folder full; 12 TB free in the group scratch	**Models, environments, caches and runs live in /scratch/cmpe295-bertin (umask 0007, group-shared)**	**hpc_env.sh**
At most 2 nodes per user at a time (QOS limit, measured 2026-10-28); 9 of the 15 P100 nodes were down that day	**Runs pack one configuration per GPU on 4-GPU A100 nodes (a Slurm job script, not yet committed to main): 4 configurations at once on one node, about 2 s per BFCL episode for Qwen3-0.6B against about 15 s on a P100**	**slurm/, C2**
Node cs002 cannot create a CUDA context	**Excluded in the A100 jobs (--exclude=cs002); a smoke test runs before each batch**	**smoke_s1.sbatch**

Architectural Style

The architecture combines three styles.

- **Batch-sequential pipeline (pipe and filter).** Configs become episodes, episodes become step logs, logs become failure labels, and labels become statistics. Each stage persists its output before the next stage reads it, so any stage can rerun alone. Revising the failure codebook relabels frozen logs without rerunning any model.
- **Layered plugin architecture.** The Episode Runner (C2) depends only on the benchmark (C1), backend (C3), and repair (C7) interfaces. Implementations are registered by name, so changing one experimental factor, including the backend, changes one config field (NFR-7), which is what a paired comparison needs.

- **Client-server at the inference boundary, plus a job layer.** Each backend runs as a `llama-server` process on the node's loopback interface that C2 calls over HTTP. This separates the model's memory and energy from the harness's, and a backend crash cannot corrupt the logs. The Experiment Config & Registry (C11) turns run specs into Slurm array jobs, and the registry, not Slurm, decides what still needs to run, so a walltime kill costs at most the episode in flight. Training runs in its own Slurm partition and shares only versioned model files with the rest of the system.

One Agent Step, End to End

1. C11 expands the config into run specs and submits them as Slurm jobs. On a node, C2 starts an episode with a fixed seed, and C1 supplies the task, its tool schemas, and a mock tool environment.
2. C2 renders the conversation with the model's own chat template. The repair pre-hook (C7) attaches a GBNF grammar (R1) or picks the model copy for this step (R2c).
3. C3 sends the rendered token IDs to `llama-server` on the CUDA or CPU backend. The Cost Profiler (C9) marks the start and end and samples GPU power and memory in between. The backend returns text, time to first token (TTFT), and token counts.
4. C2 parses any tool call, and the Failure Classifier's (C6) validators flag whether it parsed, whether the tool exists, and whether the arguments satisfy the schema.
5. C1's environment executes a tool call and returns the result. A message to the user gets the next user turn, scripted for BFCL or served by the Simulated-User Cache (C4) for τ²-bench.
6. The Step Logger (C5) appends the step record to JSONL. At the end of the episode, C1's official checker records the verdict.
7. After the run, C5 writes Parquet, C6 assigns failure types using whole-episode context, and Analysis & Stats (C10) joins the C9 samples to the steps and runs the pre-registered tests.

Layers: Function and Technology

Experiment Control

The Experiment Config & Registry (C11) declares grids in YAML (Hydra-style) and gives every run an immutable ID and a manifest. The manifest records the commit, lockfile hash, model SHA-256, llama.cpp build and backend, GPU model, driver and CUDA versions, node name, and simulator snapshot (FR-E16). C11 groups pending runs by model file and backend into Slurm array jobs sized under the partition walltime, and a job that is killed resumes from the registry on its next submission. Configs live in Git; large artifacts are versioned with DVC or on the Hugging Face Hub, staged on the cluster's login node.

Benchmark and Episode Layer

C1 wraps BFCL multi-turn and single-turn [52] and τ²-bench-verified [57, 56] behind one task schema: tools as JSON Schema, initial state, and the official checker. C2 is a Python 3.12 asyncio loop that enforces each benchmark's step and error limits and resumes a killed batch without duplicate episodes. C4 sits between τ²-bench's user simulator and the open-weight model that plays the user, served greedily by llama-server on a GCP L4 Spot VM (DEP-3); the same server answers the harness's assertion judge. C4 keys each request by a SHA-256 hash of the simulator file hash, sampling settings, seed, and conversation prefix, stores replies in SQLite, and keeps a ledger of VM-hours. BFCL's user turns are scripted, so BFCL never calls C4. **Update (Table 19a): because jobs cannot reach an outside VM, the simulated user is planned as an A100 cluster job from M2, and the GCP L4 VM becomes the fallback.**

Inference Layer on the Cluster

C3 exposes one `generate(messages, tools, constraint)` call over llama-server [1], built for the CUDA and CPU backends from the same pinned tag. C3 renders the chat template itself and sends token IDs to the raw completion endpoint, so both backends see identical prompt tokens and no request picks up the lazy grammar that the chat endpoint attaches when tools are passed (FR-E4). C7 plugs into C3 through a pre-hook. R1 adds a GBNF grammar in the model's native tool-call format that admits free text or schema-valid calls; its latency cost is measured on both backends [4]. R2 supplies higher-precision weight files or, for step-selective, a second llama-server process holding the Q8_0 file. R3 checkpoints arrive as ordinary model variants.

Training and Export

The QAD Training Pipeline (C8) runs PyTorch and Hugging Face Transformers as Slurm jobs on one H100 or A100 node, checkpointing at least every 30 minutes. It trains the QAD arms [13, 37] and the train-longer control full-parameter in BF16 on identical data, order, and steps (sizing in Chapter 3). The export step (`convert_hf_to_gguf.py`, then a writer that emits the trained Q4_0 codes and scales directly) produces GGUF files that are verified on both backends (Chapter 7). The same tools build the baseline and R2 files, and vendor QAD checkpoints [2] enter as ordinary variants.

Measurement and Analysis

C5 appends JSONL during runs and writes typed Parquet afterward. C6 combines JSON Schema validation with rules to assign the failure taxonomy of Chapter 6. C9 reads `llama-server` timings, samples GPU power, clocks, and memory through NVML (`pynvml`) every 200 ms, and samples host memory with psutil. C10 uses pandas, SciPy, statsmodels, and matplotlib to produce the damage map, the repair comparison matrix, and the paper figures.

Key Decisions and Trade-offs

Table 19. Architectural decisions and the alternatives we rejected.

Decision	Choice	Rejected alternative	Reason
Runtime	llama.cpp on its CUDA and CPU backends	vLLM; mlx-lm on Apple silicon	Every model in the grid ships as GGUF, and Ollama and LM Studio run llama.cpp; vLLM targets batched datacenter serving [112]; mlx-lm needs Apple hardware we cannot allocate on the cluster
Compute	SJSU CoE HPC under Slurm	The members' laptops	About 1,150 machine-hours of evaluation and 20 to 30 H100-hours of training do not fit one laptop; the cluster runs 8 or more evaluation jobs at once and trains every core model full-parameter
Backend process	`llama-server` over HTTP on loopback	In-process bindings by default	Separates model memory and energy from the harness, contains crashes, and matches deployment; llama-cpp-python is used only when we need logits in-process
Prompt path	Our own template rendering to the raw completion	Server chat endpoint with tools	Identical token IDs on both backends; the chat endpoint attaches a lazy tool-call grammar that would partly repair the baseline

Decision	Choice	Rejected alternative	Reason
	endpoint		
Second code path	The CPU backend on the same GGUF file	A second runtime	Same weights, same rounding, different kernels: isolates kernel effects from format effects, and is the path Ollama uses on machines without a GPU
Simulated user	Self-hosted open-weight model on a GCP L4 Spot VM, greedy, behind a content-addressed cache **Update: an A100 job on the cluster from M2; the VM is the fallback (Table 19a).**	A commercial LLM API	No API bill or deprecation risk; the user model is versioned by file hash and released with the study; identical prefixes get identical user turns in every condition; the cost is comparability of absolute τ^2 scores with the leaderboard, which this paired study does not need
Logs	JSONL during runs, Parquet after	A database server	Crash-safe appends on a shared file system and human-readable lines; columnar, typed files for analysis and release
Labeling	Validity flags online; failure types offline	Online-only labels	Some types need whole-episode context; the codebook can change without rerunning models

Reproducibility and Measurement Validity

Several design choices protect measurement validity:

- **Traceability.** Run IDs are hashes of the config cell and seed, so rerunning a finished run is a no-op and every result row resolves to a manifest that names the node, GPU, driver, and build.
- **Pairing.** All conditions share tasks, prompts, parser, seeds, simulator model file, and llama.cpp build. The cache makes user turns identical whenever conversation prefixes match.
- **Pinned server settings.** Slots, batch size, and prompt caching are fixed for all correctness runs, with caching off because llama.cpp does not guarantee bit-identical logits across batch sizes [1] (NFR-2). Cost runs are separate single-stream blocks on an exclusively allocated node with warm-up and randomized interleaving (NFR-3).
- **Clean baselines.** Baseline requests carry no grammar, as the server logs confirm, and a repair acts only through the C7 hook. Two conditions therefore differ only in the repair.
- **Backend parity.** Every model and format is run greedily on both backends for 200 calibration steps (FR-E17), so kernel differences are measured before any pooling.
- **Registered analysis.** C10 reads only frozen logs, and its tests are registered on OSF before the main study [75].

Table 20. Components, their function and technology, data flow, and owners.

Component	Function	Technology	Inputs → Outputs	Owner
C1 Benchmark Adapter	Load tasks, tools, envs; official scoring	Python; BFCL, τ^2-bench-verified repos	Repos → tasks, envs, verdicts	Vritika
C2 Episode Runner	Drive model, tool, and user turns	Python 3.12, asyncio	Run specs, tasks → steps, episodes	Ekant
C3 Inference	One generate call over	`llama-server`,	Prompt, constraint	Ekant

Component	Function	Technology	Inputs → Outputs	Owner
Backends	the CUDA and CPU backends	CUDA, GGUF	→ text, timings	
C4 Simulated-User Cache	Cached user turns from the self-hosted simulator	SQLite, SHA-256, LiteLLM, `llama-server` on GCP L4	Prefix → user turn, ledger	Vritika
C5 Step Logger	One record per step	JSONL, Parquet (pyarrow)	Step events → step table	Ekant
C6 Failure Classifier	Validity flags; failure types	Python rules, JSON Schema	Steps, references → labels	Vritika
C7 Repair Modules	R1 grammar, R2 bits, R3 variants	GBNF, `llama-quantize`, routing	Request, context → constraint or copy	Ekant (R1); Yashashav (R2, R3)
C8 QAD Training Pipeline	QAD, control, export, verification	PyTorch, HF Transformers, llama.cpp tools, Slurm	FP weights, data → GGUF files	Yashashav
C9 Cost Profiler	Memory, latency, tokens, energy	NVML (pynvml), psutil, runtime timers	Markers, samples → cost table	Ekant
C10 Analysis & Stats	Registered tests, damage map, matrix	pandas, SciPy, statsmodels, matplotlib	Labels, costs → figures, tables	Vritika
C11 Experiment Config & Registry	Grids, run IDs, manifests, Slurm jobs	YAML (Hydra-style), Git, DVC or HF Hub, `sbatch`	Config → run specs, jobs, manifests	Yashashav

Chapter 6. Project Design

Guide to the Workbook 2 rubric. This workbook keeps the chapter titles of the development template used for Workbook 1; in the research template, Chapter 6 is Evaluation Methodology and Chapter 7 is System Design/Methodology. Table 20a shows where each rubric item is answered.

Table 20a. Where each Workbook 2 rubric item is answered.

Rubric item	Where	What is there
WB2-6 Evaluation Methodology	**Ch. 3 Baseline Approaches (Tables 11–15); Ch. 7 Measurement Validity, Statistical Analysis Plan, Performance Evaluation Plan; Ch. 8 Pilot Results (Table 35a) and Walking skeleton (Table 36a)**	**Testbed setup, parameters, datasets (BFCL, τ²-bench-verified), the BF16 and 4-bit baselines and repair controls, the measures; baselines executed: the Oct 8 pilot and the M1 runs of BF16, Q4_K_M and Q4_0 on CUDA and CPU plus R1–R3 (252 of 253 episodes)**
WB2-7 System Design/Methodology	**Ch. 5 System and Infrastructure Architecture (Figures 1 and 1a, Tables 19, 19a and 20); Ch. 6 (Figures 2–4, Tables 21–25); Ch. 8 Walking skeleton and Reproducing M1**	**Architecture and components, deployment on the CoE HPC and what the cluster changed; class, sequence and data design; the implemented skeleton and step-by-step reproduction commands (Colab, laptop, CoE HPC)**
WB2-8 Implementation Plan and Progress	**Ch. 8 (Tables 31–37b); changes since Workbook 1 in bold**	**Environment, spikes, pilot, M1 results, progress log, plan to M2, cluster onboarding, decisions**
WB2-9 Project Schedule	**Ch. 9: Figure 6 (Gantt), Figure 6a and Table 39a (PERT), Tables 38–40**	**Both semesters, per-member tasks, dependencies, critical path and float, milestones and ownership**

This chapter gives the design that the M1 walking skeleton implemented (Chapter 8, Tables 36 and 36a). The interface contracts in Table 21 are frozen in testbed/spec.py; the design is refined as each milestone adds components.

Class Design

The class diagram shows the main abstractions of the testbed. The component ID in each class header (C1 to C11) links it to the architecture in Chapter 5. The testbed is written in Python 3.12: abstract classes are `abc.ABC` subclasses, and records such as Step and Episode are frozen dataclasses.

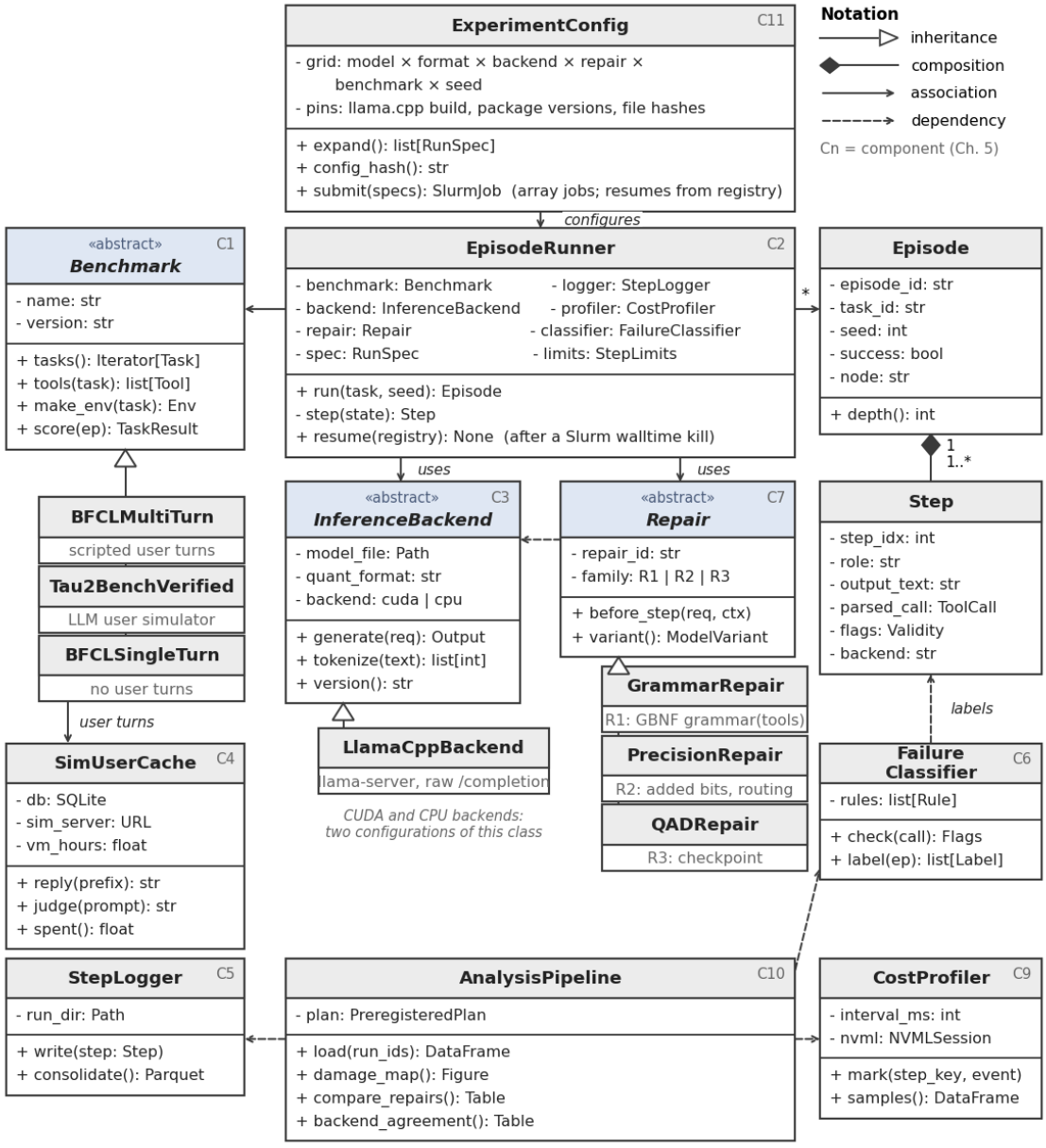


Figure 2. UML class diagram of the testbed's core abstractions. Italic names in blue headers are abstract classes; the component ID appears at the right of each header. The CUDA and CPU backends are two configurations of LlamaCppBackend, and ExperimentConfig submits run specs as Slurm array jobs.

The design uses four patterns from the standard catalog [113]:

- **Strategy (backends and repairs).** EpisodeRunner holds an InferenceBackend and a Repair through their abstract types and never branches on the concrete class. A plugin registry maps config names to classes, so a new backend or repair needs one module and one config file (NFR-7). LlamaCppBackend is an HTTP client for llama-server; the CUDA and CPU backends are two registered configurations of it that differ only in which binary C11 launches.

- **Adapter (benchmarks).** BFCLMultiTurn and Tau2BenchVerified wrap the official repositories' task formats, environments, and checkers behind the Benchmark interface; a third adapter covers BFCL single-turn. The runner never imports benchmark code, and scoring always calls the unmodified official checker.
- **Proxy (simulated-user cache).** SimUserCache exposes the same completion call as the LLM client that τ²-bench's user simulator already uses. The simulator does not know it is cached. The proxy forwards only misses, and it also enforces the budget, so it is both a caching and a protection proxy.
- **Chain of responsibility (failure rules).** FailureClassifier evaluates its rules in a fixed precedence order, and the first match labels the step (see Failure Taxonomy Design).

Table 21. Interface contracts that the controlled comparison depends on.

Interface	Key methods	Contract
Benchmark	`tools`, `make_env`, `score`	Fresh environment per episode; `score` calls the official checker unchanged
InferenceBackend	`generate`, `tokenize`	Same prompt tokens, seed, and settings on both backends; returns text, token counts, TTFT, and decode time; raises a typed error on crash or timeout
Repair	`before_step`, `variant`	May add a constraint or choose a model copy; never changes prompt, sampling, or seed; logs every decision
SimUserCache	`reply`, `spent`	Same key returns the same reply; an unreachable simulator or an exhausted VM-hour ceiling raises an error that the runner treats as a retry
StepLogger	`write`, `consolidate`	Append-only, one line per step; consolidation is lossless

Sequence Design

The sequence diagram (Figure 3) shows one agent step. The repair pre-hook returns a grammar (R1), a model copy (R2 step-selective), or nothing, and Cost Profiler markers bracket the generation while NVML is sampled every 200 ms (loop fragment). Failure Classifier flags are recorded for analysis only, so the testbed observes failures without changing how episodes unfold. The alt fragment separates tool calls from messages to the user. On τ²-bench the Simulated-User Cache answers those messages and calls the simulator only on a miss (opt fragment); on BFCL the next scripted turn comes from the adapter.

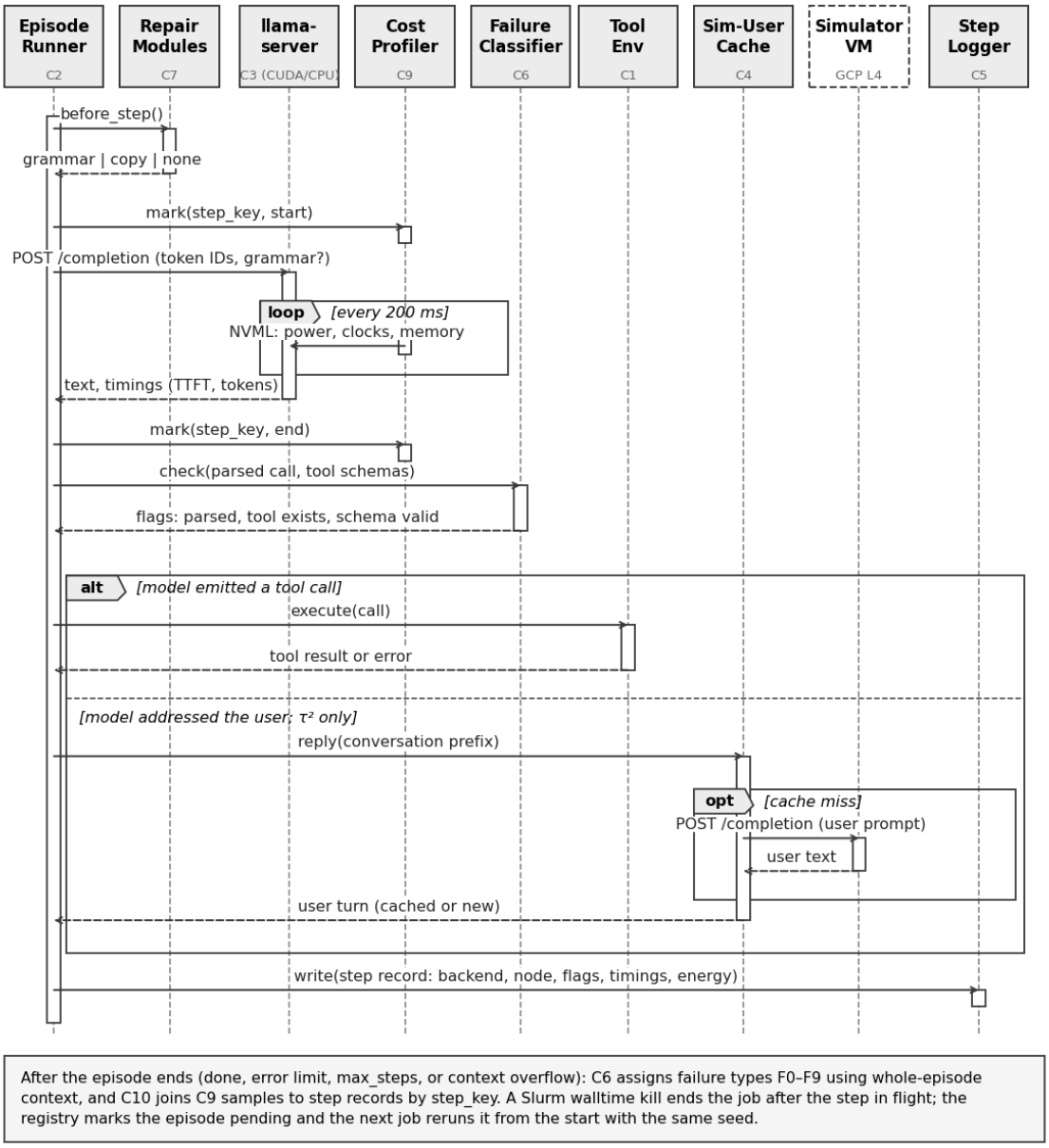


Figure 3. UML sequence diagram of one multi-turn agent step on a cluster node. Solid arrows are calls, dashed arrows are returns, and activation bars show when each participant is busy. The note below the diagram covers episode end and Slurm walltime kills.

Error handling:

- **Backend crash or timeout.** The runner retries up to three times with the same seed. If the step still fails, the episode is marked as an infrastructure error, excluded, and listed, as the exclusion rules in Chapter 7 specify.
- **Slurm walltime kill.** The job receives SIGTERM shortly before its limit; the runner finishes the step in flight, flushes the log, and marks the episode pending in the registry, so the next job resumes it. An episode cut mid-way is rerun from the start with the same seed.

- **Simulator unreachable.** If the GCP VM is preempted or stopped, SimUserCache raises a connection error; the runner marks the episode pending and stops scheduling τ²-bench episodes until the server answers a health check, so no truncated conversation enters the analysis. A VM-hour ceiling in the ledger stops scheduling the same way.
- **Benchmark limits.** The τ²-bench step and error limits and BFCL's per-turn step limit (a turn with more than 20 steps is force-terminated and scored incorrect [53]) end episodes as outcomes, not exclusions.

Data Design

The step record is the central data structure. The Step Logger writes one JSON line per step to `runs/RUN_ID/steps.jsonl` next to the run's `manifest.json`. After the run, consolidation writes a Parquet dataset partitioned by benchmark, model, and format, with the schema version stored in the file metadata. Episode records (verdict, number of steps, termination reason), C9 cost samples, and the C4 reply store are kept in separate tables.

Frozen interfaces (M1). Three contracts in testbed/spec.py are frozen and change only through a pull request reviewed by all three members: RunSpec (one fully specified episode; its run_id is a SHA-256 over every field, so the same spec always gets the same id and duplicates cannot occur), StepRow (one row per model step, written by C5 and labeled by C6), and C3's single call generate(messages, tools, constraint, seed), which returns a GenerateResult on both backends.

Table 22. Step-record schema (one row per conversation step).

Field	Type	Description
run_id	string	Hash of config cell and seed; key to the run manifest
config_hash	string	SHA-256 of the canonical experiment config
model_id	string	Hugging Face repository and revision of the original checkpoint
variant_id	string	SHA-256 of the weight file that produced this step
node	string	Cluster node name; GPU model, driver, and CUDA version are in the run manifest
quant_format	enum	BF16, Q4_0, Q4_K_M, IQ4_XS, Q5_0, Q6_K, Q8_0, and so on
backend	enum	llama.cpp-cuda or llama.cpp-cpu
backend_version	string	llama.cpp build and commit
repair_id	string	none, or the R1, R2, or R3 variant or control
benchmark	enum	BFCL multi-turn, τ²-bench-verified, or BFCL single-turn
task_id	string	Benchmark task identifier
seed	int	Trial seed, shared across conditions
episode_id	string	Hash of run_id and task_id
step_idx	int	Index of the step within the episode, from 0
depth_bin	enum	Pre-registered bin of step_idx
role	enum	assistant, tool, or user
step_type	enum	tool_call, user_reply, or end (assistant steps only)
prev_event	enum	task_start, user_turn, tool_result, or tool_error
prompt_sha256	string	SHA-256 of the exact prompt token IDs

Field	Type	Description
prompt_tokens	int	Prompt length in tokens
output_text	string	Raw model output
completion_tokens	int	Generated tokens
parsed_call	JSON	Tool name and arguments, or null
tool_name	string	Name of the called tool, or null
call_parsed	bool	A well-formed call in the model's native format was found
tool_exists	bool	tool_name is in the task's tool list
args_valid	bool	Arguments parse as a JSON object
schema_valid	bool	Arguments satisfy the tool's JSON Schema
tool_result	string	Environment output or error text, or null
route	enum	R2c routing decision (low or high precision), or null
failure_type	enum	F0 to F9, or null; set offline by C6
latency_ms	float	Request sent to last token, client clock
ttft_ms	float	Request sent to first token
decode_ms	float	Generation time after the first token
energy_mj	float	Idle-subtracted GPU-board energy from NVML in the step window (from C9); null on the CPU backend
peak_rss_mb	float	Peak host resident memory of the backend process in the step window (from C9)
peak_gpu_mb	float	Peak GPU memory of the backend process in the step window from NVML (from C9); null on the CPU backend
timestamp	string	Step start, ISO 8601 in UTC

The entity-relationship diagram shows how step records connect to the rest of the registry. An Experiment has many Runs, a Run has many Episodes, and an Episode has many Steps. Each Run applies one Repair (the unrepaired baseline is the repair "none") and loads one ModelVariant, or two for the step-selective variants, so each Step records which variant produced it. A ModelVariant produced by retraining points to its TrainingJob (QAD or the train-longer control). A Step has at most one Failure label and any number of CostSamples in its time window. Simulated-user replies live in a separate content-addressed store with no foreign keys, because one reply can serve many runs.

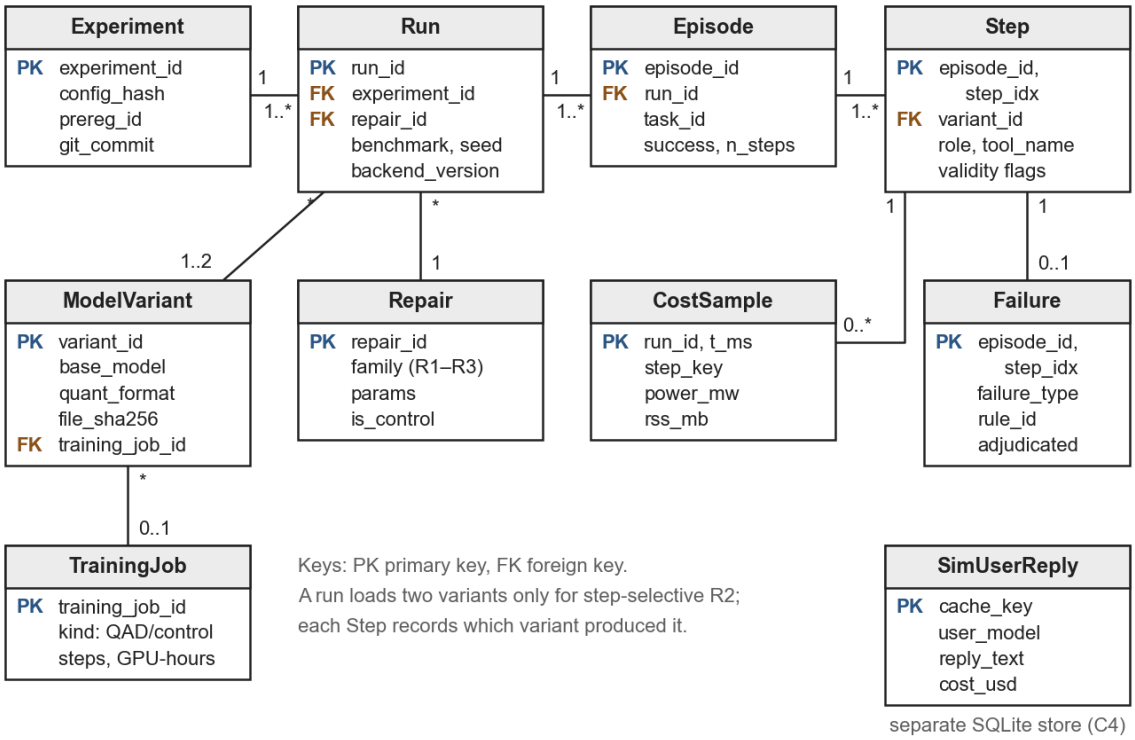


Figure 4. Entity-relationship diagram of the experiment registry and step logs. Multiplicities follow UML notation.

Failure Taxonomy Design

Each failed assistant step receives exactly one label. The classes below are the ten classes of Chapter 3 (FR-E8) with their detection rules; F9 covers errors in what the agent tells the user, which τ²-bench checks separately from tool calls. Rules run in the precedence order F2, F1, F3, F7, F4, F5, F8, F6, F9, and the first match wins. Format failures therefore take priority over semantic ones. A failure that belongs to the whole conversation is attached to the step where it becomes visible: the first repeated call for F7, and the ending step for F6.

Table 23. Failure taxonomy (codebook v0; frozen as v1 before the main study).

ID	Type	Definition	Detection rule	Example
F1	Hallucinated tool	Call names a tool that is not offered	call_parsed and not tool_exists	Calls a plausible tool name absent from the tool list
F2	Malformed call	Intended call cannot be parsed	Call markers present and not call_parsed, or not args_valid	Arguments cut off before the closing brace
F3	Schema-invalid arguments	Existing tool; arguments break its schema	tool_exists and not schema_valid	A required ID missing, or a string where an integer is required
F4	Wrong tool	Valid call to a tool that the reference does	Aligned to the reference trajectory, the tool name	Cancels a booking when the task requires

ID	Type	Definition	Detection rule	Example
		not expect at this point	matches no pending expected action	changing it
F5	Wrong argument value	Expected tool, valid schema, wrong values	Name matches a pending action; normalized arguments differ	Correct tool with the wrong order ID or date
F6	Premature termination	Ends the task or turn while required actions remain	Episode, or BFCL turn, ends with expected actions still pending	Tells the user the change is done without calling the tool
F7	Repetition loop	Same call repeated with no new information	Identical name and normalized arguments at least 3 times in a row (threshold pre-registered)	Re-fetches the same user record until the step limit
F8	Policy violation (τ^2)	Action forbidden by the domain policy	State-changing action outside the expected set that a codebook policy rule flags	Performs a write action the policy forbids for this customer
F9	User-facing response error (τ^2)	Message to the user is wrong or omits required information	A required-output check fails, or the reply contradicts the latest tool result	Quotes an amount that differs from the tool's result
F0	Other	A failed step that matches no rule	Fallback; share monitored	Reviewed by hand; frequent patterns feed the next codebook

F1 to F3 need only the step itself. F4, F5, F6, and F8 need the reference (BFCL's ground-truth trajectories or τ^2-bench's expected actions). Both benchmarks can admit more than one valid path, so uncertain alignments go to human review. Chapter 7 gives the labeling protocol and κ targets. If F0 exceeds 10% of failures, we revise the codebook before freezing it.

Experiment Grid Design

An experiment config lists values for each axis plus constraints. C11 expands the Cartesian product, removes combinations that the constraints forbid, and applies the inclusion set from Stage 1. Each remaining cell and seed becomes a run spec with a hash-derived run ID. The registry tracks each run spec as pending, running, done, or failed. A restart therefore resumes where it stopped, and a finished run is never recomputed. Correctness runs are grouped by model variant and backend into Slurm array jobs to avoid reloading weights. Cost runs are interleaved in random order on an exclusively allocated node.

Table 24. Axes of the Essential grid (details and episode counts in Chapter 3).

Axis	Values	Constraint
Model	Qwen3-0.6B, Qwen3-1.7B, LFM2.5-1.2B-Instruct (core); Qwen3-4B, SmolLM3-3B	R3 only on the core models
Backend and format	CUDA: BF16, Q4_0, Q4_K_M, IQ4_XS; CPU: BF16 (parity only), Q4_0	Same GGUF file on both backends
Repair	none; R1; R2a uniform; R2b layer-selective; R2c step-selective (two-copy); R3 arms and control	Repairs start from Q4_0; R1 also runs at 16 bits and on the CPU backend as controls

Axis	Values	Constraint
Benchmark	BFCL multi-turn, τ²-bench-verified, BFCL single-turn	Tasks limited to the per-model inclusion set
Trial	4 sampled trials plus 1 greedy pass	Same seeds in every condition

For example, one run spec is Qwen3-0.6B, Q4_0 on the CUDA backend, R1 grammar, τ²-bench-verified, trial 2. Its episodes are paired task by task with the same trial of the Q4_0 baseline and of the BF16 reference. C11 packs run specs for the same model file and backend into one Slurm array job so that weights load once per node.

Pre-registration. Before the main study (milestone M2), we register the following on OSF [75]: hypotheses H1 to H4, the inclusion and exclusion rules, the step-type definitions and depth bins, codebook v0 (v1 is frozen at M3 after the κ study and registered as an amendment), the tests, the Holm families [72], and the commit hash of the analysis code. The Experiment entity stores the registration ID. C10 labels any analysis whose code hash differs from the registered one as exploratory, so a confirmatory result cannot silently come from edited code.

CLI and Report Mockup

The testbed has no end-user GUI. Researchers use a command-line interface built on Hydra-style overrides, and readers use a static report.

Table 25. Planned command-line interface.

Command	Purpose	Component
`uv run python -m testbed.run +experiment=NAME`	Expand the grid, submit pending episodes as Slurm array jobs, and resume after interruption; `local=true` runs in the current shell	C11, C2
`uv run python -m testbed.run +experiment=NAME dry_run=true`	List run specs, episode counts, and projected simulator hours without running	C11, C4
`uv run python -m testbed.replay run_id=ID`	Re-execute one run from its manifest with cached user turns	C11, C4
`uv run python -m testbed.cache report`	Hits, misses, and simulator VM-hours against the ceiling	C4
`uv run python -m testbed.classify run_id=ID`	Consolidate logs to Parquet and assign failure types	C5, C6
`uv run python -m testbed.profile +experiment=NAME`	Cost runs with warm-up and interleaving on an exclusive node	C9
`uv run python -m analysis.report +analysis=prereg`	Registered tests, damage map, comparison matrix, HTML report	C10
`uv run python -m train.qad +train=NAME`	Submit QAD or control training as a Slurm job, with checkpoint and resume	C8
`uv run python -m train.export ckpt=PATH`	Export to GGUF Q4_0 and verify on both backends	C8

The damage-map report is a static HTML site built by C10 from released logs. Filters (model, format, backend, repair, benchmark, failure type) sit above a heatmap of step type × depth bin. Each cell shows the failure-rate ratio against the 16-bit reference, with counts, the 95% CI, and run IDs on hover; cells significant after Holm correction are outlined, so significance does not rely on color alone, and clicking a cell lists example steps. A second view shows the repair comparison matrix with a Pareto plot of failure reduction against each cost. The mockup in Figure 5 predates the backend filter.

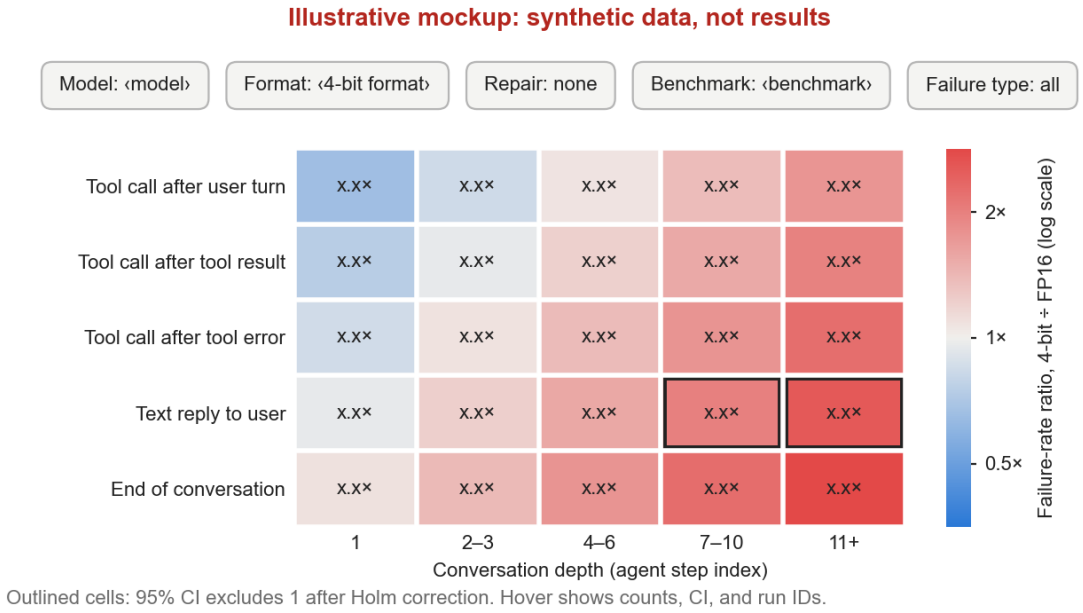


Figure 5. Illustrative mockup of the damage-map view. All values are synthetic placeholders, not results.

Chapter 7. QA, Performance, Deployment Plan

This chapter is the quality plan, updated with M1: the checks added at M1 are described below, and the measured baselines are in Chapter 8 (Tables 35a and 36a). Because the project is a measurement study, quality assurance covers both software correctness and measurement validity: the failure counts must describe the models, not the harness.

Quality Assurance and Testing

Software Testing

Unit tests (pytest) target the functions whose bugs would silently change failure counts: the tool-call parser for each model family's native format, the JSON-schema validator, each Failure Classifier (C6) rule, and the Simulated-User Cache (C4) key. Classifier rules get positive and negative fixtures from real pilot transcripts. The cache key must change exactly when an input to the simulated user's reply changes (conversation prefix, user model file hash, sampling parameters, seed); a wrong key replays the wrong reply.

Property-based tests use Hypothesis [114] to generate malformed outputs (truncated JSON, nested quotes, Unicode, prose around the call). The parser must never crash, never accept a call the validator rejects, and must survive a serialize-then-parse round trip.

Integration tests run a full episode through C1 to C6 on Qwen3-0.6B with a fixed seed and cached simulated-user replies. A parity test sends the same conversation to the CUDA and CPU backends on the same GGUF file: the rendered chat template and token IDs must be identical, and greedy outputs must agree above a pilot-set threshold (exact equality is not expected, because the kernels differ; FR-E17 reports the measured share).

Two checks were added at M1. The planted-effect check gives C10 synthetic paired data (200 tasks × 2 seeds) with a known 0.15 drop in accuracy; it must detect the drop and must not flag a no-effect control (at M1: McNemar p = 1.7e-18, interval [0.117, 0.183]; control p = 1.00). The clean-clone rerun has a second member rebuild the environment and the model files from scratch and rerun 10 episodes; it passes when the plan expands to the same experiment hash and the verdicts agree (scripts/clean_clone_check.py). The C4 replay check reruns a finished τ^2 episode from the cache alone and passes with zero simulator calls (at M1: 8 hits, 0 misses).

Regression tests replay frozen golden episodes from the cache on the same machine, backend build, and model file; step logs must be byte-identical once wall-clock fields are stripped. A difference is either an intended change, regenerated in a reviewed commit, or new non-determinism to investigate.

Continuous integration on GitHub Actions will run unit, property, and CPU-backend integration tests on Linux for every pull request; because the CPU backend is one of the two measured code paths, CI exercises production code rather than a stand-in. CUDA-backend tests and all performance measurements run on the cluster through a Slurm job that the release checklist requires before each tag, with reports committed to the repository. CI is not yet set up; it is planned for M2.

Table 26. Software test plan.

Test level	What is tested	Tool	Pass criterion	Owner
Unit	Tool-call parser, schema validator	pytest	Every fixture yields the expected call or typed error	Ekant
Unit	Classifier rules (C6), cache keys (C4)	pytest	Rules fire only on positive fixtures; keys equal iff inputs equal	Vritika
Unit	Config expansion, run IDs (C11)	pytest	Grid expands as expected; IDs stable across machines	Yashashav
Property	Parser, validator, cache keys	Hypothesis	No crash; no invalid call accepted; round trip holds	Ekant, Vritika
Integration	End-to-end episode, Qwen3-0.6B, fixed seed	pytest, llama.cpp CPU	Episode completes; log passes schema	Ekant
Integration	CUDA vs CPU backend parity on one GGUF file	pytest, Slurm job	Identical token IDs; agreement above pilot threshold	Ekant
Integration	Registry resume after a Slurm walltime kill	pytest, Slurm job with a 10-minute limit	No missing or duplicate episodes	Yashashav
Integration	QAD export (C8) on real 4-bit path, both backends	llama.cpp perplexity tool	KL and top-1 agreement within tolerance	Yashashav
Regression	Golden step logs replayed from cache	pytest	Deterministic fields byte-identical	Ekant
CI	Unit, property, CPU-backend integration	GitHub Actions	Green before merge to main	Ekant

Measurement Validity

Classifier accuracy. Before the main runs, two team members independently label a stratified sample of about 300 pilot steps (all failure classes plus valid steps) from a written codebook, blind to model, precision, and repair. Inter-rater Cohen's κ [99] must be at least 0.80, the edge of the top band on the usual interpretation scale [115]; the third member adjudicates. The classifier must then reach $\kappa \geq 0.80$ against the adjudicated labels, with per-class precision and recall reported; if it falls short, we revise the rules and re-test on a fresh sample. Assuming 90% raw and 30% chance agreement, 300 items give $\kappa \approx 0.86$ with a 95% CI half-width of about ± 0.05. A smaller check on main-study data tests whether classifier error differs by precision condition, which would bias amplification ratios directly. **M1 check (2026-10-27): 100 fresh skeleton episodes on a P100 (job 97528: BF16, Q4_K_M, Q4_0, R1, R2; 686 steps); a stratified sample of 30 steps (6 classes) was labeled blind from the written codebook by an AI rater (a different model from the one that drafted the classifier): raw agreement 1.00, Cohen's κ 1.00. This tests that the written rules are unambiguous, not human agreement; Vritika labels the same 30 steps, and the 300-step human check stays before the main runs. The rater flagged a codebook gap: a tool that returns an error payload (for example, no such file) is labeled ok_call, since exec_error requires the**

executor's own error; codebook v0 (M2) decides whether tool-level errors get their own class.

Reproducing published numbers. Each BF16 reference must match published scores for the same model, benchmark version, and prompting mode within a pre-registered tolerance. If it does not, we audit the chat template, tool-call format, sampling settings, and benchmark version before any repair comparison. Because τ^2-bench-verified corrects tasks of the original τ^2-bench, published scores are compared only with runs on the same task-set version.

Determinism. Each task gets 4 sampled trials with fixed seeds plus 1 greedy pass, and all versions are pinned. Correctness runs use one llama-server slot per job, fixed batch and context sizes, and prompt caching off, which llama.cpp warns can make results non-deterministic [1]; cost runs turn caching on, as a deployed agent would. Kernels that are not batch-invariant cause most run-to-run variation in LLM serving [74], and Jang et al. saw a repeated 456-episode run shift by about 1.5 points [21]. Jobs land on different nodes, so we run each BF16 reference cell twice and use the replicate spread as the noise floor. CUDA-versus-CPU divergence (FR-E17) is reported separately.

Real 4-bit code path. QAD (C8) trains in PyTorch with simulated quantization, but deployment uses llama.cpp's CUDA and CPU kernels; a mismatch in block size, scale format, or rounding would train one quantizer and measure another. For each exported checkpoint we (1) confirm that every quantized tensor in the GGUF file has the intended type; (2) compare next-token distributions of the fake-quant and exported models on held-out text and logged agent prompts (mean KL divergence, top-1 agreement, perplexity) with llama.cpp's perplexity tool [1]; and (3) require the export-vs-fake-quant KL to be small relative to the BF16-vs-4-bit KL (ratio fixed after the pilot, for example below 10%), on both backends. The same audit covers the R2 mixed-precision variants and the train-longer control.

Statistical Analysis Plan

Hypotheses H1 to H4, their tests, the unit of analysis, and the Holm families are defined in Chapter 3 and registered on OSF before the main data collection [75]; pilot runs only set thresholds and check power.

The unit of analysis is the task, with the 4 sampled trials averaged within a task before paired tests against the 16-bit reference; failure rates are failures per agent step. Effect sizes are the rate ratio (amplification) and the difference in failures per 100 steps, with 95% CIs and tests from a paired cluster bootstrap that resamples tasks with all their trials (10,000 resamples) [101], as recommended for correlated LLM evaluation items [69] and as Jang et al. did [21]. **Mixed-effects models are a registered secondary analysis, reported for every primary contrast (below);** Holm–Bonferroni [72] corrects within each hypothesis family, whose sizes are fixed at registration (10 contrasts for the RQ2 family, which sets the power target in NFR-8). BFCL multi-turn has no LLM-simulated user, so its trials differ only through sampling, and the task count bounds the sample.

Reporting dispersion (advisor review, 2026-10-16). Every bar or point that shows a mean carries its dispersion, and the caption says which: ± 1 SD across the 4 sampled

trials (task means first, then SD over trials), and the 95% interval from the paired cluster bootstrap for every effect. Greedy runs are deterministic, so for them the figure shows the 95% Wilson interval over tasks and the caption says SD = 0. Tables give mean ± SD and n. C10 enforces this: the plotting function has no default for the error bars, so a figure without them fails the report job.

Mixed-effects analysis (in, after the 2026-10-16 review). Three models, fitted with lme4::glmer in R 4.4 called from C10 (statsmodels has no frequentist binomial or Poisson mixed model): (A) failures per episode, Poisson, failures ~ configuration + (1 | task), offset log(steps), switched to negative binomial (glmer.nb) when the Pearson dispersion exceeds 1.5; (B) step-level failure, binomial, failure ~ precision × step type + precision × turn depth + (1 | task/episode), which answers where in an episode 4-bit damage appears; (C) episode success, binomial, success ~ configuration + (1 | task). Decision rule: a primary conclusion is robust when the mixed model agrees with the bootstrap in direction and in Holm-corrected significance; if not, it is reported as not robust, with both estimates. If a model fails to converge we refit with the bobyqa optimizer, then drop the episode intercept, then report the failure. The R script and its planted-effect check are due at M2 and frozen in the OSF registration with the rest of the plan.

Power. Given the total failure events of one type in a paired FP/4-bit comparison, the 4-bit share is 0.5 under the null and 0.6 at a 1.5× amplification (assuming equal step counts). Detecting this with 80% power at two-sided $\alpha = 0.05/10$ (Holm's first step with 10 tests per family) needs about 330 events, times an unknown clustering design effect D; the table assumes 10 agent steps per episode. For scale, Jang et al. ran 456 episodes per cell (114 tasks × 4 trials) and saw FP tool-name hallucination rates from below 1% to about 20% of tool calls [21]. At 456 episodes, a 10% base rate is detectable even at D = 3, 5% only if D is below about 1.7, and 2% not at all, so rare types are pooled or reported as exploratory. We will recompute the table from pilot data before registering.

Table 27. Approximate episodes (tasks × trials) needed to detect a 1.5× rate ratio (80% power, α = 0.05/10, 10 steps per episode).

FP base rate per step	Expected events per FP/4-bit episode pair	Episodes, D = 1	Episodes, D = 3
2%	0.50	681	2,043
5%	1.25	273	817
10%	2.50	137	409
20%	5.00	69	205

Exclusion rules, fixed in advance:

- Tasks the 16-bit reference fails in all 4 sampled trials are excluded per model from confirmatory tests (they reflect missing capability, not quantization); we report the counts and an all-task sensitivity analysis. A model with fewer than 120 included multi-turn tasks moves to single-turn-only confirmatory analysis, and its multi-turn cells are reported descriptively.
- Infrastructure errors (backend crash, simulator timeout, node failure) are retried up to three times with the same seed; episodes that still fail are excluded and listed. Episodes interrupted by a Slurm walltime limit are rerun from the start, not excluded.
- Episodes that hit the step or error limit are outcomes, not exclusions.

Performance Evaluation Plan

A repair's cost is what it adds to unrepaired 4-bit on the same task and seed, measured by the Cost Profiler (C9) on one exclusively allocated P100 node and, for the CPU backend, on one exclusive compute node. CPU-backend cost is memory, TTFT, and throughput only, because the cluster does not expose CPU power counters to users. Laptops are for development only. The P100 is a 2016 datacenter GPU, so we report per-token and per-bit costs and make no claims about laptops or phones.

Table 28. Cost metrics.

Metric	Definition	Tool	Protocol	Reported as
Peak memory	Peak GPU memory of the process; max host RSS	NVML per-process query; psutil sampler; llama.cpp buffer log	Per episode	Median, IQR (GB)
Model size	Bytes on disk of the GGUF file	File system	Once per artifact	GB; bits per weight
TTFT	Request sent to first streamed token	Client timer; llama-server `timings`	Per step, binned by prompt length	Median, IQR (ms)
Decode speed	Generated tokens ÷ decode time	llama-server `timings`; llama-bench	Per step	Median, IQR (tokens/s)
Extra tokens	Tokens per task minus unrepaired 4-bit (grammar-forced, retries, longer turns)	Step Logger (C5)	Paired by task × seed	Median Δ and %
Energy per task	Sum of GPU-board power × interval, idle subtracted; not measured on the CPU backend	NVML (pynvml)	Per task, 200 ms samples	Median, IQR (J)
One-time cost	QAD and control training; conversion	C8 run logs	Per training run	GPU-hours by GPU type

llama-server returns per-request timings [1]; we cross-check them against client-side timers.

Energy. NVML reports board power at a resolution NVIDIA documents as ±5 W on this GPU generation and integrates over a moving window, so we sample every 200 ms, subtract a same-session idle baseline, and compare repairs only on one GPU model; whole-system energy is not reported. **One-time costs** record GPU type and GPU-hours from Slurm accounting, including resume overhead.

Protocol. Cost blocks run on an exclusively allocated node with GPU clocks logged per sample, one llama.cpp build and driver for the whole study, and no other jobs on the node. Cost runs replay cached simulated-user replies, so API latency never enters TTFT, and keep prompt caching on, as a deployed agent would. Each block starts with a warm-up workload until decode throughput is stable (three consecutive windows within 5%), with thermal pressure logged. Three warm-up runs per configuration are discarded, then N = 10 repetitions (revised from pilot variance) run in randomized interleaved order so drift does not align with one repair. We report medians and IQRs, with bootstrap CIs for median ratios.

Deployment Plan

For a research artifact, deployment means a release others can run, check, and extend.

Open-source release. The testbed will be public on GitHub under Apache-2.0, which has an explicit patent grant and is compatible with llama.cpp (MIT) [1], BFCL's Gorilla repository (Apache-2.0), and τ²-bench-verified (MIT). Released weights keep their model licenses: Apache-2.0 for Qwen3 and SmolLM3, and the LFM Open License v1.0 for LFM2.5 derivatives. Releases are semantically versioned. One Dockerfile, also run as a Singularity image on the cluster, covers evaluation, training, analysis, and CI; a locked uv environment is the alternative.

Artifacts. Step-log datasets (Parquet) go on the Hugging Face Hub with dataset cards; repaired checkpoints and controls go there as GGUF Q4_0 files with model cards [111] stating base model, quantization format, repair, control, and failure rates with CIs. Configs (C11) live in the repository, and the pre-registration is frozen on OSF. The simulated user is an Apache-2.0 open model, so its generated text is released with the logs and the model file hash lets anyone regenerate it.

Reproducibility packaging targets the ACM badges [104], the same three that EuroSys 2022 artifact evaluation awarded [116]: *Available* (releases archived with a DOI, for example through Zenodo), *Functional* (documented, complete, exercisable; one command regenerates every table and figure from released step logs on a laptop, with no API or GPU cost), and *Reproduced* (someone outside the team reruns a subset from scratch within our stated CIs). We do not assume EuroMLSys runs artifact evaluation.

Paper and demo. We target EuroMLSys 2027 (backup: an ICLR 2027 workshop [92]) and an offline Expo demo (AD-8).

Table 29. Release checklist.

Item	Content	Owner	Target
Repository v0.1	C1 to C6, C11, tests, CI, license, README	Ekant	295A end
Environments	uv lockfile, Dockerfile and Singularity image (Linux/CUDA and CPU)	Yashashav	295A end
Codebook and labeled sample	Taxonomy, 300 labeled pilot steps, κ report	Vritika	295A end
OSF pre-registration	Hypotheses, exclusions, analysis code	Vritika	M2 (2026-11-06), before main runs
Repository v0.5	R1 to R3, C8 export checks, C9 profiler	Yashashav, Ekant	295B mid
Step-log dataset v1	FP and 4-bit runs with dataset card	Vritika	295B mid
Paper submission	EuroMLSys 2027	All	295B mid
Checkpoints	QAD and control in GGUF Q4_0, model cards	Yashashav	295B end
Release v1.0 with DOI	Frozen code, data, configs	Yashashav	295B end
Damage-map report	Static site built from release data	Vritika	295B end

Item	Content	Owner	Target
External reproduction	Outside rerun from cache and from scratch	All	295B end
Expo demo	Side-by-side agents on a laptop, CPU backend	Ekant	295B end

Risks to Validity

Table 30. Threats to validity and mitigations.

Type	Threat	Mitigation
Internal	Simulated-user variance changes conversations between conditions	One self-hosted greedy simulator, cached replies shared across conditions; task-cluster bootstrap
Construct	A 20B-class open simulator is weaker than the leaderboard's GPT-5.1 user, shifting which tasks are solvable	Inclusion set defined under the same simulator for every condition; FR-D5 robustness check with a stronger simulator on 5%; absolute scores not compared with the leaderboard
Internal	Inference non-determinism mimics or hides damage	Fixed seeds plus a greedy pass, pinned build and driver, one slot per job, node recorded, measured noise floor
Internal	Jobs land on different nodes of one partition	Node in every manifest; noise floor measured across nodes; cost blocks on one exclusive node
Internal	QAD gains come from extra training, not training under quantization	Train-longer-at-FP control with equal data and steps
Internal	Classifier error differs by condition	Blind labeling; $\kappa \geq 0.80$; per-condition error check
Construct	Taxonomy misses or merges failure types	Codebook from pilot data; monitor the "other" share; manual review
Construct	Grammar conflicts with the chat template (native tool-call tags, reasoning blocks)	Grammar built from each model's native format; validated at FP first
Construct	NVML board power is coarse and windowed	Same-GPU comparisons only; idle subtracted; 200 ms sampling; limits stated
External	Benchmark contamination inflates absolute scores	Paired FP vs 4-bit contrasts; published-number sanity check
External	A P100 datacenter GPU is not a laptop or phone	Report per-token and per-bit costs; CPU backend as a second path; no laptop or phone claims
External	Few models and benchmarks; the runtime changes	Three model families; two backends of one pinned build; released configs for reruns
Statistical	Rare failure types underpowered; many tests	Power plan; Holm correction; rare types exploratory

Chapter 8. Implementation Plan and Progress

This chapter covers the software environment, each member's tools, the hardware, the technology spikes, and the first prototype, then how we track progress. Version 1 (2026-09-30) was written before any experiment ran. Version 2 adds the October 2026 pilot: spikes S1 to S3, the single-turn part of S4, S6 and S7, run on Google Colab (A100 GPU and CPU) before cluster accounts were available, with their results in the new section "Pilot Results, October 2026".

Programming and Execution Environment

The testbed runs in two contexts. The cluster context, SJSU's CoE HPC (Linux nodes under Slurm, reachable from campus or the VPN [93]), runs all eleven components and produces every reported number. The laptop context is each member's own machine, which runs the llama.cpp CPU backend for unit tests, spikes, and small replays. Both share one Git repository (`testbed/`, `train/`, `configs/`, `analysis/`, `slurm/`, `spikes/`) and one lockfile. Yashashav maintains the lockfile and a Dockerfile that also builds as a Singularity image; Ekant maintains continuous integration on GitHub Actions (Chapter 7).

We use Python 3.12 everywhere, managed by uv, with conda only when a package has no wheel; both benchmark harnesses need Python ≥ 3.10 [57, 52]. Each harness gets its own uv environment because each pins its own dependencies, and the testbed talks to them only through HTTP and files. The cluster provides Lmod, CUDA modules, and Singularity but no root access [93], so the uv environment lives in `$HOME` and anything that needs a system library ships in the Singularity image.

llama.cpp is built from one pinned release tag for the whole study [1], twice on the cluster: `cmake -B build-cuda -DGGML_CUDA=ON` and `cmake -B build-cpu -DGGML_CUDA=OFF`. The build provides llama-server, llama-cli, llama-quantize, llama-imatrix, and llama-perplexity, and the repository provides `convert_hf_to_gguf.py`. Training uses the same tag so exported files match the runtime. C11 writes the llama-server version, backend, node name, and GPU, driver, and CUDA versions into every run manifest.

The testbed relies on three `llama-server` interfaces:

- the raw `/completion` endpoint, which takes the prompt as token IDs, accepts per-request `grammar` and `json_schema` fields (R1), and returns the timings the Cost Profiler reads. All reported runs use it;
- the OpenAI-compatible `/v1/chat/completions` endpoint, which attaches a lazy tool-call grammar when tools are passed and would therefore partly repair a baseline. We use it only in spike S3 to show this;
- the `/v1/completions` endpoint, used only to check our parser against bfcl-eval's local-model handlers (S4).

R1 uses llama.cpp's GBNF engine on both backends [1]. llama-cpp-python (CUDA build) is optional and used only when we need logits in-process, for example to compare a PyTorch fake-quantized model with llama.cpp token by token.

Table 31. Execution environment, version policy, and the command that verifies each layer.

Layer	Tool	Version policy	Purpose	Verified by
Scheduler	Slurm on the CoE HPC	Cluster-provided	Array jobs for evaluation; single-node jobs for training and cost blocks	`sinfo`; `sbatch --test-only slurm/skeleton.sbatch`
Interpreter	Python 3.12 via uv (conda fallback)	3.12.x; one `uv.lock`	Testbed, harnesses, analysis	`uv run python -V`
Container	Singularity image built from the Dockerfile	Rebuilt per release tag	Reproducible cluster environment	`singularity exec testbed.sif uv run python -V`
GGUF runtime	llama.cpp, CMake build, CUDA and CPU	One release tag for the study	`llama-server`, `llama-cli`	`llama-server --version`
GGUF tooling	`convert_hf_to_gguf.py`, `llama-quantize`, `llama-imatrix`, `llama-perplexity`	Same tag as the runtime	Conversion, 4-bit variants, KL checks	`llama-quantize --help`
Binding (optional)	llama-cpp-python, CUDA build	Pinned	In-process logits	`python -c "import llama_cpp"`
Constrained decoding	GBNF and JSON schema (llama.cpp)	Same tag	Repair R1	S3
Training	PyTorch, Transformers, Datasets, Accelerate (CUDA)	Pinned in the same lock	QAD pipeline (C8)	`python -c "import torch; print(torch.cuda.is_available())"`
Benchmarks	`bfcl-eval` 2026.3.23; τ²-bench-verified at a pinned commit (LiteLLM)	Pinned, own environments	Reference scoring for C1	`bfcl --help`; `tau2 --help`
Analysis	pandas, pyarrow, SciPy, statsmodels, matplotlib	Pinned	Analysis & Stats (C10)	`uv run pytest tests/analysis`
Configuration	Hydra (`hydra-core`) over YAML	Pinned	Config & Registry (C11)	`python -m testbed.run --cfg job`
Quality	pytest, pre-commit (ruff)	Hooks pinned	Tests and lint before commit	`pre-commit run --all-files`
Collaboration	Git, GitHub (Projects, Actions), Hugging Face Hub	Hosted	Code, board, CI, model artifacts	`gh auth status`; `hf auth whoami`
Simulator host	GCP `g2-standard-4` Spot VM (L4, 24 GB), `llama-server` CUDA build **Update: from M2 an A100 job on the cluster; this VM is the**	Same llama.cpp tag as the cluster	Simulated user and judge for τ²	`gcloud compute instances list`; `curl $VM:8080/health`

Layer	Tool	Version policy	Purpose	Verified by
	fallback (Table 19a).			
Energy and memory	NVML through `pynvml`	Driver-provided	Cost Profiler (C9)	`nvidia-smi --query-gpu=power.draw,clocks.sm --format=csv`

Development Tools per Team Member

Every member uses the same repository, GitHub Projects board, cluster, and Hugging Face organization; role-specific tools follow component ownership (Chapter 9). The only paid resource is the GCP L4 Spot VM for the simulated user, on Yashashav's Google Cloud credits: C4 stops τ² scheduling at $300, and expected spend is about $130. **With the simulator on the cluster, the VM is needed only as a fallback, and expected spend drops toward $0.**

Table 32. Development tools, accounts, and status per member, as of 2026-10-16, with the cluster status updated on 2026-10-26.

Member	Machine	Role-specific tools	Accounts and quotas	Status
Ekant Kapgate	Apple M4 MacBook, 16 GB unified memory (development only)	llama.cpp (CPU and CUDA builds), Slurm, NVML, psutil, GitHub Actions	CoE HPC account requested 2026-09-30, pending; GitHub; Hugging Face (read token)	Laptop set up; owner rerun of S1, S3 and S6 in progress; cluster pending **Update 2026-10-26: CoE HPC account active and the group's Slurm association fixed; first own cluster job is onboarding step 3 or 5 (Table 37b, M2). Update 2026-10-29: CI on GitHub Actions is not set up yet (M2).**
Vritika Malhotra	Personal laptop (CPU build of	`bfcl-eval`, τ²-bench-verified,	CoE HPC account requested 2026-09-30, pending; GitHub;	Harness environments

Member	Machine	Role-specific tools	Accounts and quotas	Status
	llama.cpp; model to be recorded at S4)	SQLite, SciPy, statsmodels, Jupyter	Hugging Face; OSF (pre-registration)	set up; S4 multi-turn and S5 in progress; cluster pending **Update 2026-10-26: CoE HPC account active and the group's Slurm association fixed; first own cluster job is onboarding step 3 or 5 (Table 37b, M2).**
Yashashav Devalapalli Kamalraj	Personal laptop (CPU build of llama.cpp; model to be recorded at S7)	PyTorch, Transformers, Accelerate, llama.cpp conversion tools, uv, Docker, Singularity	CoE HPC account requested 2026-09-30, pending; GitHub; Hugging Face (write token); Google Cloud project with credits and billing account (L4 quota to be requested in Sprint 3); Colab and Kaggle (fallback only)	Repository layout committed (2026-10-08); pilot spikes run on a Colab A100 as backup owner; cluster pending **Update 2026-10-26: cluster jobs run on P100, A100 and H100; llama.cpp and the model files built in /scratch.**
Shared	CoE HPC cluster	Pinned llama.cpp builds and Singularity image in a shared project directory	Access for all three members for one year (capstone), requested through the advisor (agreed 2026-10-01, filed 2026-10-11)	Advisor agreed 2026-10-01 and filed the request 2026-10-11; student IDs sent to the

Member	Machine	Role-specific tools	Accounts and quotas	Status
				course instructor 2026-10-12; activation pending. Colab used as the fallback (2026-10-08) **2026-10-26: jobs run under Slurm account coe (P100, A100 and H100 tested); group folders /scratch/cmpe295-bertin and /data/cmpe295-bertin; Colab is the fallback only.**

Hardware Prototyping and Simulation Environment

The project has no custom hardware. Its “prototyping hardware” is the cluster the models run and train on, and its “simulation environment” is the set of deterministic benchmark environments the agents act in.

Measurement and training cluster: SJSU CoE HPC

Every reported number comes from a named CoE HPC node. The cluster page (updated 4 September 2026) does not publish GPUs per node, H100 and A100 memory, storage quotas, or per-user GPU caps, and gives the GPU time limit as both 7 days and 48 hours [93]. Spike S10 records the real values from `sinfo`, `scontrol`, and `nvidia-smi` once access is granted; until then we size jobs to 48 hours. **Measured 2026-10-26: gpuqs allows 2 days 1 hour and gpuqm 7 days 1 hour, with no default walltime, so jobs size themselves to the partition and requeue (Table 19a).**

Status on 2026-10-26. Accounts for all three members were activated on 2026-10-24: login over the VPN works and each account is in the Unix groups cmpe261-fa26 and cmpe295-bertin. Every job submission failed because the accounts had no Slurm account association; we reported it to the administrator on 2026-10-24, who fixed it on 2026-10-26. The same day, test jobs on the gpuqs partition (srun, nvidia-smi) ran on a P100 (12 GB), an A100 (40 GB) and an H100 under the coe account (QOS normal, no per-user job caps); the measured partitions and limits are under Table 33. The other two members' accounts are checked in onboarding step 1.

Table 33. CoE HPC node types as published on 2026-09-30; entries marked “verify” are measured in S10.

Partition	Nodes	GPU	GPU memory	RAM	Time limit (page)	Our use
gpu	g2, g6	H100	verify (80 GB expected)	256 GB	7 days or 48 h	R3 training (C8)
gpu	g7, g13	A100	verify	256 GB	7 days or 48 h	R3 training fallback; FR-O5
gpu	g1, g3–5, g9, g11–12, g14–16	P100, 12 GB shown in the page’s example	12 or 16 GB, verify	256 GB	7 days or 48 h	All correctness runs on CUDA; RQ4 cost block
compute	c1–14, c18–20	none	—	128 GB	12 days or 24 h	CPU-backend runs; C6 and C10 jobs
condo	condo3–4, 6–7	varies	—	—	21 days, preemptible	Not used

Measured on 2026-10-26 (replaces the “verify” entries). The GPU nodes are reached through partitions gpuqs (at most 2 days 1 hour per job) and gpuqm (at most 7 days 1 hour): 15 P100 nodes (12 GB), 4 nodes with 4 A100 PCIe each (40 GB) and 3 H100 PCIe nodes (80 GB; driver 590.44). CPU partitions: defq (6 hours), cpuqs (5 days 1 hour), cpuqm (14 days 1 hour), cpuql (21 days 1 hour). No partition sets a default walltime, so every job sets --time. CUDA comes from the modules cuda11/24.11 and cuda12/24.11. Slurm sets CUDA_VISIBLE_DEVICES to the assigned GPU, but nvidia-smi inside a 1-GPU A100 job lists all four, so jobs rely on that variable and log it. The group has shared directories /scratch/cmpe295-bertin (12 TB free on the scratch file system) and /data/cmpe295-bertin (3.8 TB free); the home file system is 100% full, so model files, environments and results go to /scratch/cmpe295-bertin. The login node reaches Hugging Face, GitHub and PyPI; the compute nodes have no outbound access, so downloads run on the login node and builds, conversions and runs run in jobs (docs/cluster.md in the repository).

What fits. The largest BF16 reference, Qwen3-4B, needs 8.0 GB of weights plus about 2.4 GB of KV cache for a 16k-token conversation (2 × 36 layers × 8 KV heads × 128 dimensions × 2 bytes ≈ 0.15 MB per token). That leaves under 2 GB for buffers on a 12 GB P100, so on 12 GB nodes this one reference runs with one slot and a 12k-token context (checked in S1); on 16 GB nodes it fits at 16k. Every 4-bit and 8-bit variant and both step-selective pairs fit easily. Training memory is sized in Chapter 3; if the H100 nodes have 40 GB rather than 80 GB, Qwen3-1.7B uses an 8-bit optimizer state.

Table 34. Approximate weight memory at 16 bits and at 4.5 bits per weight (stored parameter counts).

Model	Stored parameters	BF16 weights	Q4_0 weights	BF16 reference runs on
Qwen3-0.6B	0.75 B	1.5 GB	0.42 GB	Any P100
LFM2.5-1.2B-Instruct	1.17 B	2.3 GB	0.66 GB	Any P100
Qwen3-1.7B	2.03 B	4.1 GB	1.1 GB	Any P100
LFM2.5-2.6B	2.70 B	5.4 GB	1.5 GB	Any P100

Model	Stored parameters	BF16 weights	Q4_0 weights	BF16 reference runs on
SmolLM3-3B	3.08 B	6.2 GB	1.7 GB	Any P100
Qwen3-4B	4.02 B	8.0 GB	2.3 GB	P100, one slot

Memory is parameters × bits ÷ 8, in GB of 10^9 bytes, weights only. The Hugging Face page for Qwen3-0.6B shows 0.8B because it counts the 0.16B embedding, which the model name leaves out. Q4_K_M files are about 10% larger than Q4_0 (about 4.9 bits per weight [1]); for Qwen3-0.6B the measured gap is 13% (5.15 against 4.57 bits per weight, whole file).

Throughput. No published llama.cpp figure exists for the P100, so the episode times in Chapter 3 are assumptions that S1 replaces with measurements, and the M2 pilot measures how many concurrent jobs the queue sustains (NFR-5).

Access and limits. Jobs are submitted from the login node (campus or VPN only) and monitored with `squeue`; the C11 registry, not the shell session, holds the state. Measured on 2026-10-26: no partition sets a default walltime, and the per-job maxima are under Table 33. **Every job therefore sets --time and checkpoints at least every 30 minutes.**

Development machines

Members' laptops run the CPU build of llama.cpp for unit tests, spikes, and replays from the cache. They never produce reported numbers. Ekant's Apple M4 MacBook (16 GB) also serves the Expo demo (AD-8) with the CPU backend.

Simulation environment

The agents act in the benchmarks' own simulated environments. BFCL multi-turn fixes its user turns in the dataset and scores the final state of a simulated backend, so it replays deterministically [52]. τ²-bench-verified uses an LLM-simulated user and, on 58 tasks, an LLM judge [57]. We serve both from the open-weight model on the GCP VM (Chapter 3) behind the Simulated-User Cache (C4), so a recorded episode replays free.

In M1 the simulated user and the NL judge ran on a Colab L4 next to the agent (gpt-oss-20b, MXFP4, greedy), behind the C4 cache. Planned change from M2: the simulator runs as an A100 job inside the cluster (Table 19a); the GCP L4 VM is the fallback.

Sample Programs and Technology Spikes

Each spike is a short script in `spikes/` with a README giving the command, the expected output, and what we learned. A spike is done only when the component's backup owner reruns it under their own cluster account, so at least two people know each technology.

Table 35. Technology spikes, one per technology the prototype depends on; status as of 2026-10-16, with S1 and S10 updated on 2026-10-26, and S4, S5 and S8 on 2026-10-29.

ID	Spike	Technology exercised	Owner	Success criterion	Status
S1	Build llama.cpp for CUDA	CMake,	Ekant	Same tool call parsed	Ran on

ID	Spike	Technology exercised	Owner	Success criterion	Status
	and CPU on the cluster; serve Qwen3-0.6B Q4_K_M in `llama-server` on a P100 and on a compute node; issue one tool call through the raw `/completion` endpoint	CUDA modules, Slurm `srun --gres=gpu:1`		on both backends; `timings` and `nvidia-smi` memory logged; `llama-server --version` recorded	Colab (dev), 2026-10-08; cluster rerun pending **Update 2026-10-26: passed on the cluster on a P100, an A100 and an H100 (Table 37); owner rerun by Ekant at M2.**
S2	Same request on Ekant's laptop with the CPU build, for the development path	llama.cpp CPU, uv environment	Ekant	Token IDs identical to S1; tokens/s recorded	Criterion met on Colab CPU; laptop rerun in progress
S3	Constrain tool-call output with a GBNF grammar on both backends; show that the chat endpoint with `tools` applies a lazy grammar and the raw endpoint does not	GBNF, `json_schema`, server logs	Ekant	100 sampled calls at temperature 1.0 all validate on each backend; server log shows no grammar on raw-endpoint baseline requests	Ran on Colab A100 (dev); CPU half in progress
S4	Run 5 BFCL multi-turn tasks through the official evaluator against a local `llama-server` (laptop CPU build now, cluster later)	`bfcl-eval` (`--skip-server-setup`, `LOCAL_SERVER_ENDPOINT`), OpenAI-compatible endpoint	Vritika	`bfcl generate` and `bfcl evaluate` complete; our parser agrees on all 5 tasks	Single-turn ran (dev); multi-turn in progress **Update 2026-10-29: multi-turn ran through C1 in M1 (20 tasks, Table 36a) and on the cluster (200 tasks,**

ID	Spike	Technology exercised	Owner	Success criterion	Status
					Table 36b); the 5-task agreement check against the official evaluator is still open (M2).
S5	Serve the three candidate simulator models (gpt-oss-20b, Qwen3-30B-A3B-Instruct-2507, Gemma 4 26B-A4B) on a GCP L4 with `llama-server`; run 3 τ²-bench-verified airline tasks with `--user-llm openai/<model>` and the judge pointed at the same server, through the cache, then replay	tau2 CLI, LiteLLM custom base URL, `llama-server`, GCP Spot VM, SQLite cache	Vritika	Both roles answer from the VM; replay makes zero server calls; two greedy runs give byte-identical user turns; one candidate chosen and its file hash recorded	In progress **Update 2026-10-29: gpt-oss-20b served the τ² runs at M1 as a placeholder (Colab L4); the choice among the three candidates is made at M2.**
S6	Sample NVML power, clocks, and memory around one generation on a P100 node (shared; the exclusive-allocation request is DEP-5)	`pynvml`, `--exclusive`, step timestamps	Ekant	Energy per token after idle subtraction; spread over 10 repeats reported; clock variation across repeats recorded	Ran on Colab (indicative); P100 rerun pending
S7	50-step toy QAD run on one H100 or A100 node in BF16, killed at step 25 by a 10-minute Slurm limit and resumed	PyTorch, Transformers, Accelerate, Slurm requeue, checkpoints on the shared file system	Yashashav	Resumed loss curve matches an uninterrupted run; peak GPU memory recorded; GPU memory size of the node recorded	Ran on Colab (dev); Slurm version pending
S8	Export a fine-tuned checkpoint to GGUF Q4_0 with `--pure` by writing the trained codes directly; compare logits on both backends	`convert_hf_to_gguf.py`, `llama-quantize`, `llama-perplexity`, llama-cpp-python	Yashashav	Tensor types as intended; export-vs-fake-quant KL small next to BF16-vs-4-bit KL (Chapter 7 audit) on CUDA and CPU	In progress **Update 2026-10-29: the toy QAD export ran in M1**

ID	Spike	Technology exercised	Owner	Success criterion	Status
					(196 tensors exact; KL to BF16 0.212 for Q4_0 and 0.133 for QAD Q4_0; Table 37).
S9	Two-process step-selective precision switch (Q4_0 and Q8_0 of one model, as in R2c) on one P100	Two `llama-server` instances, prompt caching, C3 routing, NVML memory	Yashashav	Each step equals the chosen copy run alone; extra prefill tokens per switch logged; resident GPU memory of the pair recorded	Planned, Sprint 4
S10	Slurm array of 8 evaluation jobs on P100 and compute nodes running 100 BFCL episodes with a minimal file-based registry (the C11 skeleton); one job killed by walltime and resumed; cluster limits and policies recorded	`sbatch` arrays, C11 registry, `sinfo`, `scontrol`, outbound access test from a compute node, login-node process policy	Ekant	No missing or duplicate episodes; concurrent-job count, walltime limit, per-user GPU cap, GPUs and GPU memory per node, compute-node internet access, and whether a long-running proxy may run on the login node (else it runs in a compute-node job) written to `docs/cluster.md`	Waiting for accounts **Update 2026-10-26: limits measured (Table 33 note); the 8-job array is onboarding step 5 (M2).**

S3 matters because the chat endpoint's lazy grammar would partly repair an "unconstrained" baseline. S9 measures the main engineering cost of step-selective precision: the second process's memory and the tokens re-prefilled at each switch. S10 replaces cluster assumptions with measurements before the walking skeleton depends on them.

Pilot Results, October 2026 (Development Data)

Before cluster accounts were available, the backup owner ran spikes S1 to S3, the single-turn part of S4, S6 and S7 on Google Colab, using an A100-SXM4-40GB GPU (driver 580.82.07, CUDA 13.0) on 2026-10-08, applying the DEP-4 fallback. Everything used llama.cpp release v0.6.0 (commit d812350), built for CUDA and CPU, Qwen3-0.6B converted to BF16 GGUF from the original checkpoint, and our own Q4_0 `--pure` and Q4_K_M files. BFCL tasks were run and scored by the official `bfcl-eval` 2026.3.23. These are development data: one model, one GPU type, and no exclusive allocation. They test the tooling and replace planning figures, and they are excluded from the study.

Under the spike rule above, these runs do not close a spike: each still needs its owner or backup to rerun it under a cluster account.

The same day we ran a diagnostic deep dive (thinking mode, third-party GGUF files, prompt caching, and grammar design) and then reran everything under one pinned protocol ("v2"): the unmodified evaluator, our own wrapper for the non-thinking prompt, prompt caching off on every request, greedy decoding plus 4 sampled trials at the model card's settings, and the full BFCL non-live group. Two gates had to pass before any number was used: the server honoured the `grammar` and `cache_prompt` fields, and v2 reproduced the earlier cache-off outputs byte for byte (400 of 400 tasks per format). Table 35a reports the v2 numbers.

Table 35a. Pilot results for Qwen3-0.6B (Colab A100, 2026-10-08). Accuracy rows: pinned v2 protocol; size, speed, memory and energy rows: spikes S1 and S6.

Measure	BF16	Q4_K_M	Q4_0 --pure
Bits per weight (whole file)	16.06	5.15	4.57
BFCL simple_python accuracy, greedy (n = 400)	83.75%	83.25%	68.50%
BFCL non-live overall, greedy (1,150 call tasks)	69.52%	66.81%	56.38%
Non-live overall, 4 sampled trials, mean ± sd	68.73 ± 0.55	65.32 ± 0.99	56.58 ± 1.65
Paired flips vs BF16 on 1,150 tasks (right→wrong / wrong→right); exact McNemar p	—	72 / 47; p = 0.027	193 / 33; p < 10^{-27}
Irrelevance, correct no-call (n = 240)	88.33%	86.25%	92.50%
simple_python: no tool call / other format failures	11 / 3	8 / 6	57 / 9
Non-live overall with R1 grammar (calls or free text)	70.42%	69.29%	58.69%
Irrelevance with a forced-call grammar (ablation)	0.42%	0.42%	0.00%
simple_python, thinking mode at model-card settings	86.00%	85.00%	71.00%
Decode tokens/s, A100 (1,024 tokens, 10 runs)	291.5	342.2	379.3
Peak GPU memory, MiB (context 8,192)	3,096	2,316	2,262
GPU energy per decoded token, mJ (mean ± sd, 10 runs)	348.6 ± 14.3	185.2 ± 9.5	134.0 ± 6.4

Dispersion (advisor review, 2026-10-16). Greedy rows are deterministic, so their SD across reruns is 0 (v2 reproduced 400 of 400 outputs byte for byte); their uncertainty is over tasks, with 95% Wilson intervals: simple_python (n = 400) BF16 83.75% [79.8–87.0%], Q4_K_M 83.25% [79.3–86.6%], Q4_0 68.50% [63.8–72.9%]; irrelevance (n = 240) 88.33% [83.7–91.8%], 86.25% [81.3–90.0%], 92.50% [88.5–95.2%]. Sampled rows are mean ± 1 SD over the 4 trials: simple_python 82.44 ±

1.14, 82.50 ± 0.74 and 68.75 ± 1.57; non-live overall 68.73 ± 0.55, 65.32 ± 0.99 and 56.58 ± 1.65 (BF16, Q4_K_M, Q4_0).

bfcl's "non-live overall" is the unweighted mean of four category scores over the 1,150 tasks that need a call; the 240 irrelevance tasks, where the right answer is no call, are scored separately. Peak memory is device-level and includes a 518 MiB baseline present before the server started. Energy is idle-subtracted from the NVML energy counter, with unlocked clocks on a shared VM, so it is indicative only.

Findings. Five results matter for the study design.

- **The published Q4_K_M collapse did not reproduce.** Taheri et al. report Qwen3-0.6B falling from 62.3% to 16.8% on BFCL V3 non-live at Q4_K_M [19]. In our harness Q4_K_M lost 2.7 points (detectable over 1,150 paired tasks, p = 0.027), and it did not collapse in thinking mode or with three third-party Q4_K_M files (unsloth 67.92%, bartowski 69.00%, Ollama 66.40% non-live). The remaining unmatched variables are that paper's prompt format, harness and BFCL version. So the damage may depend on format and prompting, which strengthens RQ1: the study should measure where the drop appears rather than assume it.
- **Q4_0 `--pure` loses 13 points, mostly by not calling the tool.** On simple_python, 57 of its failures are prose answers instead of calls (BF16: 11). We hand-labelled the 58 outputs the earlier classifier counted as no call (single rater, second rater pending): 49 answer the question directly, 26 of them with wrong or invented facts; 5 ask the user for details; 3 say they will call the tool but do not; 1 is a call attempt the parser could not read. This tool avoidance with a confident direct answer also explains why Q4_0 scores higher than BF16 on irrelevance. We propose it as its own class in codebook v0 (C6, FR-E8), for decision at M2.
- **R1 needs its free-text branch, and that limits what it can fix.** A grammar that forces a call lifted Q4_0 simple_python from 68.50% to 82.25% but scored 0 to 0.42% on irrelevance. The R1 design in Table 12 (calls or free text) removed all but one malformed, unknown-tool or schema-invalid call on simple_python across the three formats (Figure 5a) and adds 0.9 to 2.5 points on non-live overall, but it cannot stop Q4_0 answering in prose. This supports H2: R1 fixes format failures, not semantic ones. Q4_K_M with R1 (69.29%) comes within a quarter point of BF16 without it (69.52%).
- **Greedy decoding alone is not reproducible across server settings.** With prompt caching on, 61 of 400 greedy Q4_0 outputs changed; with it off, outputs reproduced byte for byte across runs and endpoints. NFR-2 already requires the caching setting to be fixed; we now pin it off and record it in every run manifest (C11).
- **The tooling works end to end.** llama.cpp builds for CUDA and CPU, the CPU build returns the same output tokens as CUDA for all three files (S2), the chat endpoint's lazy grammar is confirmed and avoided (S3), NVML energy per token is measurable (S6), and QAD training resumes from a checkpoint with an identical loss curve (S7).

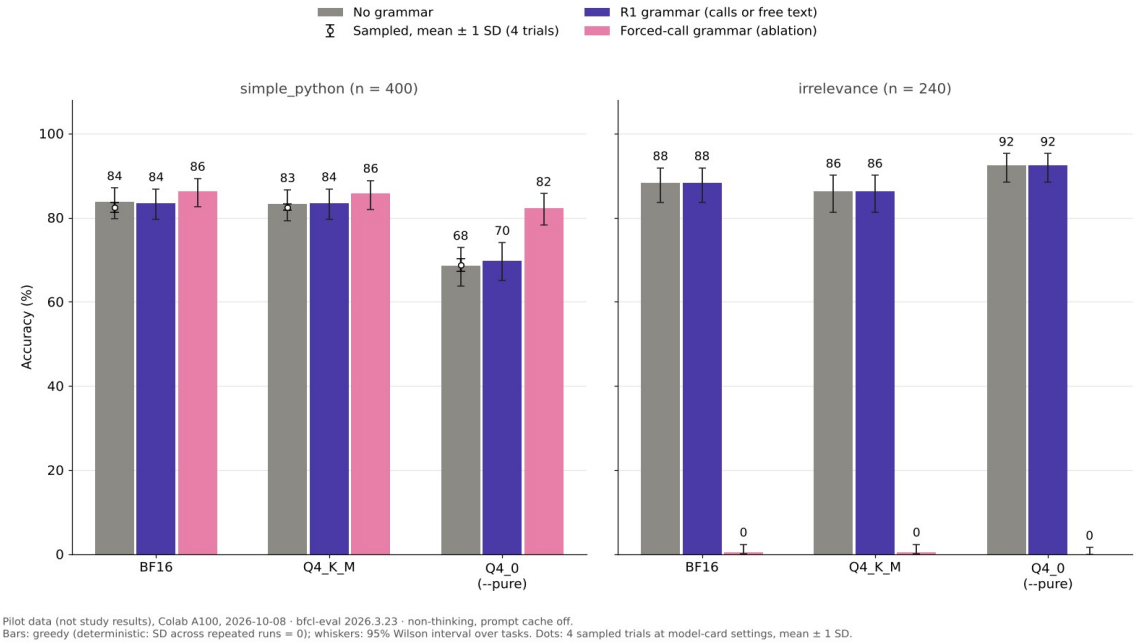


Figure 5a. Grammar effect on Qwen3-0.6B (pilot data, Colab A100, 2026-10-08). Left: BFCL simple_python (n = 400). Right: irrelevance tasks (n = 240), where the correct answer is not to call a tool. The forced-call grammar is an ablation, not repair R1. ***Updated after the 2026-10-16 review: whiskers are 95% Wilson intervals over tasks; greedy bars are deterministic (SD = 0 across reruns), so the sampled trials (white dots, mean ± 1 SD over 4 trials, no grammar) show the run-to-run spread.***

Table 35b. Design consequences of the pilot.

Finding	Consequence for the design	Owner; when
Q4_K_M is 5.15 bits per weight on Qwen3-0.6B	Log measured whole-file bits per weight for every variant; use measured, not nominal, sizes in RQ4	Yashashav; from M2
Qwen ships only Q8_0 GGUF for Qwen3-0.6B	Q4_K_M baseline described as the third-party format; every file rebuilt from the original checkpoint (unchanged)	Yashashav; done in Version 2
Prompt caching changes greedy outputs	Caching off on every request and recorded in the run manifest; noise-floor runs (NFR-2) use the same setting	Ekant (C3), Yashashav (C11); M1
Forced-call grammar breaks irrelevance	R1 keeps its free-text branch; every R1 result reported with irrelevance; forced-call kept only as a labelled ablation	Ekant (R1); M1
Tool avoidance with a direct answer accounts for most of Q4_0's extra failures	Proposed new C6 class (missing call with a direct answer), plus a separate "tag spacing" check so a call written as `<tool_call> {` is not counted as no call	Vritika (C6); codebook v0 at M2
bfcl's Qwen handler builds a thinking-mode prompt	Non-thinking mode set by a handler subclass in our repository, not an edit to the installed package; thinking mode recorded per run spec	Vritika (C1), Yashashav (C11); M1
Schema check approximate	Extend the type map before the schema-invalid	Vritika (C6); M2

Finding	Consequence for the design	Owner; when
for Java and JavaScript types	versus wrong-value split is used in analysis	
Colab energy figures are indicative only	RQ4 numbers come only from the exclusive cluster cost blocks with logged clocks (DEP-5, unchanged)	Ekant (C9); M6

Limits of the pilot. One model, one GPU type, and one run per greedy condition; per-task latencies from runs that shared the GPU are not timing measurements; the Q4_0 no-call labels have a single rater so far; and the spike rule in this chapter still applies: a spike counts as done only when its owner or backup reruns it under their own cluster account. The pilot records (status log, per-task CSVs, figures, and notebook) are kept with the repository.

Prototype Implementation Plan

Walking skeleton

The first prototype is a walking skeleton: the thinnest end-to-end path that passes through all eleven components and all three repair families on the cluster. Its purpose is to show that every technology connects before any of them is scaled up. It is milestone M1 (target 2026-10-23). A single command, `uv run python -m testbed.run +experiment=skeleton`, submits one configuration as Slurm jobs: Qwen3-0.6B at BF16 versus Q4_K_M on the CUDA backend and Q4_0 on both backends, on 20 BFCL base multi-turn tasks and 3 τ²-bench-verified airline tasks, with 2 seeds, followed by one minimal variant of each repair. The command writes a report. Its numbers are development data and are excluded from the study. If cluster access is not granted by Sprint 4, the same command runs with `local=true` on the CPU backend of a laptop, and the cluster-specific exit criteria move to M2.

Table 36. Minimal version of each component in the walking skeleton.

Component	Minimal version in the skeleton	Owner
C1 Benchmark Adapter	Loaders for the 23 tasks, tool schemas normalized to one JSON-schema form	Vritika
C2 Episode Runner	Synchronous turn loop (model, tool, user) with fixed seeds	Ekant
C3 Inference Backends	`llama-server` on CUDA and CPU behind one `generate(messages, tools, constraint)` call	Ekant
C4 Simulated-User Cache	Caching proxy in front of the GCP simulator VM; VM-hour counter **Ran on a Colab L4 at M1; the A100 cluster job from M2 (Table 19a).**	Vritika
C5 Step Logger	One JSONL record per step, converted to Parquet after each run	Ekant
C6 Failure Classifier	Rules for hallucinated tool, malformed call, invalid arguments, premature termination; 50 hand-labeled steps **Update 2026-10-29: at M1 a 30-step blind sample was rated by an AI rater (κ 1.00, Table 37); human labels follow at M2.**	Vritika
C7 R1	GBNF grammar for tool calls on both backends	Ekant
C7 R2	Uniform Q8_0 variant plus the two-process step-selective switch (from S9) **Update 2026-10-29: the S9 switch was not built for M1**	Yashashav

Component	Minimal version in the skeleton	Owner
	and moves to M2.	
C7 R3 / C8	Toy QAD checkpoint from S7, exported through the S8 path	Yashashav
C9 Cost Profiler	TTFT, tokens/s, peak GPU memory, and NVML energy for one episode	Ekant
C10 Analysis & Stats	Draft-plan tests on toy data, plus a planted-effect check on synthetic data	Vritika
C11 Config & Registry	One Hydra config; run registry keyed by config hash; one Slurm array job template	Yashashav

What ran (2026-10-24 to 2026-10-26). Cluster jobs were blocked, so M1 ran with local=true on Colab, as the fallback rule allows: an A100 for the 180 CUDA BFCL episodes, two 2-vCPU CPU runtimes for the 40 CPU-backend episodes, and an L4 for the 33 τ² episodes, with the simulated user on the same GPU. The one command expands to 253 run specs (220 BFCL, 33 τ²); 252 finished and one τ² episode is pending after the simulator returned an empty turn. Table 36a gives the results; all of them are development data.

Table 36a. M1 results: episodes valid out of episodes run (Qwen3-0.6B, non-thinking, development data).

Configuration	**BFCL multi-turn base (20 tasks)**	**τ²-bench-verified airline (tasks 7, 8, 11)**
BF16, CUDA	**0 of 40**	**0 of 6**
Q4_K_M, CUDA	**0 of 40**	**0 of 6**
Q4_0, CUDA	**0 of 40**	**0 of 6**
Q4_0, CPU	**0 of 40**	**0 of 6**
R1: Q4_0 + GBNF grammar, CUDA	**0 of 20**	**0 of 3**
R2: Q8_0, CUDA	**0 of 20**	**0 of 3**
R3: toy QAD Q4_0, CUDA	**0 of 20**	**0 of 2 (1 pending)**

Dispersion. Every cell is 0 valid episodes, identical in both seeds, so the SD across seeds is 0 and the useful interval is the 95% Wilson upper bound on the episode success rate: 0 of 40 → 0.0–8.8%, 0 of 20 → 0.0–16.1%, 0 of 6 → 0.0–39.0%, 0 of 3 → 0.0–56.2%, 0 of 2 → 0.0–65.8%. These bounds are why M2 moves the primary outcome to the step level (decisions, Chapter 8).

Episode accuracy is 0% everywhere, so M1's evidence is per step. On BFCL, missing calls rise from 6 (BF16) to 12 (Q4_K_M) and 29 (Q4_0), and malformed calls from 0 to 6; R1's grammar removes every invalid-argument step (10 to 0). Q4_0 output is identical on CPU and CUDA in only 32% of aligned steps, and even the first step of an episode matches in only 19 of 40 episodes, so the backend is a recorded factor and the FR-E17 parity test needs greedy decoding. A probe on 20 tasks gave Qwen3-4B 2 of 20 at BF16 and at Q4_0, and Qwen3-0.6B in thinking mode hit the step limit in 12 of 20 Q4_0 episodes. The first τ² task choice (airline tasks 0 to 2) let a do-nothing agent score 29 of 30, because those tasks need no

database write; the skeleton now uses only tasks whose gold actions write to the database.

Damage runs on the cluster (***2026-10-28***)

M1's 20 tasks were too few, and too hard at 0.6B, to show a difference between formats. On 2026-10-28 the team reran the skeleton's BFCL configurations on all 200 BFCL multi-turn base tasks on the cluster (4,800 episodes, 34,920 steps; four configurations at a time on a 4-GPU A100 node): BF16, Q4_K_M and Q4_0 on CUDA with 2 seeds, and R1 (grammar on Q4_0) and R2 (Q8_0) with 1 seed, same decoding as M1. Figure 5b and Table 36b give the results. The configuration for these 200-task runs (Qwen3-0.6B, 1.7B and 4B) is not yet committed to main; it is added at M2 so that the runs can be repeated. For Qwen3-0.6B, missing calls per 100 steps rise from 5.9 [4.7–7.4] (BF16) to 11.4 [9.3–13.7] (Q4_K_M) and 31.0 [26.8–35.5] (Q4_0); the intervals do not overlap. R1 removes schema-invalid calls (5.1 [3.3–7.3] to 0.2 [0.0–0.4]) but not missing calls (34.8 [29.8–40.4]), as H2 predicts; R2 brings missing calls back to 7.2 [5.5–9.1], close to BF16. Episode accuracy stays low at 0.6B (Table 36b), so the step-level outcome is the one that separates the formats. Brackets are 95% bootstrap intervals over tasks (2,000 resamples); these are development data, not the pre-registered study. Figure 5c repeats the comparison for Qwen3-1.7B and Qwen3-4B (same 200 tasks and settings), to show how episode accuracy and the missing-call damage change with model size. Valid episodes rise with size (BF16: 16, 26 and 47 of 400) and the share of turns answered in text falls, but the order of the 4-bit formats is not fixed: at 4B, Q4_K_M answers in text on 11.8% of turns against 7.1% for Q4_0 and 5.8% for BF16. The study therefore measures each format and model rather than assuming that more bits per weight mean less damage.

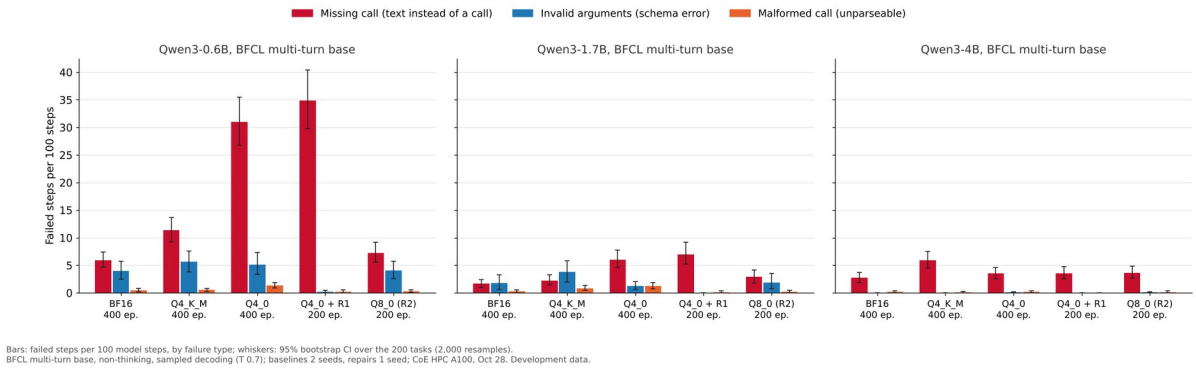


Figure 5b. Failed steps per 100 model steps by failure type, BFCL multi-turn base (200 tasks), CoE HPC, 2026-10-28. Whiskers: 95% bootstrap interval over tasks. Development data.

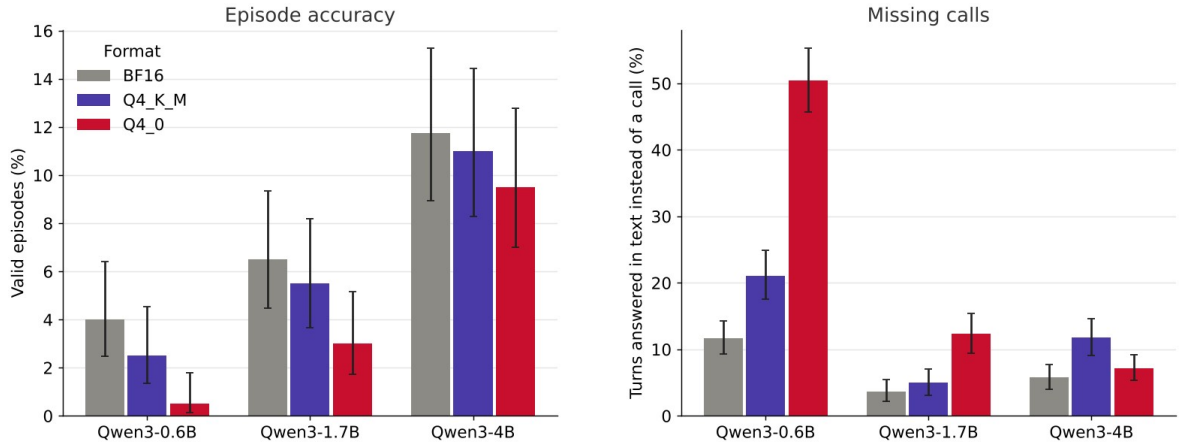


Figure 5c. Episode accuracy (95% Wilson interval) and the share of user turns answered in text instead of a call (95% bootstrap interval over tasks) by model size and format; BF16, Q4_K_M and Q4_0, 2 seeds, 200 tasks.

Table 36b. Damage runs on the cluster: valid episodes, and missing and invalid-argument steps per 100 steps with 95% bootstrap intervals (2026-10-28).

Model	Configuration	Valid episodes	Steps	Missing call /100	Invalid args /100	All failures /100
Qwen3-0.6B	**BF16**	**16 / 400**	**2,856**	**5.9 [4.7–7.4]**	**4.0 [2.5–5.7]**	**11.9 [9.9–14.2]**
Qwen3-0.6B	**Q4_K_M**	**10 / 400**	**2,700**	**11.4 [9.3–13.7]**	**5.6 [3.8–7.6]**	**19.6 [16.8–22.4]**
Qwen3-0.6B	**Q4_0**	**2 / 400**	**2,350**	**31.0 [26.8–35.5]**	**5.1 [3.3–7.3]**	**38.4 [34.0–43.1]**
Qwen3-0.6B	**Q4_0 + R1**	**2 / 200**	**1,163**	**34.8 [29.8–40.4]**	**0.2 [0.0–0.4]**	**36.3 [31.5–41.7]**
Qwen3-0.6B	**Q8_0 (R2)**	**6 / 200**	**1,414**	**7.2 [5.5–9.1]**	**4.0 [2.5–5.8]**	**12.7 [10.5–15.1]**

Qwen3-1.7B	**BF16**	**26 / 400**	**3,250**	**1.7 [1.0–2.4]**	**1.8 [0.6–3.3]**	**6.7 [4.8–8.8]**
Qwen3-1.7B	**Q4_K_M**	**22 / 400**	**3,202**	**2.2 [1.4–3.3]**	**3.8 [2.0–5.8]**	**9.7 [7.2–12.4]**
Qwen3-1.7B	**Q4_0**	**12 / 400**	**2,953**	**6.0 [4.6–7.7]**	**1.2 [0.6–2.0]**	**10.8 [9.0–12.9]**
Qwen3-1.7B	**Q4_0 + R1**	**8 / 200**	**1,504**	**7.0 [5.2–9.2]**	**0.0 [0.0–0.0]**	**8.8 [6.9–10.9]**
Qwen3-1.7B	**Q8_0 (R2)**	**17 / 200**	**1,584**	**2.9 [1.8–4.1]**	**1.9 [0.7–3.5]**	**7.4 [5.4–9.4]**
Qwen3-4B	**BF16**	**47 / 400**	**3,066**	**2.7 [1.9–3.7]**	**0.0 [0.0–0.0]**	**4.3 [3.2–5.4]**
Qwen3-4B	**Q4_K_M**	**44 / 400**	**2,913**	**5.9 [4.5–7.5]**	**0.0 [0.0–0.0]**	**7.1 [5.6–8.7]**
Qwen3-4B	**Q4_0**	**38 / 400**	**2,951**	**3.5 [2.5–4.6]**	**0.1 [0.0–0.2]**	**5.4 [4.2–6.6]**
Qwen3-4B	**Q4_0 + R1**	**21 / 200**	**1,487**	**3.6 [2.6–4.7]**	**0.0 [0.0–0.0]**	**5.1 [3.9–6.5]**
Qwen3-4B	**Q8_0 (R2)**	**20 / 200**	**1,527**	**3.7 [2.6–4.9]**	**0.1 [0.0–0.2]**	**5.4 [4.2–6.8]**

Reproducing M1, step by step

Every result in Table 36a can be rerun from the repository with the commands below (the clean-clone check in Chapter 7 does steps 1 to 5 on a fresh account).

1. Clone: git clone https://github.com/VritikaMalhotra/CMPE_295A.git (main holds everything M1 ran, including the cluster scripts; all component branches were merged on 2026-10-28).

2. Environment: bash scripts/setup_env.sh --llama --cuda (add --tau2 for τ²-bench-verified). On the CoE HPC, first source scripts/hpc_env.sh; the login node then only downloads, and sbatch slurm/build_llama.sbatch compiles llama.cpp for the P100, A100 and H100 in a job.

3. Model files: VARIANTS="BF16 Q4_K_M Q4_0-pure Q8_0" bash scripts/make_probe_models.sh Qwen/Qwen3-0.6B $TESTBED_MODELS (on the cluster: DOWNLOAD_ONLY=1 on the login node, then sbatch slurm/make_models.sbatch). The rebuilt BF16, Q4_K_M and pure Q4_0 files are byte-identical to the Oct 8 pilot's (SHA-256 checked by the run). τ² also needs scripts/get_simuser_model.sh for the simulated user.

4. Smoke test: sbatch slurm/smoke_s1.sbatch (one tool call through the testbed backend; Table 37).

5. Plan check: python -m testbed.run +experiment=skeleton dry_run=true must report 253 specs in skeleton/3e3a25e70f.

6. Run: add local=true to run here (Colab, laptop); without it the command writes a Slurm array and a report job (submit=true also submits them), each array task requeuing itself before its time limit and resuming.

7. Report and replay: report_only=true rebuilds report.md from the finished episodes; replay=<τ² run id> reruns an episode from the C4 cache and must make

zero simulator calls. The notebooks in notebooks/ wrap these steps for Colab, and docs/cluster.md is the cluster runbook.

Incremental build by milestone

After M1, each milestone widens one layer of the skeleton instead of building a new system. Exit criteria are in Table 38.

- **M1 Walking skeleton (2026-10-23).** The skeleton above.
- **M2 References and frozen plan (2026-11-06).** Full C1 coverage; BF16 GGUF conversions of all models; repeated reference runs on CUDA and FR-E17 parity runs on CPU; the inclusion filter; the NFR-5 throughput pilot; codebook v0 and the OSF pre-registration.
- **M3 Damage map v1 (2026-12-04).** Q4_0 (`--pure`), Q4_K_M, and IQ4_XS (imatrix) variants of every model, each KL-checked against BF16 on both backends; Slurm array execution; codebook v1 with the κ study; step-type and depth annotations.
- **M4 R1 and R2 (2026-12-18).** GBNF grammars per tool-call format (JSON for Qwen3 and SmolLM3, Pythonic for LFM2.5); Q5_0, Q6_K, and Q8_0; the 5.5-bit layer-selective variant (`--tensor-type`); the two-process step-selective variant. Repository v0.1, the lockfile, Dockerfile, and Singularity recipe are tagged at the end of CMPE 295A.
- **M5 R3 (2027-01-29).** QAD-KL, QAD-KL+CE, and the train-longer control for the three core models on H100 or A100 nodes, exported to GGUF Q4_0 and checked with the S8 logit test.
- **M6 Costs (2027-02-05).** The C9 protocol on exclusive P100 and compute nodes for every baseline and repair, plus R3 GPU-hours. Repository v0.5 is tagged with the paper at M7.

Progress Tracking

We track progress with four mechanisms:

- **GitHub Projects board.** Each task in the Chapter 9 sprint plan is one issue with a single assignee, a component label (C1 to C11, R1 to R3), and a milestone (M1 to M8). The board has Backlog, Sprint, In progress, Review, and Done columns. Each pull request is reviewed by the component's backup owner (Chapter 9, Task Assignment Summary), so no component depends on one person.
- **Sprint cadence.** Sprints run for two weeks, Monday to Sunday. A 30-minute sprint review on the final Friday demonstrates what runs, updates the board, and plans the next sprint.
- **Biweekly advisor meeting** with Dr. Bertin Cordova-Diba; the first review, planned for 2026-10-14, was moved to 2026-10-16 with the advisor's agreement. It covers progress on the board, blockers, and decisions. Decisions go into the meeting notes in `meetings/`, and deviations from the pre-registered plan are logged there with their reasons.
- **Burn-down checks.** We keep three running counts: (1) milestone exit criteria, one checkbox each; (2) planned versus completed runs, drawn from the C11 registry; (3) budget burn, meaning simulated-user dollars from C4 and cluster GPU-hours from Slurm accounting, each against its weekly plan.

*Table 37. Progress log (as of **2026-10-29; rows from 2026-10-24 are new**).*

Date	Item	Owner	Status
September 2026	Project abstract completed	All	Done
September 2026	Abstract presentation completed	All	Done
Sprint 1 (by 2026-09-13)	Scoping literature review (runtimes, benchmarks, repairs)	All	Done
Sprint 2 (by 2026-09-27)	Division of work agreed	All	Done
Sprint 2 (by 2026-09-27)	GitHub repository and Projects board created	Ekant	Done

Date	Item	Owner	Status
Sprint 2 (by 2026-09-27)	Laptop development environment and CPU build (S2); S2 criterion met on the Colab CPU build on 2026-10-08	Ekant	In progress
Sprint 2 (by 2026-09-27)	`bfcl-eval` and τ²-bench-verified environments (S4)	Vritika	In progress
Sprint 3 (by 2026-10-11)	GCP project, credits, L4 quota request (carried into Sprint 4; see below)	Yashashav	Not started
2026-09-30	Hardware plan changed from a personal Apple laptop to the CoE HPC cluster; CoE HPC access requested through the advisor for all three members	All; Dr. Cordova-Diba	Requested; pending
2026-09-30	Study experiments (pilot data excluded)	All	Not started
2026-10-01	Advisor agreed to request CoE HPC accounts for all three members; first advisor review scheduled	Yashashav; Dr. Cordova-Diba	Done
2026-10-08 to 10-09	Repository scaffolded with the workbook layout; pilot spike code committed and pushed to the team repository (branch attributed-history)	All (see CONTRIBUTORS.md)	Done
2026-10-08	Pilot P1 to P4 and S7 on a Colab A100 (Tables 35 and 35a)	Yashashav (backup owner)	Done; development data
2026-10-08	Deep dive: thinking mode, third-party GGUF files, prompt caching, grammar design, labels for the 58 Q4_0 no-call outputs	Yashashav	Done
2026-10-08	v2 rerun under one pinned protocol: full BFCL non-live, R1 and forced-call grammar, 4 sampled trials, thinking mode, third-party files	Code: Vritika; run: Yashashav	Done
2026-10-09	Workbook 1 presentation (14 slides) with pilot results; meeting with Prof. Gao	Yashashav; Vritika	Done
2026-10-09 to 10-12	CoE HPC: request through the course instructor, Prof. Gao (student IDs sent 2026-10-12); faculty request filed by the advisor (2026-10-11)	Vritika; Dr. Cordova-Diba	Requested; activation pending
Sprint 4 (from 2026-10-12)	Owner reruns of S1, S3, S6 from the repository	Ekant	In progress
Sprint 4 (from 2026-10-12)	S4 multi-turn check, C1 loaders, S5 simulator candidates	Vritika	In progress
Sprint 4 (from 2026-10-12)	S8 export check, C11 config skeleton, GCP L4 quota request	Yashashav	In progress
2026-10-24	**CoE HPC accounts active (login, groups); jobs blocked pending the Slurm association; request sent to the administrator**	**Yashashav**	**Done; unblocked 2026-10-26**
2026-10-24	**M1 interfaces frozen (RunSpec, StepRow, generate) on m1/interfaces**	**Yashashav; review: all**	**Done**

Date	Item	Owner	Status
2026-10-24	**Component drafts on review branches: C1, C6, C10 (m1/vritika-c1-c6-c10); C2, C3, C5, C9, R1 (m1/ekant-c2-c3-c5-c9)**	**Vritika (C1, C6, C10), Ekant (C2, C3, C5, C9, R1); cross-review**	**In review**
2026-10-24	**C11 config, registry and one-command runner; R2 Q8_0 variant; R3 toy QAD export (196 tensors exact; KL to BF16 0.212 for Q4_0, 0.133 for QAD Q4_0)**	**Yashashav**	**Done**
2026-10-24	**Skeleton on a Colab A100 (local=true): 180 CUDA BFCL episodes, report written**	**Yashashav**	**Done; development data**
2026-10-24 to 10-25	**40 CPU-backend episodes on two Colab CPU runtimes (Colab's 12-hour limit hit once; resumed); CPU speed table (llama-bench)**	**Yashashav**	**Done**
2026-10-24 to 10-25	**M2 model probe: Qwen3-0.6B thinking, Qwen3-1.7B, Qwen3-4B on 20 tasks**	**Yashashav**	**Done**
2026-10-25	**τ^2 airline (tasks 7, 8, 11) and C4 cache on a Colab L4; replay with zero simulator calls; do-nothing check on task choice**	**Ekant (τ^2 runner), Vritika (C4 cache); cross-review**	**Done; 1 episode pending**
2026-10-26	**Slurm association fixed by the administrator; test jobs ran on gpuqs on a P100, an A100 and an H100 (srun, nvidia-smi)**	**Yashashav**	**Done**
2026-10-26	**Cluster onboarding steps 1, 2 and 4 (Table 37b): limits measured; llama.cpp built for P100, A100 and H100 in a job; S1 tool call passes on a P100, an A100 and an H100; the rebuilt GGUF files are byte-identical to the Oct 8 pilot (m1/cluster)**	**Drafts: Yashashav; review: Ekant**	**Done; development data**
2026-10-27 to 11-06	**Cluster onboarding steps 3, 5 and 6 (Table 37b): bfcl-eval in a compute-node job, S10 array with a walltime requeue, S7 under a 10-minute limit; the other two members' accounts**	**Vritika, Ekant, Yashashav**	**Not started; Sprint 5**
2026-10-26 to 10-28	**Clean-clone rerun by a second member (M1 exit criterion)**	**Ekant (checks run 2026-10-27 by the team and an independent AI agent)**	**Passed twice on the CoE HPC on 2026-10-27. (1) Fresh clone with the shared build (job 97498). (2)**

Date	Item	Owner	Status
			Independent full rebuild by an AI agent in a private folder: llama.cpp compiled from source (job 97611, 21 min), all four model files rebuilt from Hugging Face and byte-identical to the pilot's (job 97612), then the check (job 97655): plan of 253 specs with the same hash 3e3a25e70f, 10 Q4_0 CUDA episodes on a P100, 60 steps none unclassified, 0 of 10 valid as in the reference. Both ran under one team account; Ekant repeats under his own
2026-10-27	**C6 second-rater check on 30 blind steps from 100 new cluster episodes (job 97528); AI rater, codebook gap found (tool-level errors)**	**Vritika (AI check: team)**	**κ 1.00 (AI rater); Vritika's labels due**
2026-10-26 to 10-28	**Review and merge the component branches to main, in order**	**All**	**Done: 5 of 5 merged (PRs #3 and #4 on**

Date	Item	Owner	Status
			2026-10-26; PRs #5 to #9 on 2026-10-28)
2026-10-29	**M1 plan items not delivered by M1 (Table 37a): CI on GitHub Actions; uv lockfile and Dockerfile; S9 two-process switch prototype**	**Ekant; Yashashav**	**Moved to M2**

Plan from October 16 to M2

The pilot showed that the toolchain works on CUDA and CPU and produced the design consequences in Table 35b. The next three weeks turn the spike scripts into the walking skeleton (M1, 2026-10-23) and then into the reference runs and frozen analysis plan (M2, 2026-11-06). Work proceeds on the Colab A100 and laptops until cluster accounts are active and moves to the cluster as soon as they are; because every run is described by a C11 run spec, the same command runs in both places and only the backend host changes.

Table 37a. Per-member plan from 2026-10-16 to M2.

Period	Ekant Kapgate	Vritika Malhotra	Yashashav Devalapalli Kamalraj
Oct 16–22 (to M1)	Rerun S1, S3, S6 from the repository; C2 synchronous runner; C3 `generate(messages, tools, constraint)` with prompt caching pinned off; C5 JSONL logger; R1 grammar with the free-text branch as default; CI on GitHub Actions	S4 5-task multi-turn check; C1 loaders for 20 BFCL multi-turn and 3 τ^2 airline tasks; non-thinking handler subclass; C6 rules v0 including the proposed missing-call class; second rater for the 58 Q4_0 labels	C11 Hydra config and registry with a run manifest (server settings, caching, thinking mode, file hashes); S8 export check; R2 Q8_0 variant and S9 switch prototype; uv lockfile and Dockerfile; cluster setup scripts
Oct 23 (M1 review)	Skeleton command runs end to end; report reproduced from a clean clone by two members	Toy statistics and planted-effect check in the report	Run registry shows every planned run spec once, with no duplicates
Oct 26–Nov 8 (to M2)	S1 and S10 on the cluster; Slurm array runner; CUDA-versus-CPU parity runs (FR-E17); concurrent-job throughput pilot (NFR-5)	Full C1 coverage; inclusion filter; S5 simulator choice and C4 cache proxy; codebook v0; OSF pre-registration draft for advisor review	BF16 GGUF conversions of all six models with KL checks; Singularity image; GCP L4 quota and simulator VM; QAD data preparation; S7 rerun under a Slurm walltime limit

Cluster onboarding

The setup scripts (SSH configuration, environment setup, and Slurm job templates) are being written this week (Table 37a), so that each step below is one command once accounts are active. Heavy work never runs on the login node, as the cluster rules require [93].

Table 37b. Cluster onboarding steps, in order.

Step	What	Owner	Done when
1	Log in over the VPN as each member; record `sinfo`, `scontrol show partition`, GPU types and memory per node, per-user limits	Ekant	Limits written to `docs/cluster.md`
2	Build llama.cpp v0.6.0 for CUDA (P100, compute capability 6.0) and CPU inside a compute-node job	Ekant	`llama-server --version` on both builds
3	Create the uv environments for the testbed and `bfcl-eval` in `$HOME`	Vritika	`bfcl --help` runs in a compute-node job
4	GPU smoke test: the S1 tool call on a P100 through `sbatch`	Ekant	Call parsed; timings and memory logged
5	S10: array of 8 jobs over 100 BFCL episodes, one killed by walltime and resumed; outbound access from a compute node (DEP-10)	Ekant; Vritika (DEP-10)	No missing or duplicate episodes
6	S7 rerun on an A100 or H100 node with a 10-minute Slurm limit	Yashashav	Resumed loss identical to the uninterrupted run

Status on 2026-10-26: step 1 is done (login on 2026-10-24, node inventory recorded). Steps 2 to 6 were blocked by the missing Slurm association until 2026-10-26, when it was fixed and test jobs ran on a P100, an A100 and an H100. Steps 2 and 4 ran the same day: llama.cpp was built for all three GPU types in a job, the S1 tool call passes on a P100, an A100 and an H100, and the GGUF files rebuilt on the cluster are byte-identical to the pilot's. One A100 node (cs002) fails to create a CUDA context and is excluded. Steps 3, 5 and 6 follow in Sprint 5 (to M2). The Slurm array and report job templates and the setup script are in the repository (slurm/, scripts/setup_env.sh).

Decisions requested from the advisor

- Confirm that the October pilot is development data and that R1 keeps its free-text branch, with the forced-call grammar kept only as a labelled ablation.
- Confirm the M1 fallback with a firm date: Version 1 moves the cluster exit criteria to M2 if access is not granted by Sprint 4; we propose 2026-10-19 as the cut-off, after which M1 runs with `local=true` on the Colab A100 and laptop CPU and the cluster criteria (two cluster accounts, S10) move to M2. **Resolved 2026-10-26: the Slurm association was fixed and test jobs ran; M1 ran on Colab under the fallback.**
- Agree to add a “missing call with a direct answer” class to codebook v0 (FR-E8), decided before the codebook is frozen at M2.
- Agree to restate RQ1’s motivation: the published 0.6B collapse did not reproduce in our harness, so the study measures where 4-bit damage appears (format, mode, prompting) rather than assuming it.
- Help to escalate cluster activation if accounts are still inactive on 2026-10-19. **Resolved 2026-10-26: the Slurm association was fixed and test jobs ran; M1 ran on Colab under the fallback.**
- **For the 2026-10-30 review: accept M1 as reached under local=true on Colab (cluster jobs were blocked until 2026-10-26), with the cluster criteria (two cluster accounts, S10) at M2.**
- **For the 2026-10-30 review: make step-level outcomes (C6 classes per step) the primary M2 outcome, since episode accuracy is 0% at 0.6B and at most 10% at 4B on these tasks.**

- **For the 2026-10-30 review: keep only τ^2 tasks whose gold actions write to the database, so that an agent that does nothing cannot pass.**
- **For the 2026-10-30 review: record the backend as a factor in every comparison and run the FR-E17 parity test with greedy decoding.**
- **For the 2026-10-30 review: choose the τ^2 simulator host for M2. The cluster's compute nodes cannot reach an outside VM, so either the simulator runs as a cluster job on an A100 or H100 (no cost; the DEP-3 fallback) and the agent jobs reach it inside the cluster, or the GCP L4 VM stays and τ^2 episodes run off the cluster. Recommended after measuring the cluster: the A100 job inside the cluster (Table 19a).**
- **For the 2026-10-30 review: confirm the dispersion convention (± 1 SD over the 4 sampled trials on every bar, 95% intervals for every effect, SD = 0 stated for greedy runs).**
- **For the 2026-10-30 review: confirm the mixed-effects analysis as a registered secondary analysis (lme4 in R) with the robustness rule in Chapter 7.**
- **For the 2026-10-30 review: confirm the two architecture views in Chapter 5 (software, Figure 1; infrastructure, Figure 1a).**
- **For the 2026-10-30 review: M4 (R1 and R2 evaluated) is planned for 2026-12-18, after the 295A demo on 2026-12-04; keep it there, or move it to 2026-12-11 so that 295A closes on a finished milestone (I and J2 have 37 days of float, so either date leaves the critical path unchanged).**
- **For the 2026-10-30 review: accept the vision-language extension (FR-O6) as optional 295B scope with a go/no-go at M6.**

Chapter 9. Project Schedule

We plan the project in two-week sprints that run Monday to Sunday, from Sprint 1 (starting 2026-08-31) to Sprint 16 (ending 2027-05-16), covering CMPE 295A (Fall 2026) and CMPE 295B (Spring 2027). The winter break (2026-12-21 to 2027-01-24) is one reduced-load sprint, W, used for unattended QAD training and as buffer. Each sprint ends with a review that doubles as a checkpoint. Eight milestones (M1 to M8) carry exit criteria, and the advisor reviews them at the biweekly meeting nearest each target date. Ownership is summarized in Table 40. The paper targets EuroMLSys 2027, whose call is not yet published. Its 2025 and 2026 deadlines fell seven to nine weeks before the workshop [117, 90], and EuroSys 2027 holds its workshops on 2027-04-19 [91], so we plan to submit on 2027-02-19 with a draft ready a week earlier. A later deadline gives the extended study more time; the backup is an ICLR 2027 workshop (suggested deadline 2027-02-01) [92].

Milestones

Table 38. Project milestones and exit criteria.

ID	Milestone	Target date	Exit criteria
M1	Walking skeleton runs end to end	2026-10-23	One command submits Qwen3-0.6B at BF16 and Q4_K_M on CUDA and Q4_0 on both backends on 23 BFCL and τ^2 tasks as cluster jobs; every step logged and classified; one minimal run of R1, R2, R3; toy stats and cost report; reproduced under two members' cluster accounts; cluster limits measured (S10)
M2	Full-precision references reproduced; analysis plan frozen	2026-11-06	BF16 references for all models and benchmarks with repeats, plus CUDA-versus-CPU parity runs; per-task results match the official evaluators; inclusion set stored; concurrent-job throughput measured; codebook v0 and pre-registration frozen on OSF before any main-study 4-bit run
M3	Damage map v1 (RQ1)	2026-12-04	Failure counts by type, step type, depth, and backend with CIs for every model, 4-bit format, and benchmark; codebook v1 frozen and 300 labeled pilot steps with κ report
M4	Repairs R1 and R2 evaluated	2026-12-18	Grammar, uniform, layer-selective, and step-selective variants run on the inclusion set; paired comparisons against the 4-bit baseline and the BF16 reference; repository v0.1, uv lockfile, Dockerfile, and Singularity recipe tagged (end of 295A)
M5	Repair R3 (QAD) and train-longer control evaluated	2027-01-29	All arms trained full-parameter on H100 or A100 nodes with identical data and steps; exports pass the logit check on both backends; evaluated on the real 4-bit path; RQ3 analysis covers all three repairs
M6	Cost profiling complete (RQ4)	2027-02-05	Memory, TTFT, tokens/s, extra tokens, and energy for every repair on one exclusive P100 node and CPU-backend latency on one compute node, plus R3 GPU-hours; comparison matrix v1
M7	Workshop paper submitted	2027-02-19 (planning date; backup ICLR 2027	Six-page paper on RQ1 to RQ4 submitted; repository v0.5 (R1 to R3, export checks, profiler) tagged for the artifact

ID	Milestone	Target date	Exit criteria
		workshop, 2027-02-01)	
M8	Open-source release v1.0 and final results	2027-05-14	Release v1.0 archived with a DOI; QAD and control checkpoints with model cards on the Hub; step-log data and damage-map report public; outside reproduction within CIs; Expo demo runs offline on a laptop

Table 38a. Scope by course (advisor review, 2026-10-16).

Course	**Dates**	**Milestones and work packages**	**Course deliverables**
CMPE 295A (Fall 2026)	**2026-08-31 to 2026-12-20**	**M1 walking skeleton (reached), M2 BF16 references, M3 damage map v1, M4 R1 and R2 evaluated; work packages A to K (with G2, H1–H2, J1–J2 and V) and L1 (QAD pipeline on the H100, the 0.6B run launched and its first checkpoint verified)**	**Workbook 1 (2026-09-30), Workbook 2 (2026-10-29), the 295A demo and report chapters 1 and 2 (2026-12-04)**
Winter break	**2026-12-21 to 2027-01-24**	**L2: QAD and control training for the remaining models (sprint W, runs unattended on the H100 with requeue)**	**None**
CMPE 295B (Spring 2027)	**2027-01-25 to 2027-05-14**	**M5 R3 evaluated, M6 costs complete, M7 workshop paper, M8 release v1.0; work packages N to U, and the optional vision-language extension (FR-O6) if it passes its go/no-go at M6**	**295B workbooks, final report, Expo demo**

Status at 2026-10-16. M1 targets 2026-10-23. Spikes S1 to S3, single-turn S4, S6 and S7 have run on Colab; S4 multi-turn, S5, S8 and S10 remain, along with the component code in Table 37a. If cluster accounts are not active by 2026-10-19 (the proposed cut-off), M1 is met under the `local=true` fallback and its cluster criteria move to M2. M2 to M8 keep their dates.

Status at 2026-10-26. M1 is reached under local=true on Colab except the clean-clone rerun by a second member, planned for 2026-10-28: one command, 253 specs, 252 episodes logged and classified (1,756 steps, none unclassified), one minimal run of R1, R2 and R3, toy statistics with the planted-effect check, and a cost table. The cluster criteria (two members' cluster accounts, S10) move to M2 (2026-11-06), as the Version 1 fallback rule provides; the cluster accepts jobs since 2026-10-26. Update 2026-10-27: the clean-clone check passed on the cluster, including an independent full rebuild from source (Table 37; the first skeleton episodes on the CoE HPC); the rerun under a second member's own account is still due.

If the paper is accepted, the workshop (2027-04-19) falls in Sprint 15.

Sprint Plan

Each member has one distinct line of work in every sprint. Shared work (literature review, paper, release) is split so that each person writes or delivers a separate part.

Table 39. Sprint plan with per-member tasks (Fall 2026 to Spring 2027; S1 to S10 refer to spikes in Chapter 8).

Sprint	Dates	Goal	Ekant Kapgate	Vritika Malhotra	Yashashav Devalapalli Kamalraj
1	Aug 31 –Sep 13	Scope and literature	Survey runtimes and 4-bit formats	Survey agent benchmarks, error-budget evidence	Survey repairs (mixed precision, QAD)
2	Sep 14–27	Environments	Laptop setup, llama.cpp CPU build; S2	Harness environments; S4	Repository layout, lockfile, Dockerfile
3	Sep 28–Oct 11	Cluster access, spikes, interfaces	S1, S3, S6 run on Colab by the backup owner (2026-10-08; development); cluster builds and S10 carried to Sprint 4	S4 single-turn run on Colab; C1 loaders and S5 carried to Sprint 4; C4 proxy to Sprint 5	S7 run on Colab; repository layout committed; S8, C11 and L4 quota carried to Sprint 4
4	Oct 12–25	Walking skeleton (M1)	Owner reruns of S1, S3, S6; C2 loop, C5 logger; C3 with caching off; R1 with free-text branch; C9 (NVML); CI; cluster onboarding and S10 when accounts are active	S4 multi-turn; C1 loaders; non-thinking handler subclass; C6 rules v0 with missing-call class; second rater for pilot labels; toy stats; plan draft	C11 config, registry and run manifest; S8 export; R2 Q8_0; S9 switch; Dockerfile and lockfile; cluster setup scripts
5	Oct 26–Nov 8	BF16 references (M2)	S1 and S10 on the cluster; CUDA-vs-CPU parity (FR-E17); Slurm array runner; throughput pilot	Full C1; inclusion filter; codebook v0; OSF pre-registration	All model conversions with `--pure`; KL checks
6	Nov 9–22	4-bit damage runs	R1 grammars per tool-call format; second labeler for the 300 pilot steps	Codebook v1; 300 labeled pilot steps, κ	R2 uniform, layer-selective; adjudicates label disagreements; QAD data
7	Nov 23 –Dec 6	Damage map (M3)	R1 runs; NVML protocol developed on a shared node	Damage map v1 (RQ1)	**R2 step-selective; QAD pipeline and data on 0.6B, short test runs (K)**
8	Dec 7–20	R1 and R2 done (M4)	R1 and R2 correctness runs finish; repository v0.1	R1, R2 analysis; RQ3 metrics	**R2 runs; launch QAD and control on 0.6B on the H100, first checkpoint verified (L1);**

Sprint	Dates	Goal	Ekant Kapgate	Vritika Malhotra	Yashashav Devalapalli Kamalraj
					Singularity recipe
W	Dec 21–Jan 24	Reduced: training, buffer	R2 switch-cost profiling; cost section outline	Analysis automation; results outline	**QAD and control on 1.2B and 1.7B; exports (L2)**
9	Jan 25–Feb 7	R3 and costs (M5, M6)	RQ4 cost blocks A, B, C on exclusive nodes (DEP-5)	R3 analysis; comparison matrix	R3 eval on both backends; GPU-hours from Slurm accounting
10	Feb 8–21	Paper (M7)	Paper: testbed, cost; v0.5 (R1, C9)	Paper: RQ1, RQ3 results, stats	Paper: repairs, QAD, related work; v0.5 (R2, R3)
11	Feb 22–Mar 7	Extended study	Testbed docs; Expo demo design; FR-D4 audit (50 episodes); AD-5 grammar compiler package	Step-log dataset v1 with the arXiv preprint; replications; FR-D4 audit (50 episodes); AD-9 hypothesis outcome table	Vendor-QAD audit on LFM2.5-2.6B (FR-D1, FR-D2); AD-4 decision guide draft
12	Mar 8–21	Extended study	Costs for added models; FR-D3 on Q4_K_M	Damage map v2; FR-D6 CPU formats; FR-D7 long-context	Combined repair (R1 with R3); FR-O5 on A100
13	Mar 22–Apr 4	Camera-ready or revision	Revise testbed, cost sections	Revise results sections	Revise repair sections
14	Apr 5–18	Release preparation	Expo demo build	Damage-map report (static site); FR-D8 step-log explorer	Checkpoints and model cards on Hub; AD-4 decision guide final
15	Apr 19–May 2	Final results, outside rerun	Support outside rerun from cache; renew cluster access	Final statistics and figures	Support outside rerun from scratch; registry freeze
16	May 3–16	Release v1.0 (M8)	Expo demo final; release CI	Report from release data; dataset cards	Release v1.0 with DOI

Sprint 4 as it happened (2026-10-12 to 2026-10-25): the interfaces were frozen, every M1 component was drafted on a review branch, and the skeleton ran on Colab (A100, CPU and L4 runtimes) instead of the cluster. Sprint 5 (2026-10-26 to 2026-11-08) adds cluster onboarding steps 2 to 6, S1 and S10 on the cluster, S5 (simulator choice), the clean-clone rerun and the merge of the component branches.

Gantt Chart



Figure 6. Project Gantt chart, Fall 2026 to Spring 2027. Bars are colored by owner and labeled with the owner's initials; gray rows are shared work whose per-member split is stated in the row label; diamonds mark milestones M1 to M8; the dashed line marks today, 2026-10-26; the shaded band is the winter break (reduced sprint W). ***Updated for Workbook 2: today line, M1 reached (open diamond), the cluster slip (hatched) and the course deadlines from Canvas (triangles).***

PERT Chart: Dependencies and Critical Path

Figure 6a shows the work of both semesters as a network of work packages (A to U) and milestones (M1 to M8) with finish-to-start dependencies; Table 39a lists each package's owner, planned dates and predecessors. Float comes from the critical-path method: durations are the planned calendar days, a forward pass from 2026-08-31 gives the earliest dates, a backward pass gives the latest, and float is the difference. The critical path is A → B → C → D → M1 → E → F → G → M2 → K → L1 → L2 → O → M6 → P → M7 → Q → R → T → M8: E, F and G all feed M2 with the same duration, and after M2 the path runs through the QAD pipeline and the H100 training (K, L1, L2), not through the damage runs or R1 and R2. The damage runs (H1) have 51 days of float and R1 and R2 (I, J2) 37, so a slip there does

not move M5 to M8; a slip in K, L1 or L2 moves everything after it, which is why the H100 training starts before the winter break. The earliest finish leaves 19 days of buffer before M8 (2027-05-14). M1 was reached on 2026-10-26 under the Colab fallback, three days after plan and inside that buffer, with only the clean-clone rerun by a second member still due.

Work is shared evenly: Workbook 2 splits the damage runs (H1, Slurm arrays; H2, labels and damage map) and R2 (J1, variant construction; J2, runs and analysis), adds the τ² simulator on the cluster (G2) and the statistics automation (V), and moves the R3 evaluation (N) to the analysis owner. Each member now leads work packages of about the same length: Ekant 104 days, Vritika 105 days and Yashashav 103 days (packages shared by all three are not counted); every package also has reviewers from the other two.

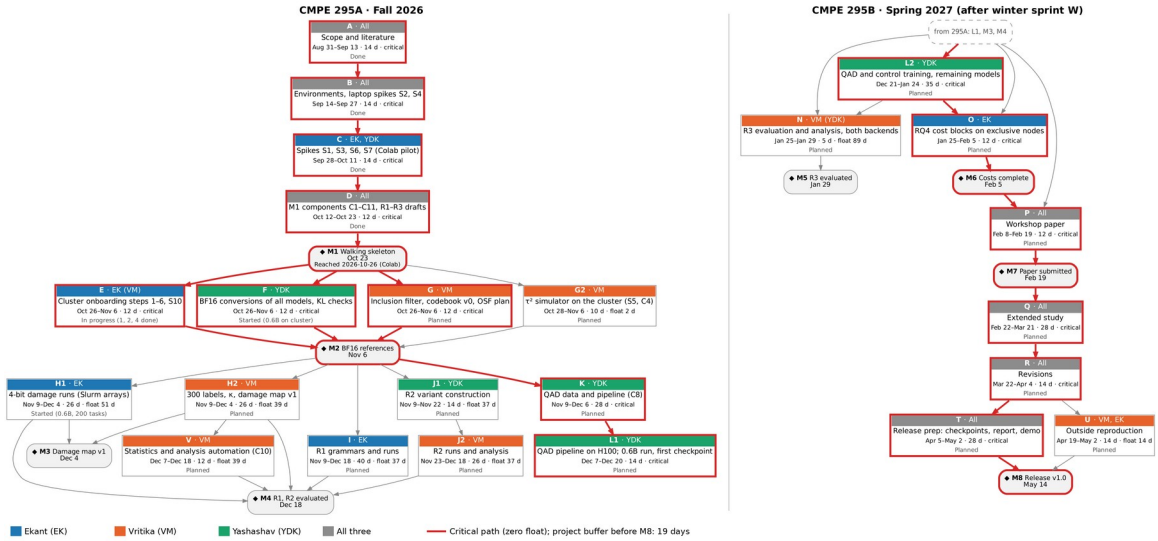


Figure 6a. PERT network for CMPE 295A and 295B. Each box gives the work package, owner, planned dates, duration, float and status on 2026-10-26; red boxes and arrows are the critical path (zero float); the dashed box on the right stands for the 295A packages that 295B work depends on.

Table 39a. PERT work packages: owner, planned dates, dependencies, float and status (2026-10-26).

ID	Work package	Owner	Planned	Depends on	Float (days)	Status
A	Scope and literature	All	Aug 31, 2026 – Sep 13, 2026	—	0 (critical)	Done
B	Environments, laptop spikes S2, S4	All	Sep 14, 2026 – Sep 27, 2026	A	0 (critical)	Done
C	Spikes S1, S3, S6, S7 (Colab pilot)	EK, YDK	Sep 28, 2026 – Oct 11, 2026	B	0 (critical)	Done
D	M1 components C1–C11, R1–	All	Oct 12, 2026 – Oct 23, 2026	C	0 (critical)	Done

	R3 drafts					
E	**Cluster onboarding steps 1–6, S10**	**EK (VM)**	**Oct 26, 2026 – Nov 6, 2026**	**M1**	**0 (critical)**	**In progress (1, 2, 4 done)**
F	**BF16 conversions of all models, KL checks**	**YDK**	**Oct 26, 2026 – Nov 6, 2026**	**M1**	**0 (critical)**	**Started (0.6B on cluster)**
G	**Inclusion filter, codebook v0, OSF plan**	**VM**	**Oct 26, 2026 – Nov 6, 2026**	**M1**	**0 (critical)**	**Planned**
G2	**τ^2 simulator on the cluster (S5, C4)**	**VM**	**Oct 28, 2026 – Nov 6, 2026**	**M1**	**2**	**Planned**
H1	**4-bit damage runs (Slurm arrays)**	**EK**	**Nov 9, 2026 – Dec 4, 2026**	**M2**	**51**	**Started (0.6B, 200 tasks)**
H2	**300 labels, κ, damage map v1**	**VM**	**Nov 9, 2026 – Dec 4, 2026**	**M2**	**39**	**Planned**
I	**R1 grammars and runs**	**EK**	**Nov 9, 2026 – Dec 18, 2026**	**M2**	**37**	**Planned**
J1	**R2 variant construction**	**YDK**	**Nov 9, 2026 – Nov 22, 2026**	**M2**	**37**	**Planned**
J2	**R2 runs and analysis**	**VM**	**Nov 23, 2026 – Dec 18, 2026**	**J1**	**37**	**Planned**
K	**QAD data and pipeline (C8)**	**YDK**	**Nov 9, 2026 – Dec 6, 2026**	**M2**	**0 (critical)**	**Planned**
V	**Statistics and analysis automation (C10)**	**VM**	**Dec 7, 2026 – Dec 18, 2026**	**H2**	**39**	**Planned**
L1	**QAD pipeline on H100; 0.6B run, first checkpoint**	**YDK**	**Dec 7, 2026 – Dec 20, 2026**	**K**	**0 (critical)**	**Planned**
L2	**QAD and control training, remaining models**	**YDK**	**Dec 21, 2026 – Jan 24, 2027**	**L1**	**0 (critical)**	**Planned**
N	**R3 evaluation and analysis, both backends**	**VM (YDK)**	**Jan 25, 2027 – Jan 29, 2027**	**L2, M3**	**89**	**Planned**
O	**RQ4 cost blocks on**	**EK**	**Jan 25, 2027 –**	**M4, L2**	**0 (critical)**	**Planned**

	exclusive nodes		Feb 5, 2027			
P	**Workshop paper**	**All**	**Feb 8, 2027 – Feb 19, 2027**	**M3, M6**	**0 (critical)**	**Planned**
Q	**Extended study**	**All**	**Feb 22, 2027 – Mar 21, 2027**	**M7**	**0 (critical)**	**Planned**
R	**Revisions**	**All**	**Mar 22, 2027 – Apr 4, 2027**	**Q**	**0 (critical)**	**Planned**
T	**Release prep: checkpoints, report, demo**	**All**	**Apr 5, 2027 – May 2, 2027**	**R**	**0 (critical)**	**Planned**
U	**Outside reproduction**	**VM, EK**	**Apr 19, 2027 – May 2, 2027**	**R**	**14**	**Planned**
M1	**Walking skeleton**	**milestone**	**Oct 23, 2026**	**D**	**0 (critical)**	**Reached 2026-10-26 (Colab)**
M2	**BF16 references**	**milestone**	**Nov 6, 2026**	**E, F, G, G2**	**0 (critical)**	**Planned**
M3	**Damage map v1**	**milestone**	**Dec 4, 2026**	**H1, H2**	**63**	**Planned**
M4	**R1, R2 evaluated**	**milestone**	**Dec 18, 2026**	**H1, H2, I, J2, V**	**37**	**Planned**
M5	**R3 evaluated**	**milestone**	**Jan 29, 2027**	**N**	**89**	**Planned**
M6	**Costs complete**	**milestone**	**Feb 5, 2027**	**O**	**0 (critical)**	**Planned**
M7	**Paper submitted**	**milestone**	**Feb 19, 2027**	**P**	**0 (critical)**	**Planned**
M8	**Release v1.0**	**milestone**	**May 14, 2027**	**T, U**	**0 (critical)**	**Planned**

Task Assignment Summary

Release items follow the checklist in Chapter 7. Every component has one primary owner, who is accountable for it, and one backup, who reviews its pull requests, reruns its spike, and can take it over if needed. Each member backs up four components, and nobody backs up their own work.

Table 40. Ownership, deliverables, and backups per member.

Member	Primary ownership	Key deliverables	Backup for
Ekant Kapgate	C2 Episode Runner, C3 Inference Backends, C5 Step Logger, R1 grammar decoding, C9 Cost Profiler	Testbed core on llama.cpp's CUDA and CPU backends; cluster builds and Slurm runner; repository v0.1 and CI; R1 grammars; RQ4 cost matrix; Expo demo; paper sections on the testbed and costs	C1, C4, R2, C8
Vritika	C1 Benchmark	Adapters and inclusion set; self-hosted simulator and	C2, C5,

Member	Primary ownership	Key deliverables	Backup for
Malhotra	Adapter, C4 Simulated-User Cache, C6 Failure Classifier, C10 Analysis & Stats	cache; failure codebook and labeled sample; OSF pre-registration; damage map (RQ1); RQ3 analysis; step-log datasets; damage-map report; paper results and statistics	C9, C11
Yashashav Devalapalli Kamalraj	R2 added bits, R3 with C8 QAD Training Pipeline, model conversion and 4-bit checks, C11 Config & Registry	Uniform, layer-selective, and step-selective variants; QAD and control checkpoints on the Hub; verified GGUF exports; uv lockfile, Dockerfile and Singularity image, run registry and Slurm templates; release v1.0 with DOI; paper sections on repairs and related work	C3, C6, R1, C10

Schedule Risks and Buffers

- **Cluster activation (live in October 2026).** The advisor filed the request on 2026-10-11 and the course instructor collected student IDs on 2026-10-12. Buffers: the spikes already ran on a Colab A100; the setup scripts make onboarding a single session; and the Version 1 rule moves the cluster criteria of M1 to M2 (proposed cut-off 2026-10-19). **Accounts became active on 2026-10-24 and jobs ran from 2026-10-26 (bullet below).**
- **Realized and resolved (2026-10-24 to 2026-10-26): accounts were active but jobs were blocked by a missing Slurm association, fixed on 2026-10-26. Mitigation used: the Colab fallback; M1 ran there with local=true, and the cluster criteria move to M2.**
- **Cluster access or queues slip.** Only faculty can request accounts, the GPU time limit is stated as both 7 days and 48 hours, and the two H100 and two A100 nodes are shared college-wide [93]. Buffers: S10 measures the real limits before M1; evaluation runs on the less contended P100 and compute nodes; training checkpoints every 30 minutes and requeues; the QAD pipeline is tested in Sprint 7 (K), the 0.6B training starts on 2026-12-07 (L1), and sprint W trains the rest (L2). Without the cluster, the skeleton runs on laptops and R3 falls back to free Colab and Kaggle GPUs for Qwen3-0.6B. **Update 2026-10-26: access works, and the limits are measured (gpuqs 2 d 1 h, gpuqm 7 d 1 h).**
- **Simulator host.** Only τ^2-bench-verified needs a simulated user [52]. Buffers: the C4 cache replays finished episodes free, the runner retries on preemption, the VM stops between batches, a cluster A100 is the fallback host, and BFCL alone carries RQ1 to RQ3. **Update: the in-cluster A100 job becomes the primary host and the VM the fallback (Table 19a).**
- **The field moves fast.** A paper on one of these repairs could appear before our submission. Buffers: each member checks new work in their own area weekly (Ekant: runtimes and constrained decoding; Vritika: benchmarks and evaluation; Yashashav: precision allocation and QAD), and Sprints 11 and 12 leave room to add a newly published repair.
- **Scope growth from the vision-language extension (FR-O6). Buffers: no work before M5, a one-day spike, a go/no-go at M6, and BFCL and τ^2 alone carry RQ1 to RQ4 if it is dropped.**
- **Finals and an unknown deadline.** Sprints 7 and 8 overlap Thanksgiving and finals, when the cluster is busiest. Buffers: long runs are resumable arrays launched in Sprint 7; the cost blocks (about 60 exclusive node-hours) are scheduled for Sprint 9, after finals; and planning to the earliest plausible paper date turns a later deadline into slack.
- **Machine time.** The R1 and R2 runs between M3 and M4 need about 300 of the 1,150 machine-hours. At 8 concurrent jobs that is about 40 wall-clock hours; if the M2 pilot shows fewer than 4, the pre-registered reduction ladder (NFR-5) applies before any milestone moves.
- **Built-in slack.** Each sprint is planned at about 80% of capacity. Sprint W and Sprints 11 and 12 absorb slips without moving M7 or M8.